

INFO 490 MG:
A. I.
Web-Application Development

MOHIT GUPTA

Week 5 (Part-2):
Authentication (Django + Google)
+
External Database

Index

1. Part-1 (Done)

- a. Deployment (Done)

- b. Test API + Charts (Done)

2. Part-2

- a. Django auth

- b. Code Cleanup

- c. External Database

- d. Google OAuth

1. User Authentication (Django's Auth.)

- a. Internal Users
- b. External Users

Users



- Developers
- Employees

(Current sub-ection)



- Non-employees

(client sign-up using google
OAuth: upcoming sub-section)

1.a Internal Users

1. The layman approach:

Manually register the internal users on the website using admin panel



127.0.0.1:8000/admin/



1 Log in | Django site admin

Django administration 

Username:

Password:

Log in

127.0.0.1

Site administration | Django site admin

Django administration

WELCOME, MOHITG2. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups

[+ Add](#) [Change](#)

Users

[+ Add](#) [Change](#)

STUDENTS

Enrollments

[+ Add](#) [Change](#)

Sections

[+ Add](#) [Change](#)

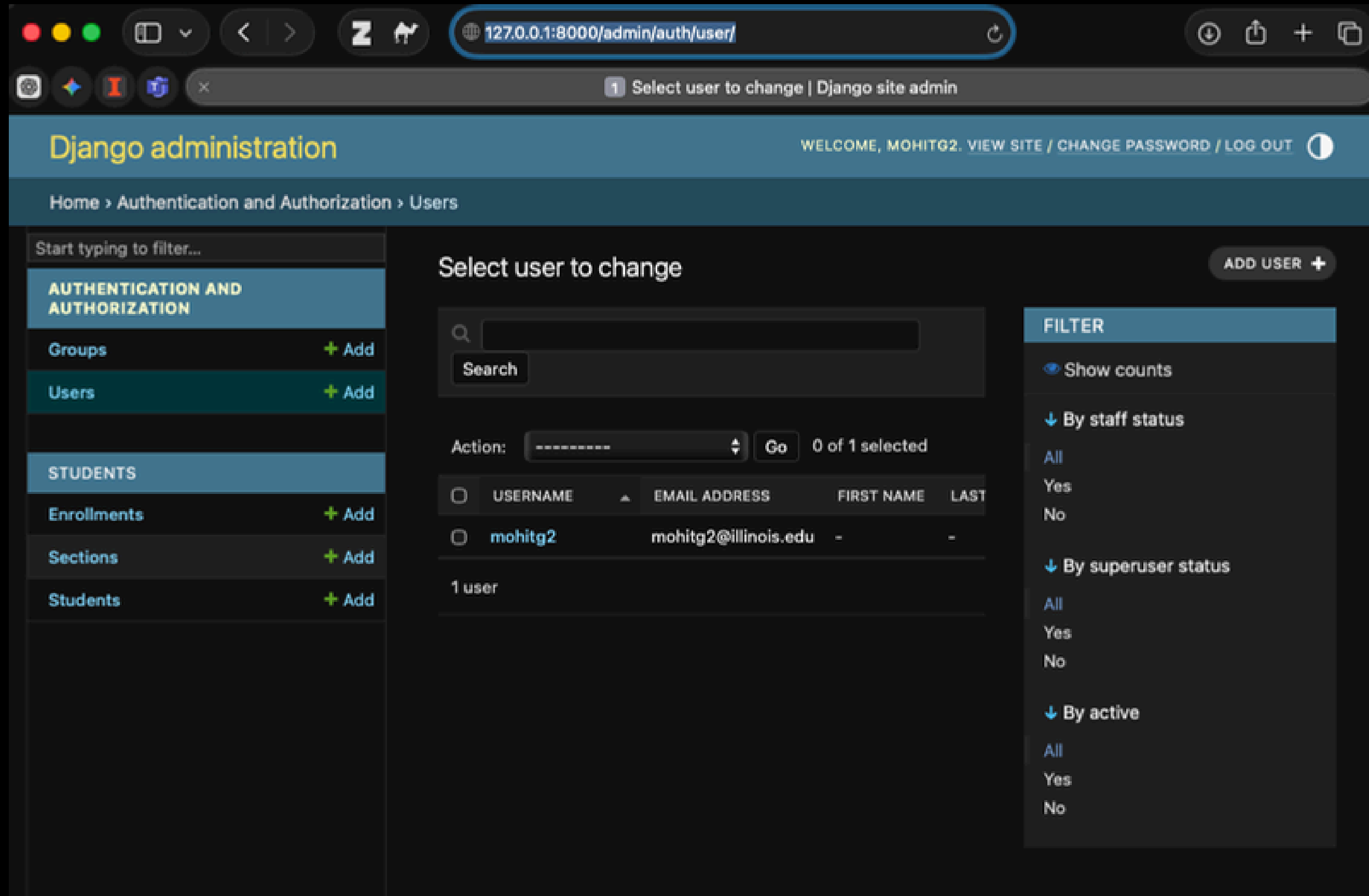
Students

[+ Add](#) [Change](#)

Recent actions

My actions

None available



127.0.0.1

mohitg2 | Change user | Django site admin

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

STUDENTS

Enrollments + Add

Sections + Add

Students + Add

Change user

mohitg2

Username:

mohitg2

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

algorithm: pbkdf2_sha256 iterations: 1000000 salt: deZ3Fe***** hash: mL840*****

Reset password

Raw passwords are not stored, so there is no way to see the user's password.

Personal info

First name:

Last name:

Email address:

mohitg2@illinois.edu

Permissions

☒ Active

Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☒ Staff status

Designates whether the user can log into this admin site.

☒ Superuser status

Designates that this user has all permissions without explicitly assigning them.

Choose all groups

Remove all groups

The groups this user belongs to. A user will get all permissions granted to each of their groups. Hold down "Control", or "Command" on a Mac, to select more than one.

User permissions:

Available user permissions

Choose user permissions by selecting them and then select the "Choose" arrow button.

Filter

Administration | log entry | Can add log entry
Administration | log entry | Can change log entry
Administration | log entry | Can delete log entry
Administration | log entry | Can view log entry
Authentication and Authorization | group | Can add group
Authentication and Authorization | group | Can change group
Authentication and Authorization | group | Can delete group
Authentication and Authorization | group | Can view group
Authentication and Authorization | permission | Can add permission
Authentication and Authorization | permission | Can change permission
Authentication and Authorization | permission | Can delete permission
Authentication and Authorization | permission | Can view permission
Authentication and Authorization | user | Can add user

Choose

Remove

Chosen user permissions

Remove user permissions by selecting them and then select the "Remove" arrow button.

Filter

Remove

Choose all user permissions

Remove all user permissions

Specific permissions for this user. Hold down "Control", or "Command" on a Mac, to select more than one.

Important dates

Last login:

Date: 2025-10-17 Today

Time: 02:57:44 Now

Note: You are 5 hours behind server time.

Date joined:

Date: 2025-09-16 Today

Time: 14:47:13 Now

Note: You are 5 hours behind server time.

SAVE

Save and add another

Save and continue editing

Delete

2. Better approach

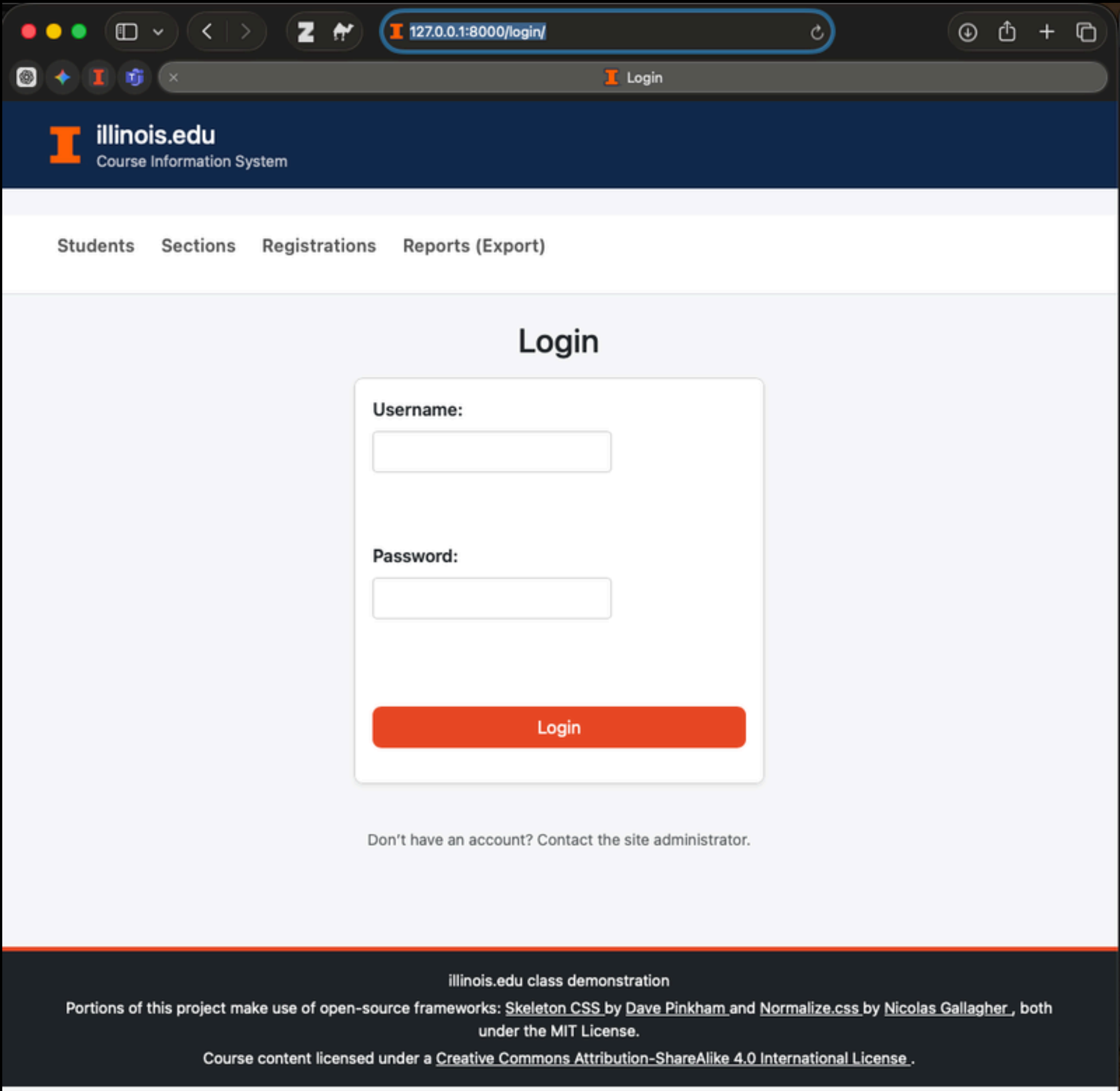
- **Create a login page.**
- **Let internal users enter the website using it.**

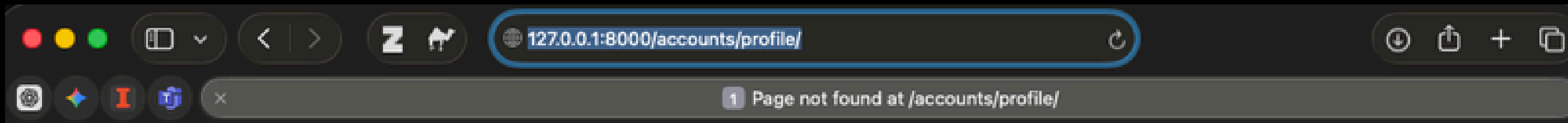
Let's begin script integration.

```
23  # new addition/changes: LoginView
24  path(route: 'login/',
25  <>    LoginView.as_view(template_name='students/login.html'),
26      name='login_urlpattern'),
27
28  ]
29
```

```
urls.py  file1_1.py  <> student_list.html  <> login.html x
1  <!-- Paste code here: templates/students/login.html -->
2  <!-- Writing new code for displaying the aggregated data. -->
3
4  {% extends "students/base.html" %}
5
6  {% block title %}
7      Login
8  {% endblock %}
9
10 {% block content %}
11
12     <h2>Login</h2>
13     <form method="POST">
14         {% csrf_token %}
15         {{ form.as_p }}
16         <button type="submit">Login</button>
17     </form>
18
19 {% endblock %}
```

Output.





Page not found (404)

Request Method: GET

Request URL: http://127.0.0.1:8000/accounts/profile/

Using the URLconf defined in `illinois.urls`, Django tried these URL patterns, in this order:

- 1.
2. `admin/`
3. `student/` [name='student-list-url']
4. `section/` [name='section-list-url']
5. `enrollment/` [name='enrollment-list-url']
6. `student/<int:primary_key>/` [name='student-detail-url']
7. `charts/sections.png` [name='chart-sections']
8. `feedback/` [name='feedback-url']
9. `function-add-student/` [name='function-add-student-url']
10. `class-add-student/` [name='class-add-student-url']
11. `api/ping-httpresponse/` [name='api-ping-httpresponse']
12. `api/ping-json/` [name='api-ping-json']
13. `api/function-students/` [name='api-students']
14. `api/sections/students/` [name='api-students-per-section']
15. `api/sections/enrollments/` [name='api-enrollments-per-section']
16. `api/class-students/` [name='api-students']
17. `charts/enrollments/` [name='enrollments-chart-page']
18. `charts/enrollments.png` [name='enrollments-chart-png']
19. `api/weather/` [name='api-weather']
20. `reports/` [name='export-reports-url']
21. `export/students.csv` [name='export-students-csv']
22. `export/students.json` [name='export-students-json']
23. `login/` [name='login_urlpattern']

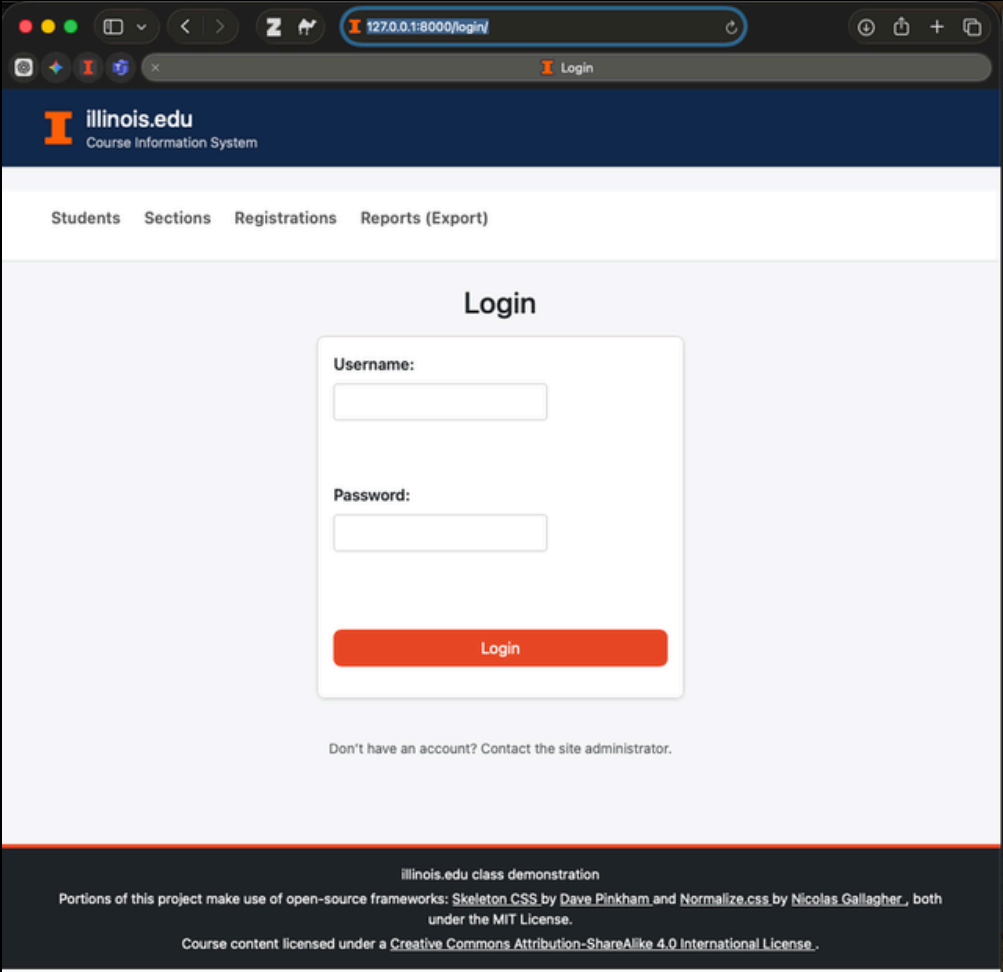
The current path, `accounts/profile/`, didn't match any of these.

You're seeing this error because you have `DEBUG = True` in your Django settings file. Change that to `False`, and Django will display a standard 404 page.

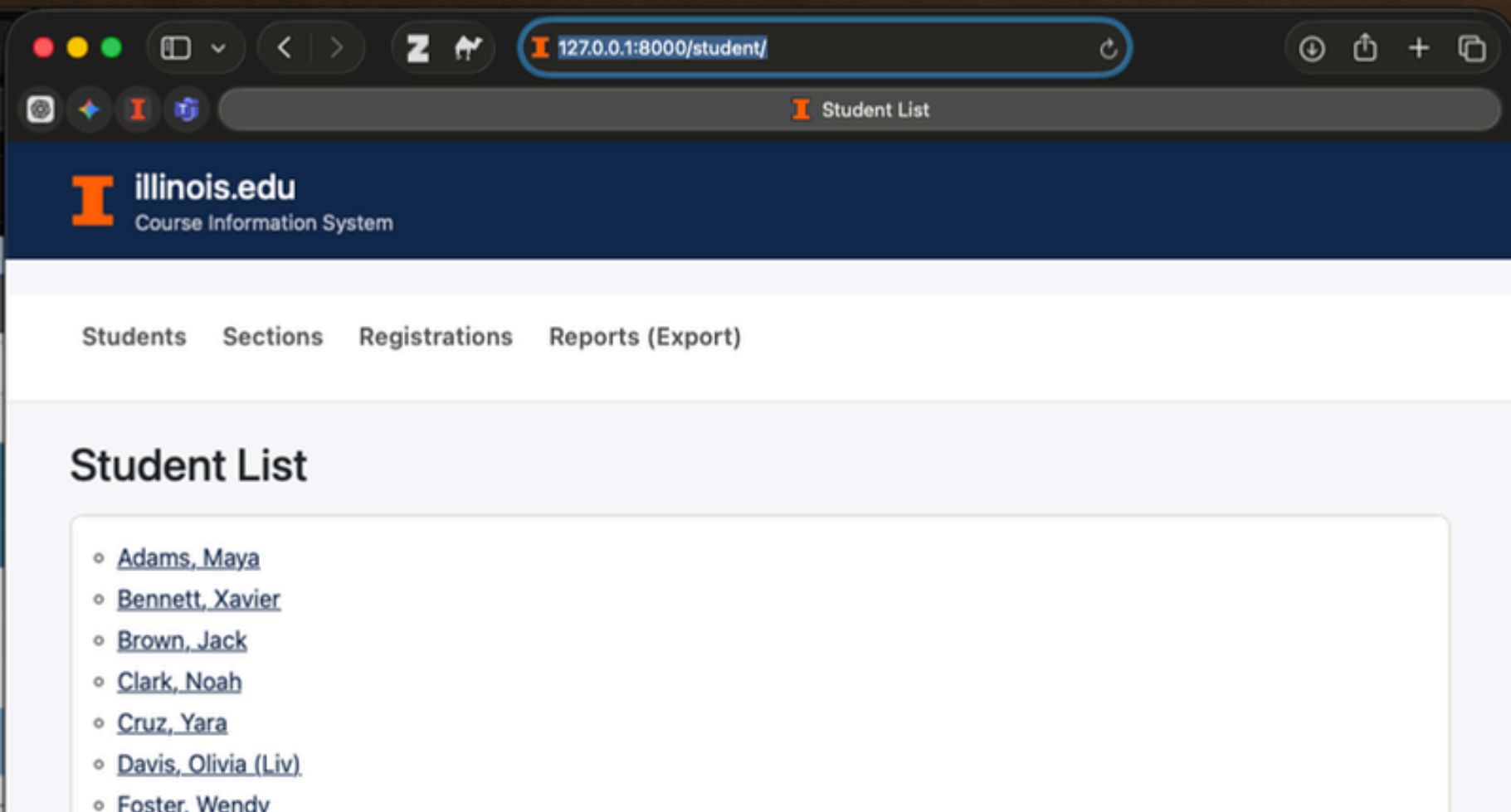
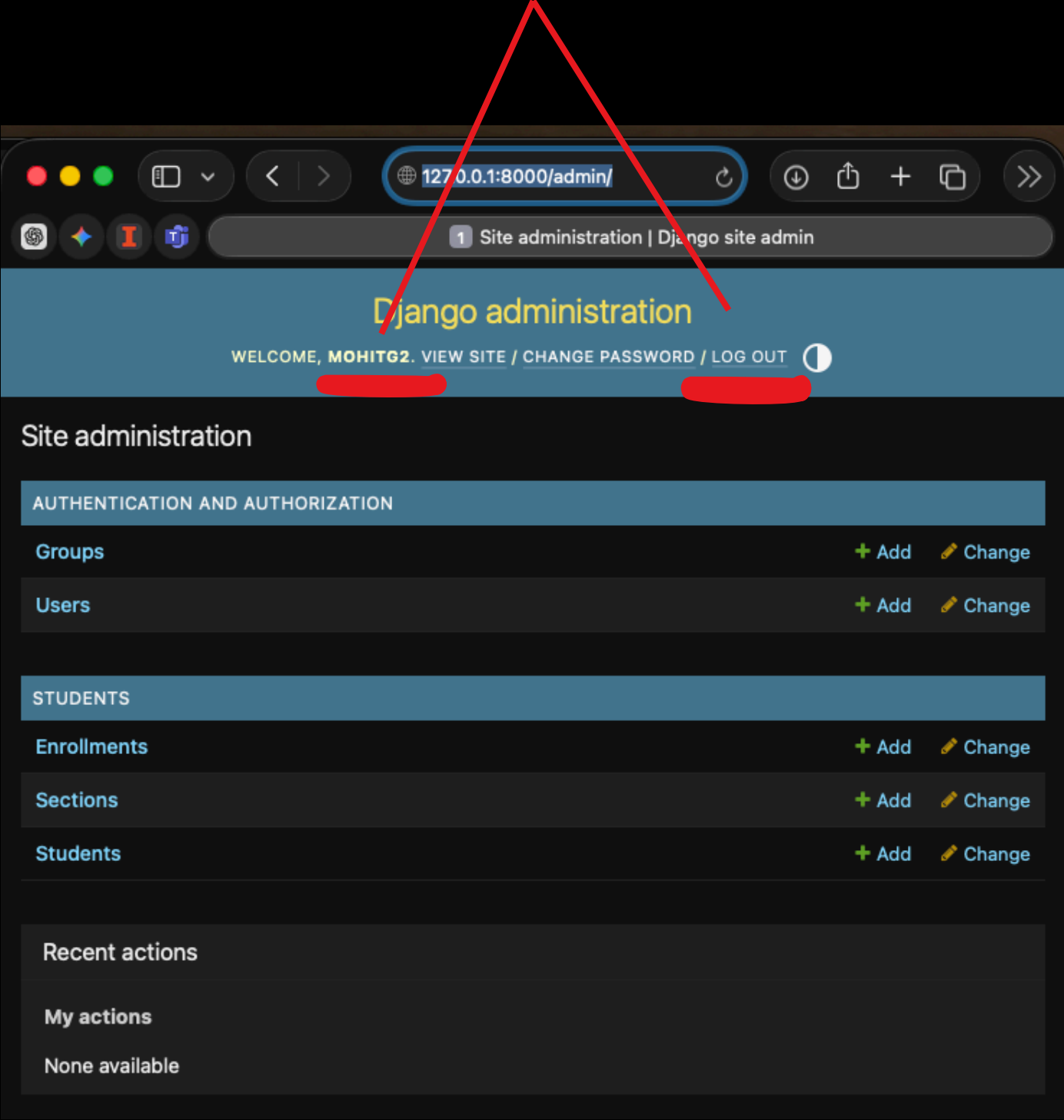
- Our login worked as expected but given that we did not set any home page after login, django is trying to take us to “**accounts/profile**”
- And as we did not set it up yet, it shows all the other available links.

Let's change it in settings.py

Currently, to login, you need to use this link.



And to logout, you've to go to admin panel to log-out. Let's change this



**So, let's build a URL path for
Logout feature.**

illinois/urls.py

```
26
27     path(route: 'logout/',
28           LogoutView.as_view(),
29           name='logout_urlpattern'),
30 ]
31
```

base.html

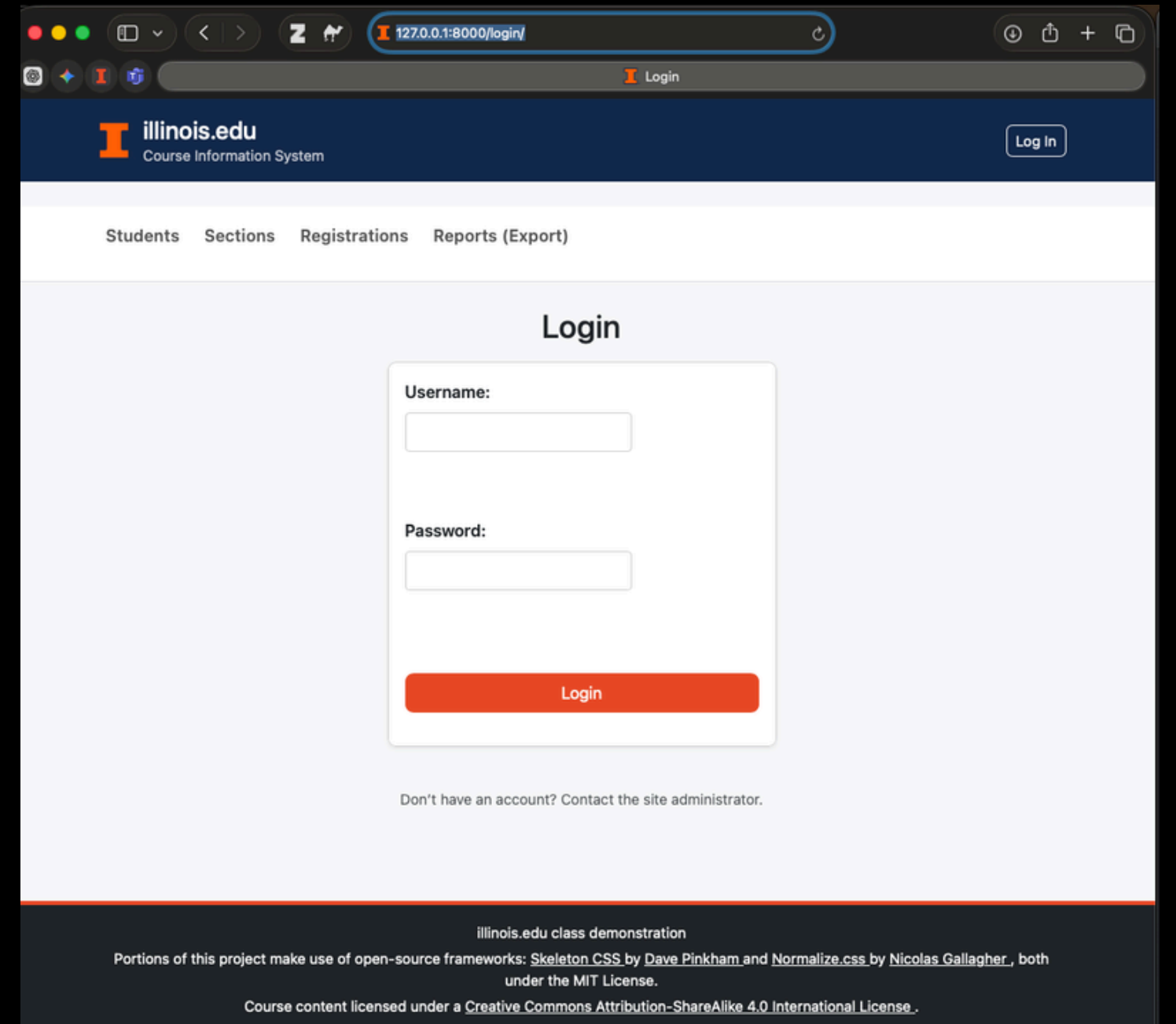
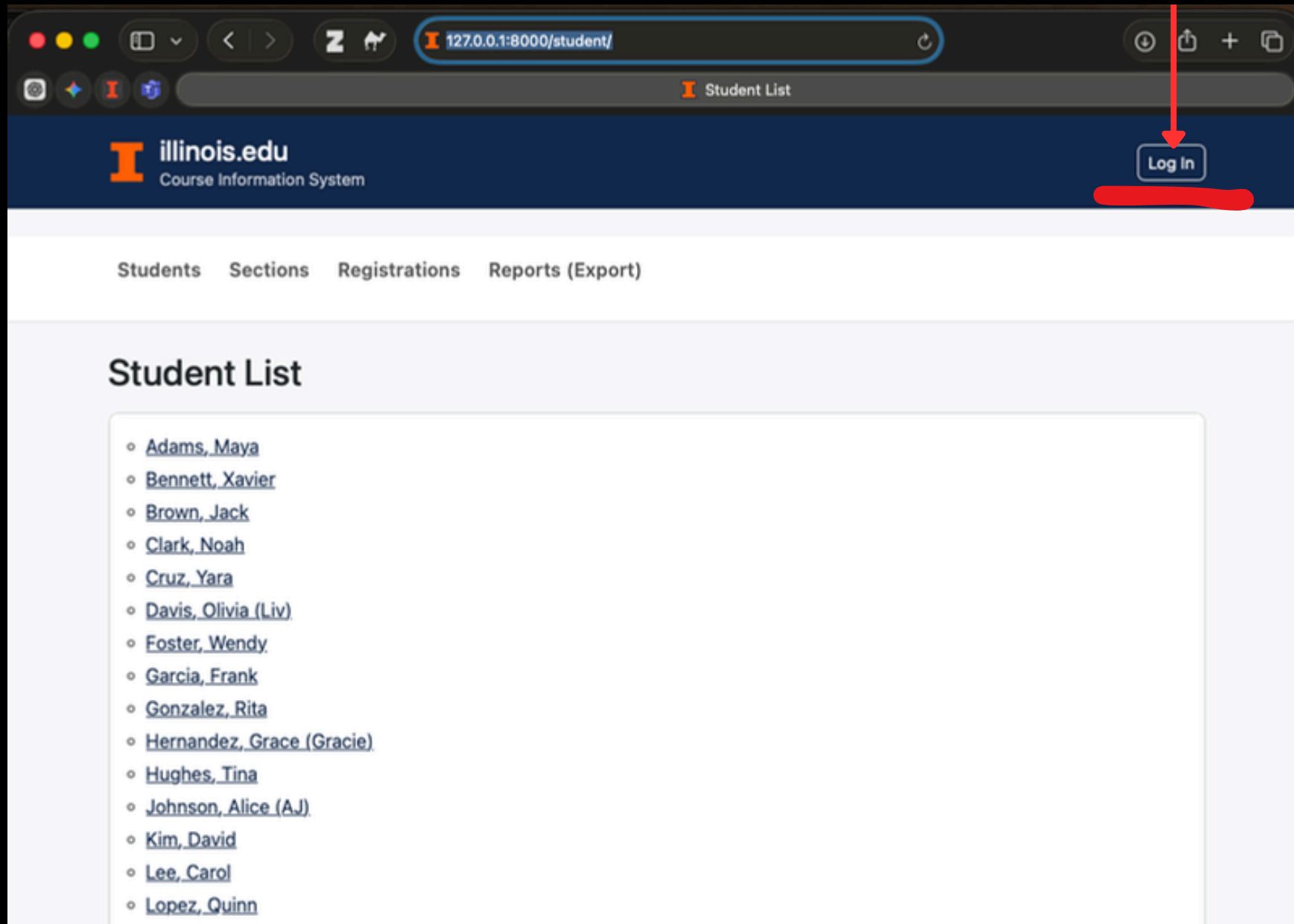
```
45 <!-- Right: Login / Logout links -->
46 <ul class="nav ms-auto">
47     {% if user.is_authenticated %}
48
49         <form action="{% url 'logout_urlpattern' %}" method="post" class="ms-auto">
50
51             <!--IMPORTANT: If you do not include csrf token tag here,
52             your logout will not work as intended. -->
53             {% csrf_token %}
54
55             <button class="btn btn-sm btn-outline-light" type="submit">
56                 Log Out, {{ user.get_username }}
57             </button>
58
59         </form>
60     {% else %}
61
62         <a class="btn btn-sm btn-outline-light ms-auto"
63           href="{% url 'login_urlpattern' %}">Log In</a>
64
65     {% endif %}
66 </ul>
```

settings.py

```
267
268 # New addition/changes:
269 # This will set where the user will go after logging out.
270 # I'm entering the "URL NAME PATTERN" for the 'login/' url here = 'login_urlpattern'
271 # You can locate it in the students/urls.py
272 LOGOUT_REDIRECT_URL = 'login_urlpattern'
273
274
275
```

settings.py

new
login/logout
button



**But we still need to make sure that
the data through navigation pane
is only visible after the login.**

This sets the “after Login-Logout” behavior

```
265    
266  # New addition/changes: (already done in previous steps)  
267  # This will set where the user will go after logging out.  
268  # I'm entering the "URL NAME PATTERN" for the 'login/' url here = 'login_urlpattern'  
269  # You can locate it in the students/urls.py  
270  
271  # This will also set a universal/global behavior that where to take user  
272  # after they login/logout  
273  LOGIN_REDIRECT_URL = 'student-list-url'  
274  LOGOUT_REDIRECT_URL = 'login_urlpattern'  
275
```

views.py

Added:

- LoginRequiredMixin : For classes

```
144
145 # New class
146 <img alt="enrollment icon" data-bbox="85 315 135 330"/> /enrollment/
147 class EnrollmentListView(LoginRequiredMixin, ListView):
148     model = Enrollment
149     context_object_name = "enrollment_rows_for_looping"
150
151     # new addition/changes:
152     # Purpose of redirect_field_name:
153     # redirect_field_name is the URL parameter name Django uses to store
154     # the "next" page the user wanted to visit before being redirected to the login page.
155     # So, once the login is successful, the user will be redirected to where they
156     # originally wanted to go.
157
158     # Now, if an anonymous user tries to open:
159     ### http://127.0.0.1:8000/enrollment/
160
161     # Django will automatically redirect them to:
162     ### http://127.0.0.1:8000/login/?next=/enrollment/
163     redirect_field_name = 'next' # ?next=/requested/path
```

views.py

Added:

- redirect_field_name : For functions

```
421 <img alt="api icon" data-bbox="565 465 580 485"/> /api/ping-json/
422 @login_required(login_url='login_urlpattern')
423 def api_ping_jsonresponse(request):
424     return JsonResponse({"ok": True})
```


base.html

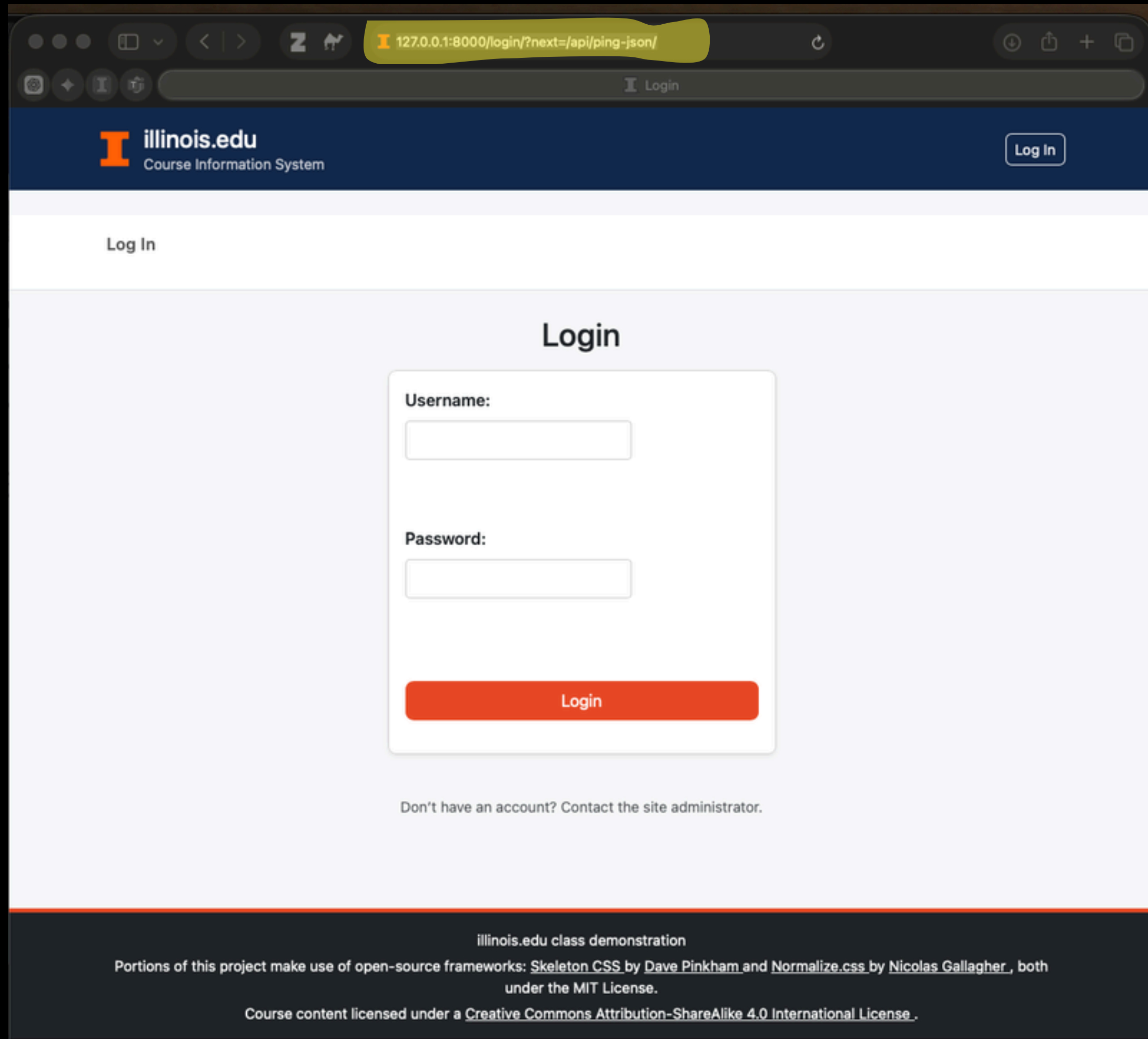
```
70 <!-- Navigation -->
71 <!-- New addition/changes: user login required to view data -->
72 <nav class="navbar navbar-expand-lg navbar-light bg-white border-bottom mb-4" aria-label="Primary">
73     <div class="container">
74
75         <ul class="navbar-nav gap-2">
76             {% if user.is_authenticated %}
77                 <li class="nav-item"><a class="nav-link" href="{% url 'student-list-url' %}">Students</a></li>
78                 <li class="nav-item"><a class="nav-link" href="{% url 'section-list-url' %}">Sections</a></li>
79                 <li class="nav-item"><a class="nav-link" href="{% url 'enrollment-list-url' %}">Registrations</a></li>
80                 <li class="nav-item"><a class="nav-link" href="{% url 'export-reports-url' %}">Reports (Export)</a></li>
81             {% else %}
82                 <li class="nav-item"><a class="nav-link" href="{% url 'login_urlpattern' %}">Log In</a></li>
83             {% endif %}
84         </ul>
```

previously when no
user authentication
required

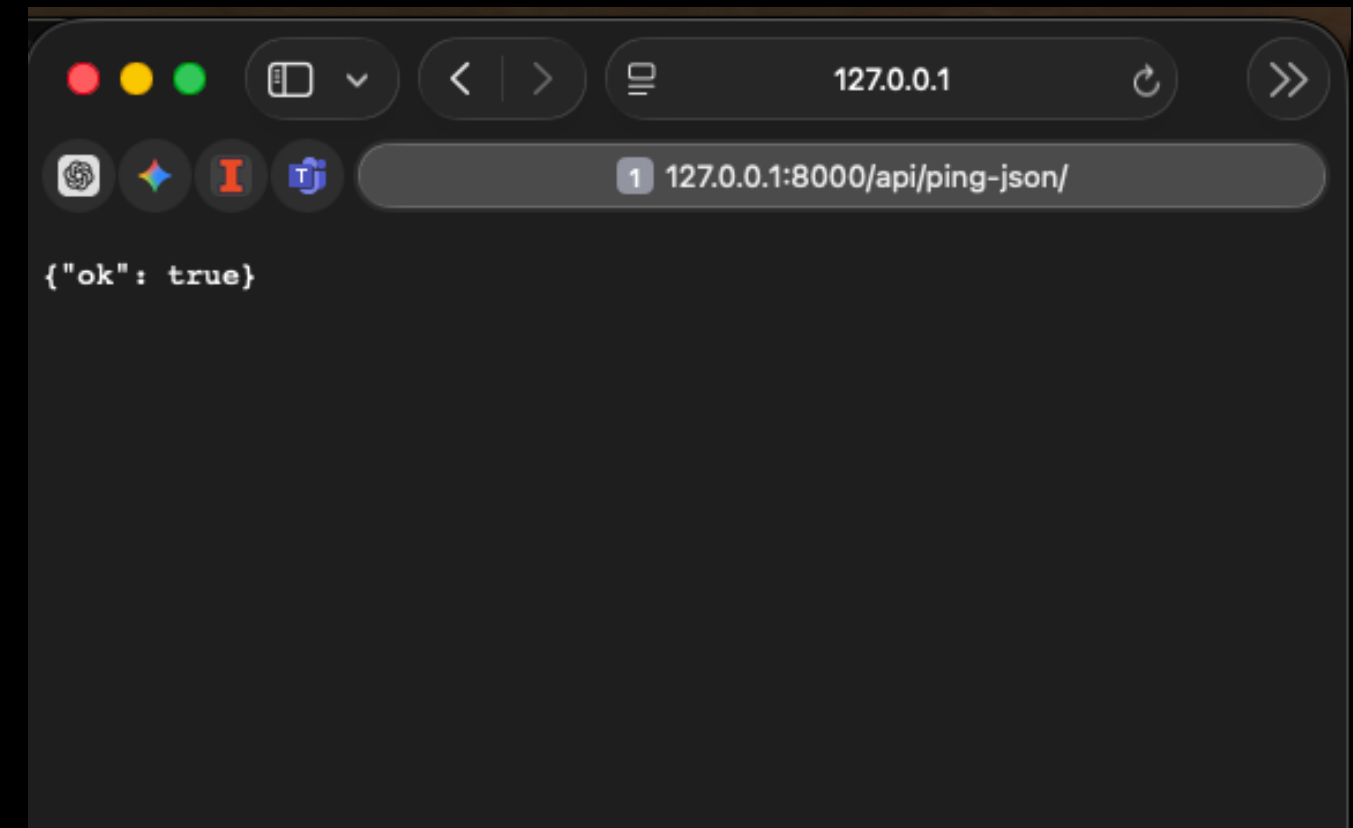
A screenshot of a web browser showing the 'illinois.edu Course Information System' login page. The browser's address bar shows '127.0.0.1:8000/login/'. The page has a dark blue header with the university logo and a 'Log In' button. Below the header is a light blue navigation bar with links: 'Students', 'Sections', 'Registrations', and 'Reports (Export)'. The main content area is light blue and features a 'Login' form with 'Username:' and 'Password:' labels, input fields, and a red 'Login' button. At the bottom of the form area, it says 'Don't have an account? Contact the site administrator.' The footer is dark blue and contains text about the project's use of open-source frameworks and licensing.

Now, when user
authentication is
required

A screenshot of the same web browser showing the 'illinois.edu Course Information System' login page. The browser's address bar shows '127.0.0.1:8000/login/'. The page has a dark blue header with the university logo and a 'Log In' button. Below the header is a light blue navigation bar with a link: 'Log In'. The main content area is light blue and features a 'Login' form with 'Username:' and 'Password:' labels, input fields, and a red 'Login' button. At the bottom of the form area, it says 'Don't have an account? Contact the site administrator.' The footer is dark blue and contains text about the project's use of open-source frameworks and licensing.



Now after the user authentication, you can access the api data



1.b External Users

Final Output

Login

Username:

Password:

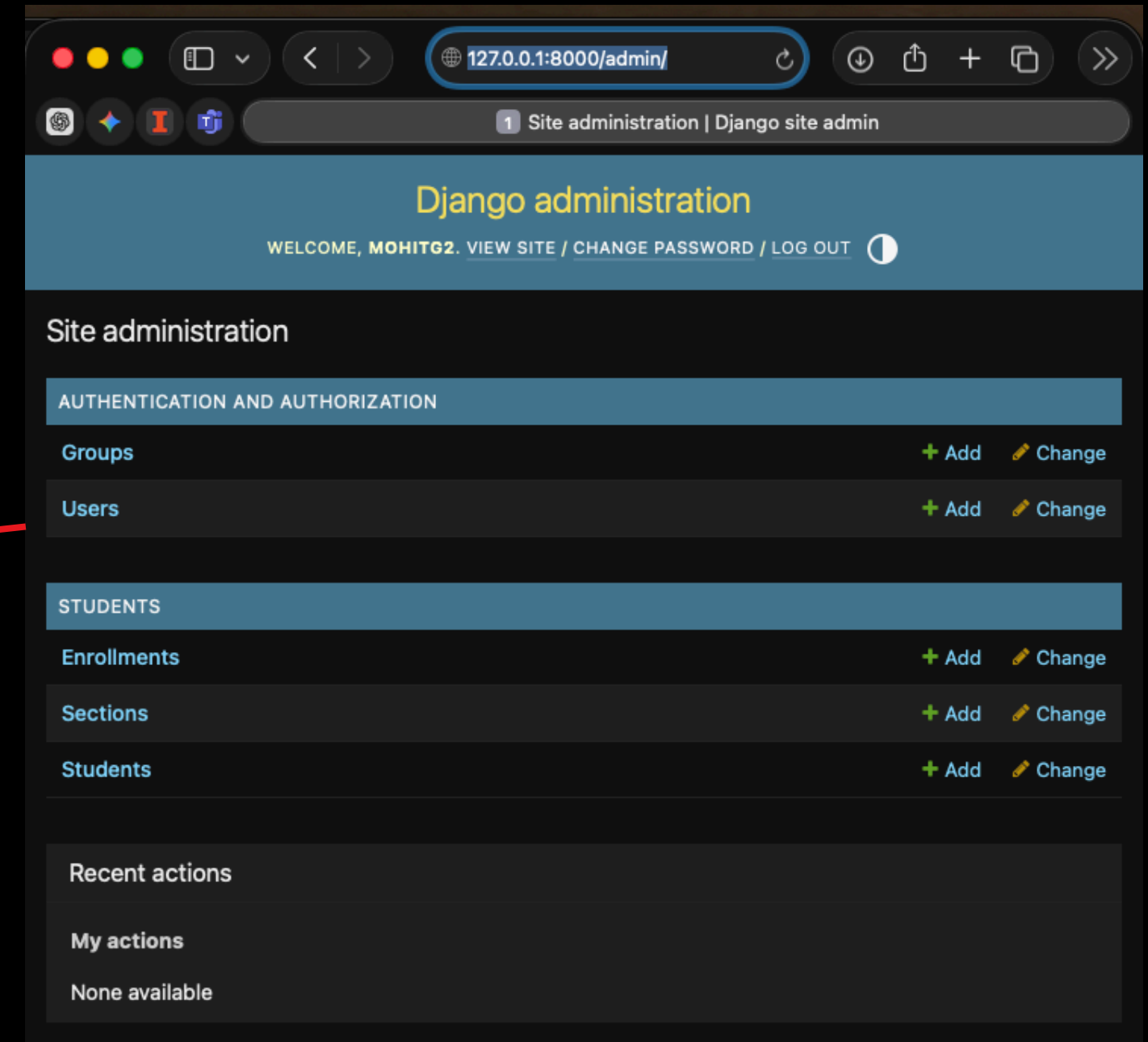
Login

Don't have an account? [Sign Up](#)

Registering the public users

**(With no access to the admin
page)**

To learn the basics, we are going to store the users in the in-built “Users” model that Django gave us.



Now we are going to restrict access of public to this page.

Django administration

WELCOME, MOHITG2. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Authentication and Authorization > Users > mohitg2

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups

+ Add

Users

+ Add

STUDENTS

Enrollments

+ Add

Sections

+ Add

Students

+ Add

Change user

mohitg2

Username:

mohitg2

HISTORY

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

algorithm: pbkdf2_sha256 iterations: 1000000 salt: deZ3Fa***** hash: mLlH0*****

Reset password

Raw passwords are not stored, so there is no way to see the user's password.

Personal info

First name:

Last name:

Email address:

mohitg2@illinois.edu

Permissions

☒ Active

Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☒ Staff status

Designates whether the user can log into this admin site.

☒ Superuser status

Designates that this user has all permissions without explicitly assigning them.

Groups:

Available groups

Choose groups by selecting them and then select the "Choose" arrow button.

Q

Filter

Choose all groups

Chosen groups

Remove groups by selecting them and then select the "Remove" arrow button.

Q

Filter

Remove all groups

➡

⬅

The groups this user belongs to. A user will get all permissions granted to each of their groups. Hold down "Control", or "Command"

31

List of fields the USER table has:

Field name	Description	Example
id	Auto Primary Key	1
password	Hashed Password	pbkdf2_.....
last_login	Datetime of Last Login	2025-10-26 20:22:00
is_superuser	Boolean	False
username	Chosen username	mohitg2
first_name	Optional	Mohit
last_name	Optional	Gupta
email	Email address entered via the form	mohitg2@illinois.edu
is_staff	For admin access	False
is_active	True by Default	True
date_joined	Timestamp	2025-10-26 20:22:00

```

class StudentSignUpForm(UserCreationForm):
    """
    This form creates a new Django user (NOT a Student row).
    Fields:
    - username
    - password1
    - password2 (confirm password)
    We're also adding email to store contact info.
    """

    #
    email = forms.EmailField(
        required=True,
        help_text="We'll use this email if we need to contact you."
    )

    class Meta:

        # This is a pre-existing table in Django where we are already storing our internal user's data
        model = User
        fields = ["username", "email", "password1", "password2"]

```

- Creating a form to save signup data in the default User table.
- **We are also enabling mandatory requirement for the “email” field and a text for support message.**

```

from .forms_auth import StudentSignUpForm

def signup_view(request): 2 usages (2 dynamic)
    """
    ~~~~~
    Show a sign-up form (GET),
    create an account (POST),
    then log the user in automatically.
    """
    if request.method == "POST":
        form = StudentSignUpForm(request.POST)
        if form.is_valid():
            # 1. Create the user in auth_user table
            new_user = form.save()

            # 2. Log them in immediately (no separate login step)
            login(request, new_user)

            # 3. Send them somewhere in the app
            return redirect("student-list-url")
        else:
            form = StudentSignUpForm()

    return render(request, template_name: "students/signup.html", context: {"form": form})

```

- We'll import StudentSignUpForm from the **forms_auth.py** file that we just created and validate the response.
- The intended file is **views.py**

- Lastly, setting the changes in the urls.py
- And adding **LOGIN_URL** variable in the **settings.py**
- Just by using **LOGIN_URL**, you can avoid multiple usages of `login_url = 'login_urlpattern'` (like in the screenshot)

```

54 # new addition/changes: auth routes
55 <> path(route: "login/", LoginView.as_view(template_name="students/login.html"), name="login_urlpattern"),
56
57 path(route: "logout/", LogoutView.as_view(next_page="login_urlpattern"), name="logout_urlpattern"),
58
59 path(route: "signup/", views.signup_view, name="signup_urlpattern"),
60
61
62 ]

```

```

266 # New addition/changes: This automatically sets where to send people if they are not logged in
267 # settings.py
268 LOGIN_URL = "login_urlpattern" # where to send people if not logged in
269

```

```

279 # views.py
280 class StudentListView(LoginRequiredMixin, ListView):
281     model = Student
282     template_name = "students/student_list.html"
283     login_url = 'login_urlpattern' # required if LOGIN_URL not in settings
284     redirect_field_name = 'next' # optional (Django uses 'next' by default)

```

- Now creating a new signup form in **students/signup.html**
- And adding the signup button in the existing **students/login.html**

**We are now just three steps away
from a deployable project.**

Activity time!

2. Code Clean-up

**As we are nearing the end of
Phase 1, let's clean-up the code.**

Now, login is mandatory.
Without it, you will not be able to get in.

127.0.0.1:8000/login/?next=/student/

illinois.edu Course Information System

Log In

Log In

Login

Username:

Password:

Login

Don't have an account? [Sign Up](#)

illinois.edu class demonstration

Portions of this project make use of open-source frameworks: Skeleton CSS by Dave Pinkham and Normalize.css by Nicolas Gallagher , both under the MIT License.

Course content licensed under a Creative Commons Attribution-ShareAlike 4.0 International License .

illinois.edu Course Information System

Log Out, mohitg2

Data Forms APIs & Export Charts Reports

Student List

Adams, Maya

Bennett, Xavier

Brown, Jack

Clark, Noah

Cruz, Yara

Previous Page 1 of 6 Next

None Search

result

Section Statistics

Students per Section		Students per Term	
Section	Students	Term	Students
INFO-390-MG — Web App Dev I	2	Fall 2025	2
INFO-490-MG — Web App Dev II	1	Fall 2026	24
INFO-590-MG — Web App Dev III	24	Spring 2026	1

Bar Chart: Students per Section

Students per Section

Section	Students
INFO-390-MG	2
INFO-490-MG	1
INFO-590-MG	24

3. External Database

1.

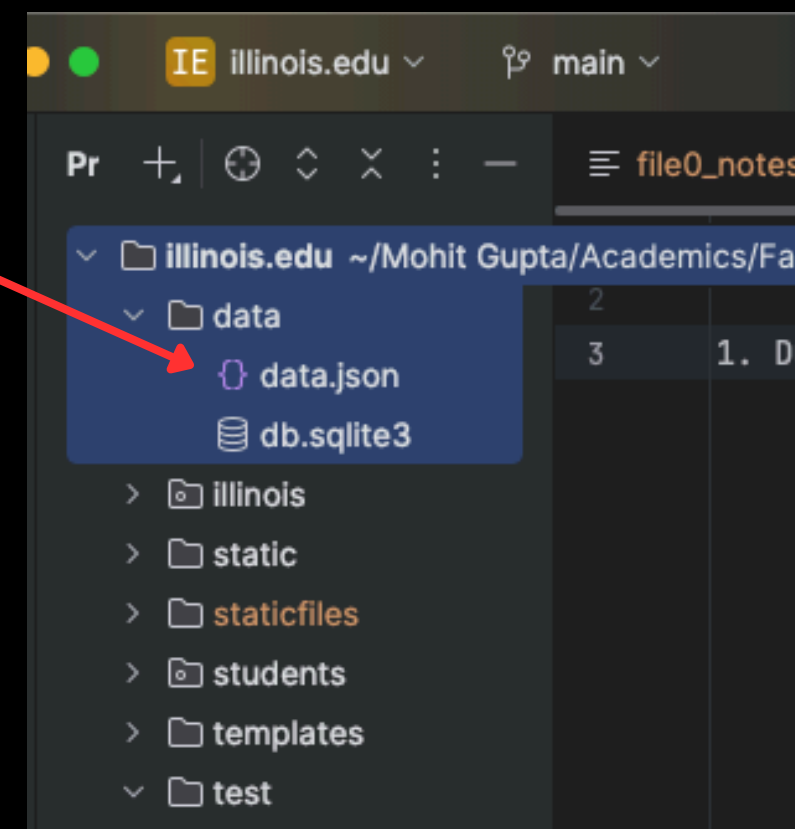
Preserving the current
dataset from
the local machine

Let's save our
current local DB to
json so that our
data is preserved

Run on local machine:

```
python manage.py dumpdata \
  --exclude auth.permission \
  --exclude contenttypes \
  --indent 2 > data/data.json --settings=illinois.settings.development
```

- You will then see the new data here.



IE illinois.edu main

Current File

Pr +, -

data.json x

illinois.edu ~/Mohit Gupta

data

data.json

db.sqlite3

illinois

static

staticfiles

students

templates

test

venv

.gitignore

manage.py

External Libraries

Scratches and Consoles

Django Structure

dj Settings development.py

Admin

Models

Views

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

```
[
{
  "model": "auth.user",
  "pk": 1,
  "fields": {
    "password": "pbkdf2_sha256$1000000$de23FaDHQC5U95e2qy2tV4$mLLH0JHXQExggGHVZ6adyy9LUiI00Z4dZjE6EQPIgs=",
    "last_login": "2025-10-30T01:43:49.519Z",
    "is_superuser": true,
    "username": "mohitg2",
    "first_name": "",
    "last_name": "",
    "email": "mohitg2@illinois.edu",
    "is_staff": true,
    "is_active": true,
    "date_joined": "2025-09-16T14:47:13.261Z",
    "groups": [],
    "user_permissions": []
  }
},
{
  "model": "auth.user",
  "pk": 2,
  "fields": {
    "password": "pbkdf2_sha256$1000000$YHyd1JisXfYFwDXkJLUlvY$lhW+rI0Dj88naXwq8efUHcng6u6b/QQES4Hh1adIVzQ=",
    "last_login": "2025-10-27T04:42:28.836Z",
    "is_superuser": false,
    "username": "mohitg27",
    "first_name": "",
    "last_name": "",
    "email": "mohitg27@illinois.edu",
    "is_staff": false,
    "is_active": true,
    "date_joined": "2025-10-27T04:42:28.556Z",
    "groups": [],
    "user_permissions": []
  }
},
]
```

- **But why datadump? Why not copy paste it from local to production server?**

1. db.sqlite3:

- If your project uses db.sqlite3, you can use the same local db and push it on github and then use it on the production server. The approach is absolutely fine.

2. External database (MySQL or Postgres):

- But when you need to upload data from db.sqlite3 to an external database like Postgres or MySQL, you'll need to reload the data from the local dump to the external server.

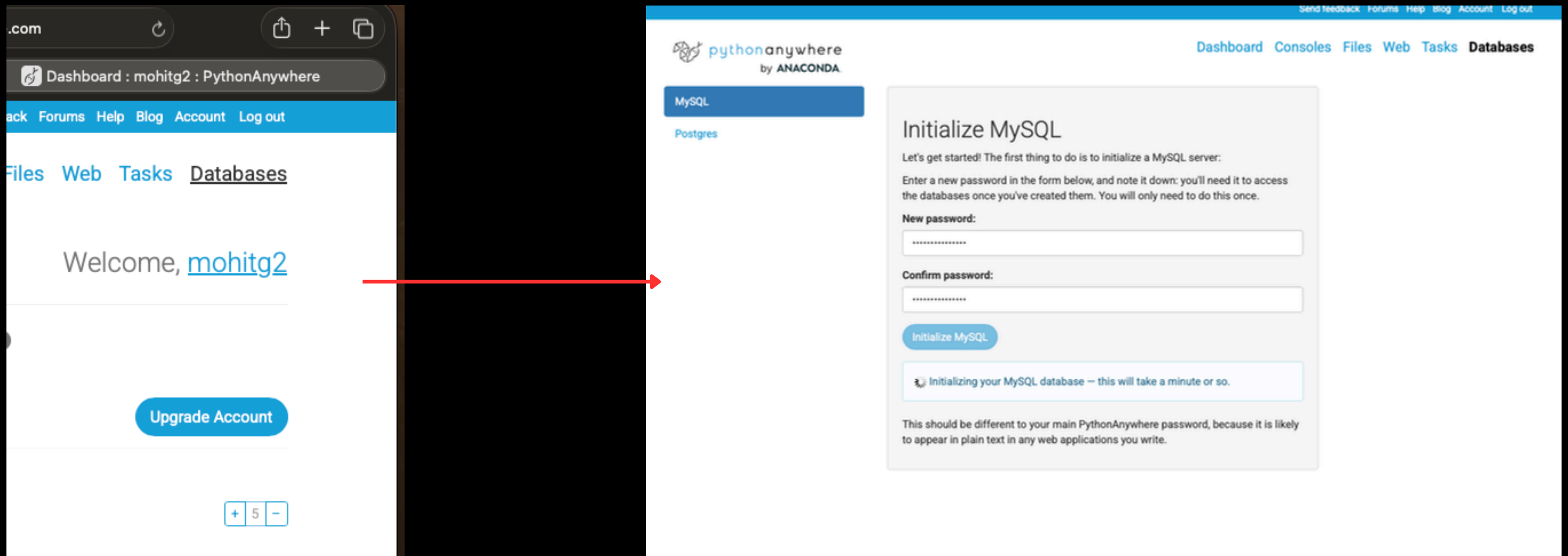
When you want to load the data on any server of your choice, just run:

- `python manage.py loaddata data/data.json`

3.a Creating & Connecting to External DB

We'll be using
PythonAnywhere's services
here, but you can use any
service to create database
as well.

- On the dashboard, click Databases.
- Create a password to access your MySQL server (this is not a database yet).
- I used: **grangerlibrary**



- Now let's create a database
- You will also see the details you'll need to connect to your MySQL database in Django

- Create the following db:

- MySQL DB Name:
 - info390mg-mysql-database

Send feedback Forums Help Blog Account Log out

pythonanywhere
by ANACONDA

Dashboard Consoles Files Web Tasks **Databases**

Your MySQL password has been changed. ✕

MySQL
Postgres

MySQL settings

Connecting:
Use these settings in your web applications.

Database host address: mohitg2.mysql.pythonanywhere-services.com
Username: mohitg2

Since you have a free account, you will only be able to connect to your database using code running inside PythonAnywhere.

Your databases:
Click a database's name to start a MySQL console logged in to it.

Name
mohitg2\$default
mohitg2\$info390mg-mysql-database

Create a database

Your database names always start with your username + "\$". There's no need to type that prefix in below, though: PythonAnywhere will automatically add it.

Database name:

Create

- **You we have a MySQL server on PythonAnywhere with:**
- **Host:**
 - `mohitg2.mysql.pythonanywhere-services.com`
- **User:**
 - `mohitg2`
- **Database for our project:**
 - `mohitg2$info390mg-mysql-database`
- **Password for MySQL Server:**
 - `grangerlibrary`

**Now we've the
MySQL server
initialized,
and we've created
a new
MySQL Database.**

**We're now ready for Django-
side development!**

3.c Installing the required packages

Install:

- `pip install mysqlclient`
- `pip install pymysql`
- `pip freeze > illinois/settings/requirements.txt`

```
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % pip install pymysql
Collecting pymysql
  Downloading pymysql-1.1.2-py3-none-any.whl.metadata (4.3 kB)
  Downloading pymysql-1.1.2-py3-none-any.whl (45 kB)
Installing collected packages: pymysql
Successfully installed pymysql-1.1.2

(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % pip freeze > illinois/settings/requirements.txt
```

- In case you're getting an error installing mysqlclient, use this step

```
1 # Where: illinois/__init__.py
2
3  Why:
4 ## if you are not able to install: pip install mysqlclient, then use this:
5 ## That line makes Django think "oh, this is MySQLdb" even though it's PyMySQL.
6
7 # Add:
8
9 import pymysql
10 pymysql.install_as_MySQLdb()
11
12
```

- **Now plugging the database credentials**

```
production.py x
1  # Paste in: illinois/settings/production.py
2  
3  # We will first import everything from base.py (our new settings.py file tha
4  from .base import *
5
6  DEBUG = False
7
8  # Enter your PythonAnywhere account here in the allowed hosts.
9  # Replace it with your name:
10 ALLOWED_HOSTS = ['localhost', '127.0.0.1', 'mohitg2.pythonanywhere.com']
11
12 DATABASES = {
13     "default": {
14         "ENGINE": "django.db.backends.mysql",
15         "NAME": "mohitg2$info390mg-mysql-database",
16         "USER": "mohitg2",
17         "PASSWORD": "graingerlibrary",
18         "HOST": "mohitg2.mysql.pythonanywhere-services.com",
19         "PORT": "3306",
20     }
21 }
22
23
```


3.d Migrations & Loading Existing Data

**Now that we've connected to
the new DB, we need to:**

1. transfer the migrations to our new database,

and

2. fill it up with existing data.

- Now, when you try running any command on the production server, it is going to give you an error.

- That error is only specific for PythonAnywhere.
 - PythonAnywhere only let's you access the database from within the production server.

```
Terminal Local x + v
(mysenv-django) mohitgupta@nonits-MacBook-Pro-2267 illinois.edu % python manage.py runserver --settings=illinois.settings.production
Performing system checks...

System check identified no issues (0 silenced).
Exception in thread django-main-thread:
Traceback (most recent call last):
  File "/Users/mohitgupta/anaconda3/envs/mysenv-django/lib/python3.13/site-packages/pymysql/connections.py", line 661, in connect
    sock = socket.create_connection(
          (self.host, self.port), self.connect_timeout, **kwargs
    )
  File "/Users/mohitgupta/anaconda3/envs/mysenv-django/lib/python3.13/socket.py", line 864, in create_connection
    raise exceptions[0]
  File "/Users/mohitgupta/anaconda3/envs/mysenv-django/lib/python3.13/socket.py", line 849, in create_connection
    sock.connect(sa)
    ^^^^^^^^^^^^^^^^^
TimeoutError: timed out

During handling of the above exception, another exception occurred:

lib/python3.13/site-packages/django/db/backends/base/base.py", line 279, in ensure
lib/python3.13/site-packages/django/utils/asyncio.py", line 26, in inner
lib/python3.13/site-packages/django/db/backends/base/base.py", line 256, in connect
ams)
AAAA
lib/python3.13/site-packages/django/utils/asyncio.py", line 26, in inner
lib/python3.13/site-packages/django/db/backends/mysql/base.py", line 256, in get_
lib/python3.13/site-packages/pymysql/connections.py", line 365, in __init__

File "/Users/mohitgupta/anaconda3/envs/mysenv-django/lib/python3.13/site-packages/pymysql/connections.py", line 723, in connect
raise exc
pymysql.err.OperationalError: (2003, "Can't connect to MySQL server on 'mohitg2.mysql.pythonanywhere-services.com' (timed out)")

illinois.edu > illinois > settings > 🐍 production.py
```

Connecting:

Use these settings in your web applications.

Database host address: mohitg2.mysql.pythonanywhere-services.com

Username: mohitg2

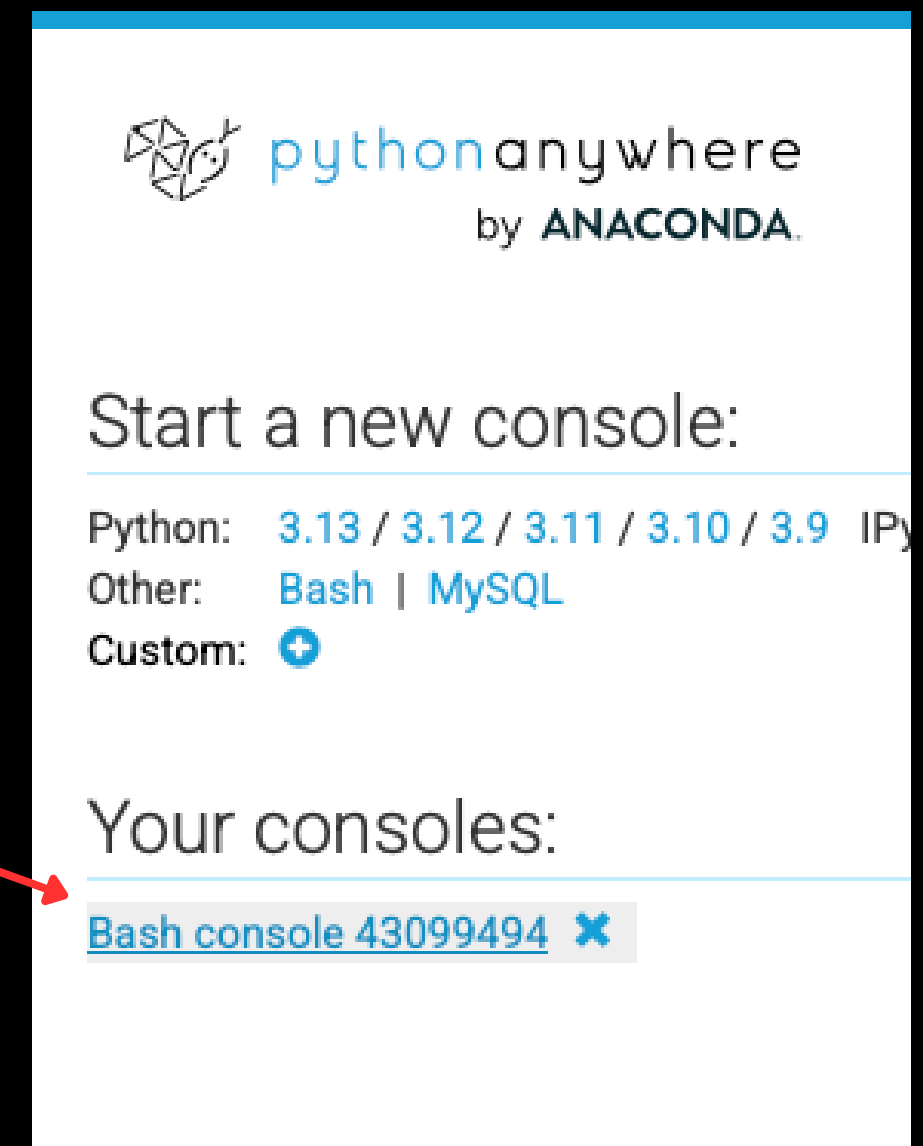
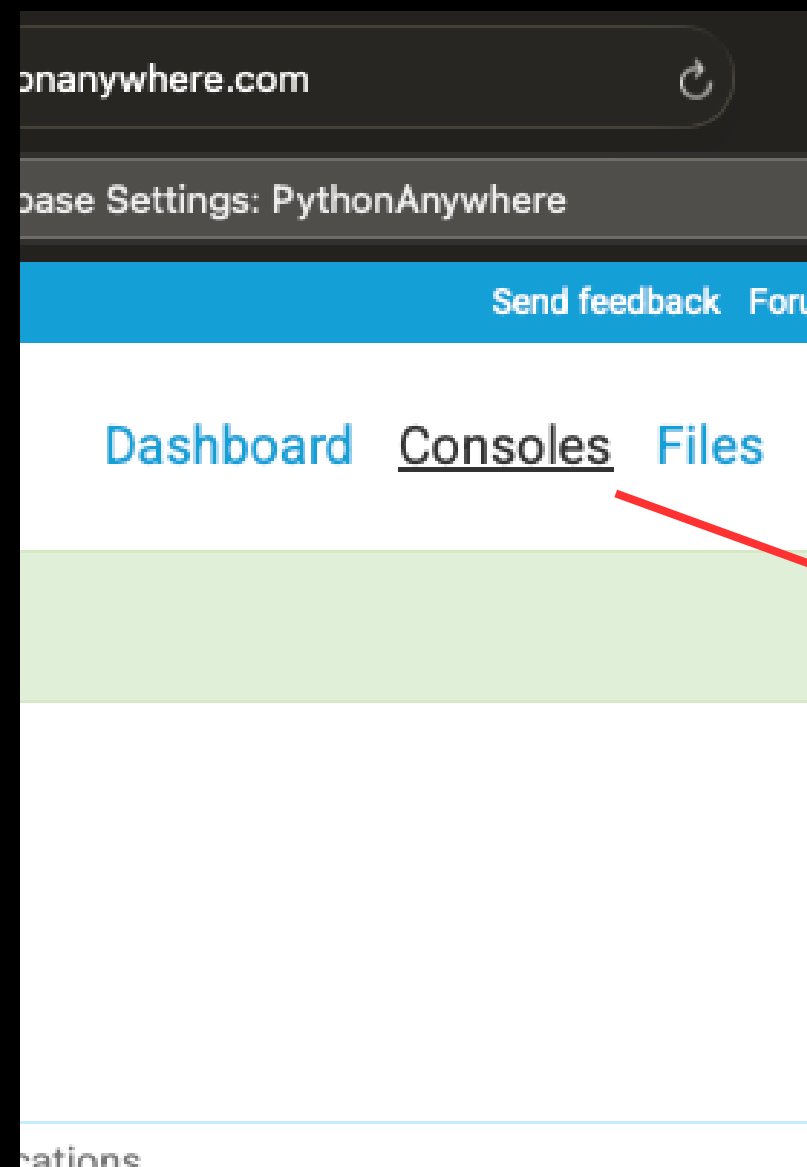
Since you have a free account, you will only be able to connect to your database using code running inside PythonAnywhere.

- **Push everything to GitHub.**

```
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % git push origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 10 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 1.98 KiB | 1.98 MiB/s, done.
Total 10 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/27guptamohit/illinois.edu.git
 1d39a27..4f8981c  main -> main
```

3.e Git Pull on PythonAnywhere

Go back to
Python Anywhere
Console



- First locate your project:

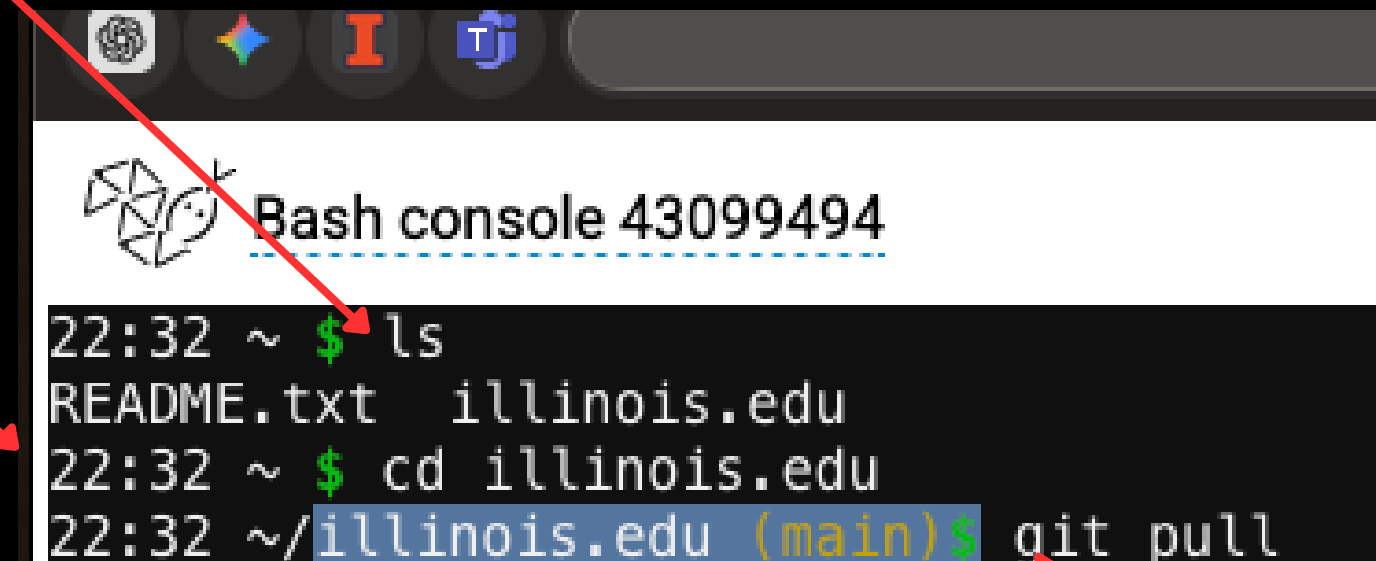
- `ls`

- Open your project using change directory command:

- `cd {name of your project}`

- Do a git pull to update your project:

- `git pull`

A terminal window titled "Bash console 43099494" showing a sequence of commands and their output. The first command is 'ls', which outputs 'README.txt' and 'illinois.edu'. The second command is 'cd illinois.edu', which changes the directory. The third command is 'git pull', which is currently being executed. The prompt shows the current directory as '~/illinois.edu' and the branch as '(main)'. Red arrows from the text on the left point to the 'ls' command, the 'cd illinois.edu' command, and the 'git pull' command in the terminal.

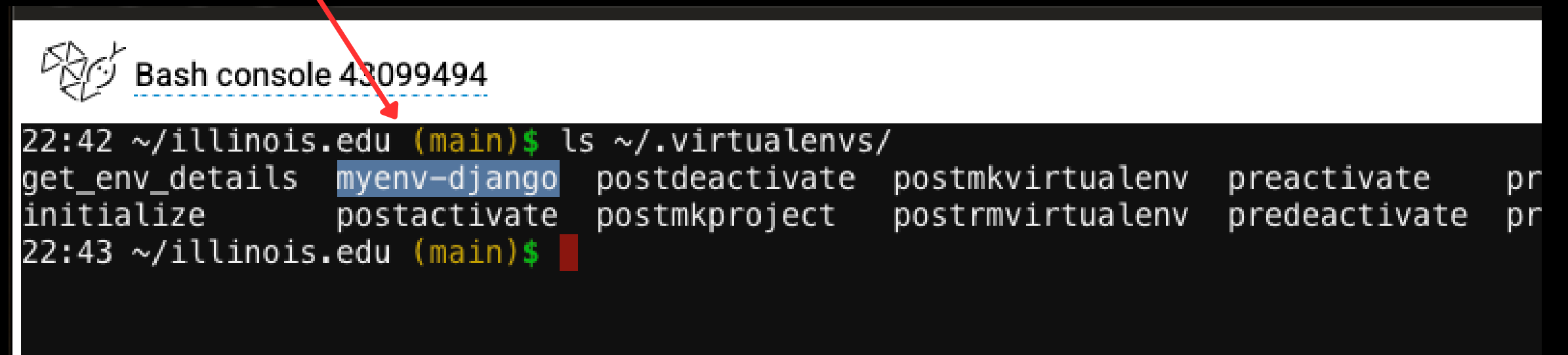
```
22:32 ~ $ ls
README.txt  illinois.edu
22:32 ~ $ cd illinois.edu
22:32 ~/illinois.edu (main)$ git pull
```

- If you have successfully logged into your project's directory, it will show the current working directory as

- `~/ {your directory's name} (main) $`

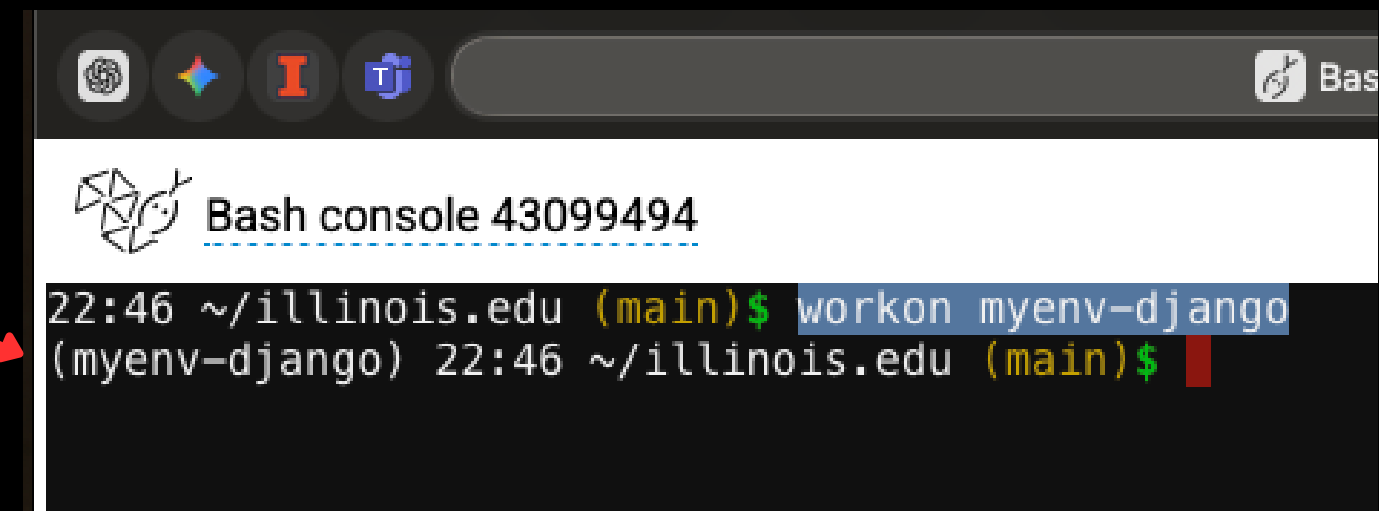
- We now need to activate our environment as I can't see any active:

- `ls ~/.virtualenvs/`



```
Bash console 43099494
22:42 ~/illinois.edu (main)$ ls ~/.virtualenvs/
get_env_details  myenv-django  postdeactivate  postmkvirtualenv  preactivate  pr
initialize      postactivate  postmkproject  postrmvirtualenv  predeactivate  pr
22:43 ~/illinois.edu (main)$
```

- **Activate using:**
 - **workon {name of your env}**



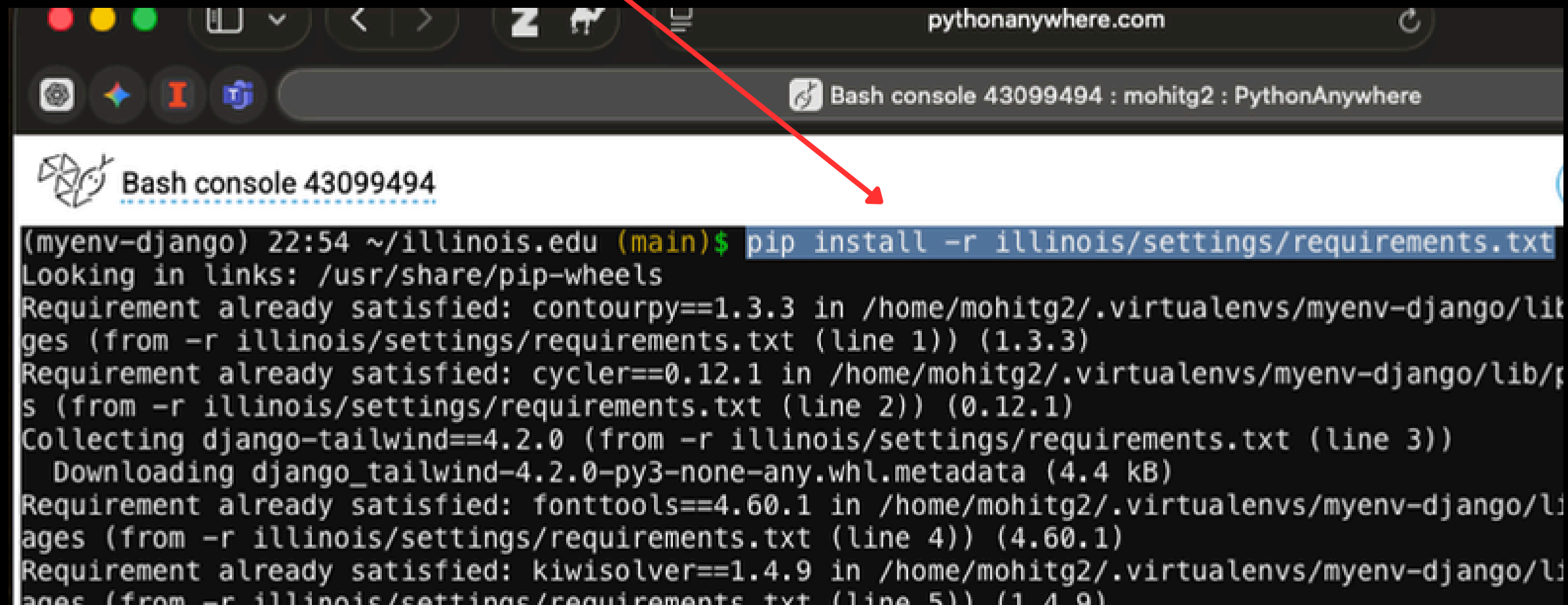
A screenshot of a terminal window titled "Bash console 43099494". The terminal shows the command `workon myenv-django` being executed. The prompt changes from `22:46 ~/illinois.edu (main)$` to `(myenv-django) 22:46 ~/illinois.edu (main)$`. A red arrow points from the text "workon {name of your env}" in the list above to the `workon` command in the terminal.

```
22:46 ~/illinois.edu (main)$ workon myenv-django
(myenv-django) 22:46 ~/illinois.edu (main)$
```

- Now install the new requirements from requirements.txt.

Only the requirements not installed will be installed:

`pip install -r illinois/settings/requirements.txt`



The screenshot shows a terminal window titled "Bash console 43099494 : mohitg2 : PythonAnywhere". The terminal output shows the command `pip install -r illinois/settings/requirements.txt` being executed. The output indicates that several requirements are already satisfied and one, `django_tailwind`, is being collected and downloaded. A red arrow points from the text "Only the requirements not installed will be installed:" to the command in the terminal.

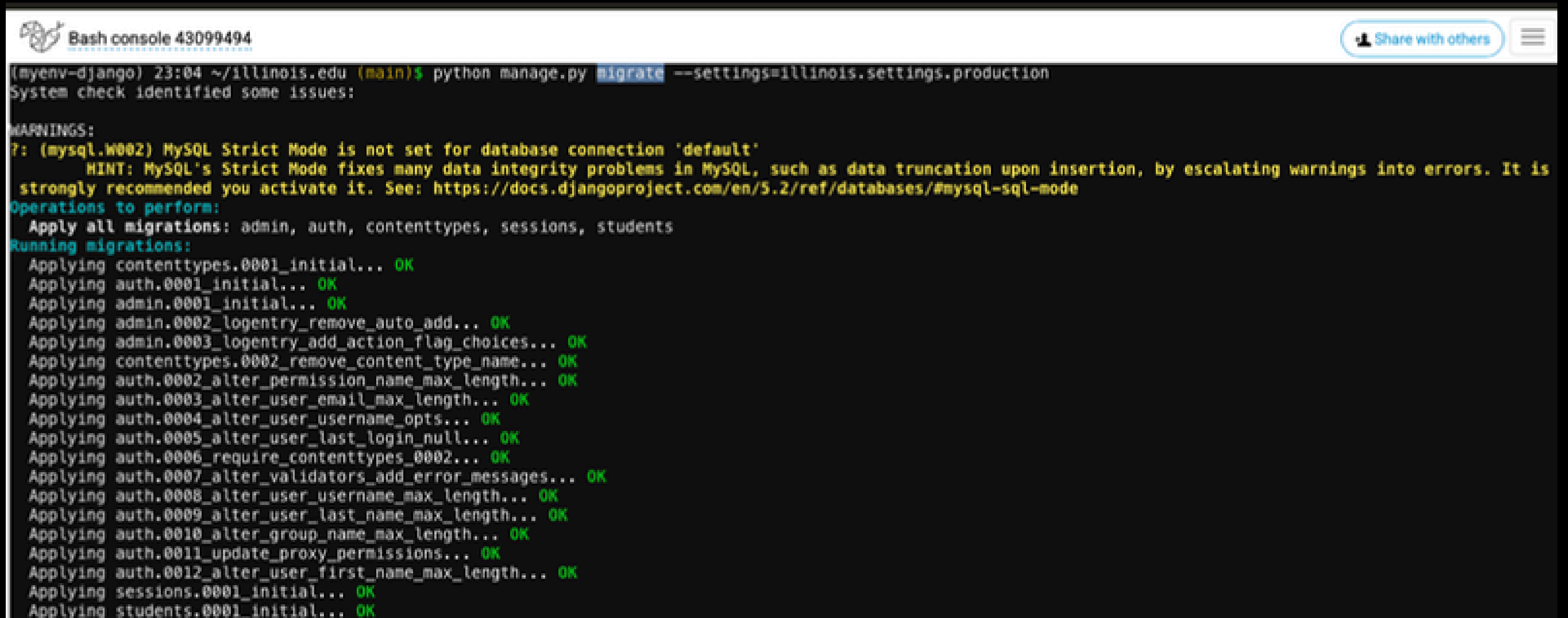
```
(myenv-django) 22:54 ~/illinois.edu (main)$ pip install -r illinois/settings/requirements.txt
Looking in links: /usr/share/pip-wheels
Requirement already satisfied: contourpy==1.3.3 in /home/mohitg2/.virtualenvs/myenv-django/lib/python3.8/site-packages (from -r illinois/settings/requirements.txt (line 1)) (1.3.3)
Requirement already satisfied: cycycler==0.12.1 in /home/mohitg2/.virtualenvs/myenv-django/lib/python3.8/site-packages (from -r illinois/settings/requirements.txt (line 2)) (0.12.1)
Collecting django_tailwind==4.2.0 (from -r illinois/settings/requirements.txt (line 3))
  Downloading django_tailwind-4.2.0-py3-none-any.whl.metadata (4.4 kB)
Requirement already satisfied: fonttools==4.60.1 in /home/mohitg2/.virtualenvs/myenv-django/lib/python3.8/site-packages (from -r illinois/settings/requirements.txt (line 4)) (4.60.1)
Requirement already satisfied: kiwisolver==1.4.9 in /home/mohitg2/.virtualenvs/myenv-django/lib/python3.8/site-packages (from -r illinois/settings/requirements.txt (line 5)) (1.4.9)
```

3.e Migrations

- It is safe to run makemigrations to make sure that we migrated the latest changes in models.py.
 - `python manage.py makemigrations --settings=illinois.settings.production`
 - `python manage.py showmigrations --settings=illinois.settings.production`

```
Bash console 43099494
(myenv-django) 22:58 ~/illinois.edu (main)$ python manage.py makemigrations --settings=illinois.settings.production
No changes detected
(myenv-django) 22:59 ~/illinois.edu (main)$ python manage.py showmigrations --settings=illinois.settings.production
admin
[ ] 0001_initial
[ ] 0002_logentry_remove_auto_add
[ ] 0003_logentry_add_action_flag_choices
auth
[ ] 0001_initial
[ ] 0002_alter_permission_name_max_length
[ ] 0003_alter_user_email_max_length
[ ] 0004_alter_user_username_opts
[ ] 0005_alter_user_last_login_null
[ ] 0006_require_contenttypes_0002
[ ] 0007_alter_validators_add_error_messages
[ ] 0008_alter_user_username_max_length
[ ] 0009_alter_user_last_name_max_length
[ ] 0010_alter_group_name_max_length
[ ] 0011_update_proxy_permissions
[ ] 0012_alter_user_first_name_max_length
contenttypes
[ ] 0001_initial
[ ] 0002_remove_content_type_name
sessions
[ ] 0001_initial
students
[ ] 0001_initial
```


- When you migrate, it will create our project's DB:
 - `python manage.py migrate --settings=illinois.settings.production`



The screenshot shows a terminal window titled "Bash console 43099494". The command executed is `python manage.py migrate --settings=illinois.settings.production`. The output indicates that a system check identified some issues, specifically a warning about MySQL Strict Mode. It then lists the operations to perform: applying migrations for admin, auth, contenttypes, sessions, and students. Finally, it shows the progress of applying these migrations, with each step marked as "OK".

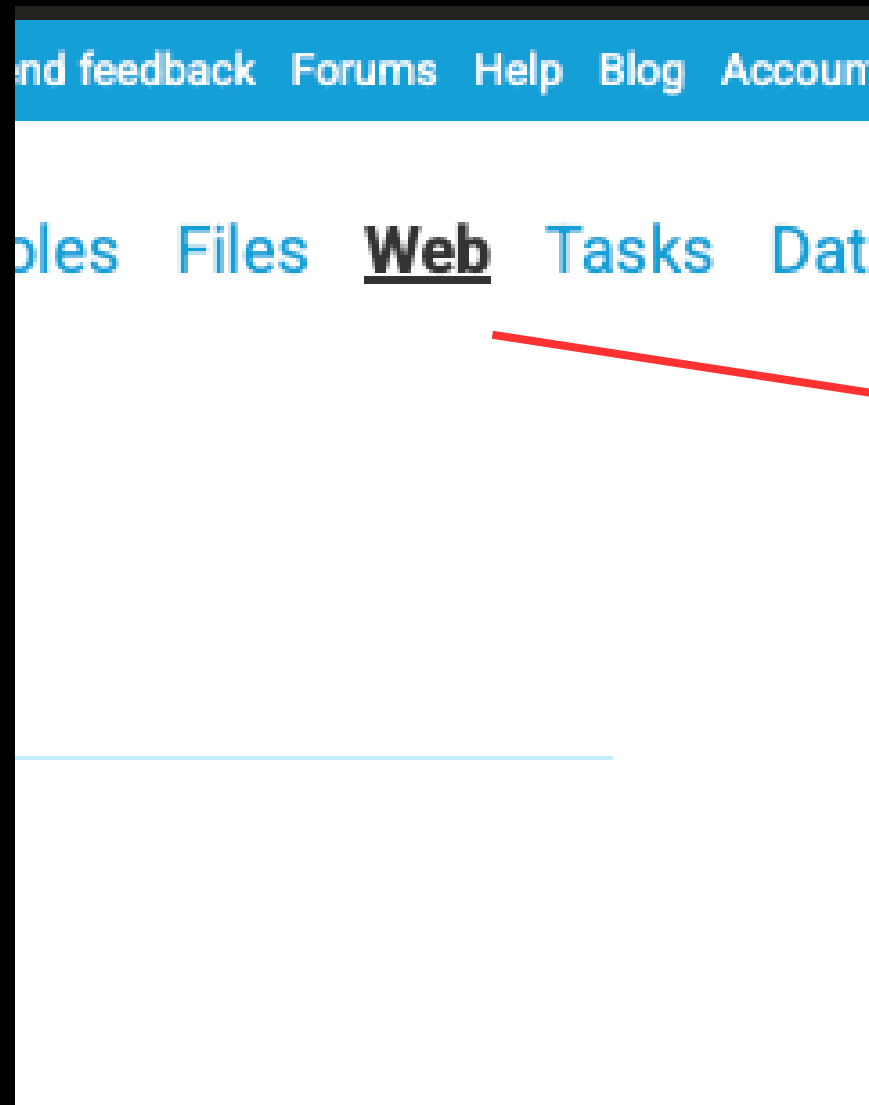
```
(myenv-django) 23:04 ~/illinois.edu (main)$ python manage.py migrate --settings=illinois.settings.production
System check identified some issues:

WARNINGS:
?: (mysql.W002) MySQL Strict Mode is not set for database connection 'default'
   HINT: MySQL's Strict Mode fixes many data integrity problems in MySQL, such as data truncation upon insertion, by escalating warnings into errors. It is
   strongly recommended you activate it. See: https://docs.djangoproject.com/en/5.2/ref/databases/#mysql-sql-mode
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, students
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
  Applying students.0001_initial... OK
```

- Now, upon running showmigrations, you'll see that all of them are applied.
 - `python manage.py showmigrations --settings=illinois.settings.production`

```
(myenv-django) 23:04 ~/illinois.edu (main)$ python manage.py showmigrations --settings=illinois.settings.production
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
[X] 0012_alter_user_first_name_max_length
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
sessions
[X] 0001_initial
students
[X] 0001_initial
```

Reload the production server for it to take effect.



Configuration for [mohitg2.pytho](#)

Reload:

 Reload mohitg2.pythonanywhere.com

Best before date:

We're happy to host your free website – and keep it free – but you'll need to log in at least once every 3 months from today" button below. We'll send you an email when it's disabled so that you don't forget to do that. [See here for more](#)

But now when you try to login,
you will not be able to do so.

Because we only created superuser for
db.sqlite3, and this is A FRESH DATABASE.

So we need to recreate the superuser.

We now have two choices (Choose one):

1. Create superuser if we are not importing
the data.json → Section 3.f

2. Load the superuser data if it was present
in the data.json → Section 3.g

illinois.edu
Course Information System

Log In Sign Up

Login

Please correct the errors below.

- Please enter a correct username and password. Note that both fields may be case-sensitive.

Username:

Password:

Login

Don't have an account? [Sign Up](#)

illinois.edu class demonstration

Portions of this project make use of open-source frameworks: [Skeleton CSS](#) by [Dave Pinkham](#) and [Normalize.css](#) by [Nicolas Gallagher](#), both under the MIT License.

Course content licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](#).

3.f

Method 1:

**Create superuser
from scratch**

- Create superuser using same credentials for me:
- `python manage.py createsuperuser --settings=illinois.settings.production`
- Username: `mohitg2`
- Email: `mohitg2@illinois.edu`
- Password: `graingerlibrary`



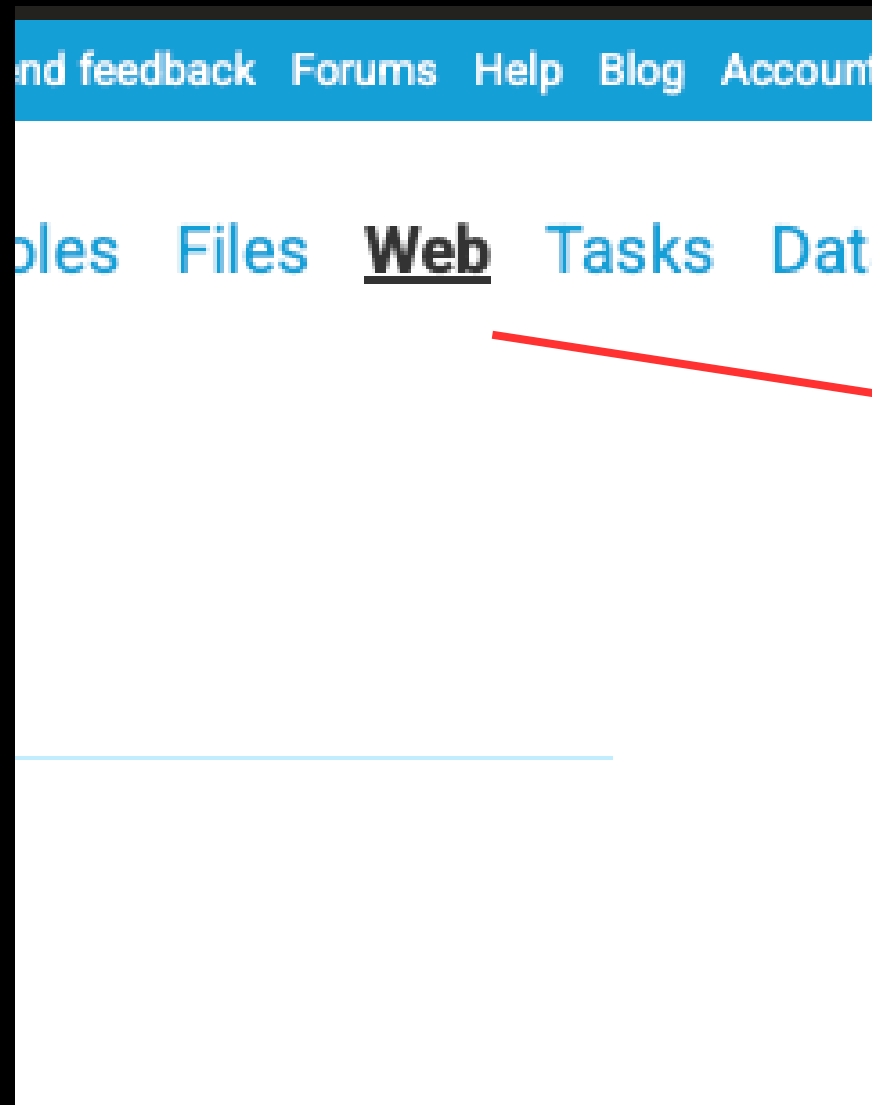
Bash console 43099494

Share with others



```
(myenv-django) 23:25 ~/illinois.edu (main)$ python manage.py createsuperuser --settings=illinois.settings.production
Username (leave blank to use 'mohitg2'): mohitg2
Email address: mohitg2@illinois.edu
Password:
Password (again):
Superuser created successfully.
(myenv-django) 23:26 ~/illinois.edu (main)$
```

Reload the production server for it to take effect.



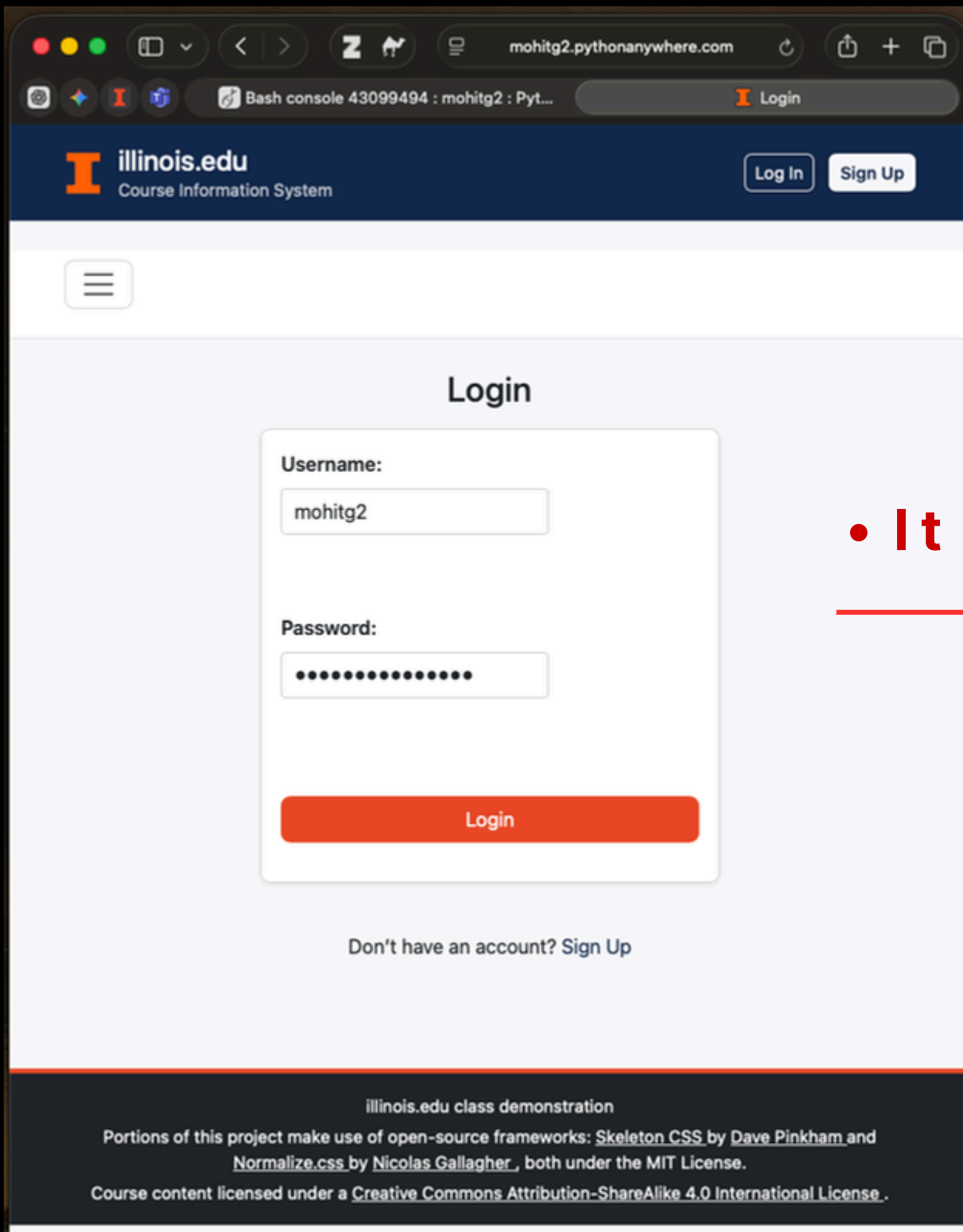
Configuration for [mohitg2.pytho](#)

Reload:

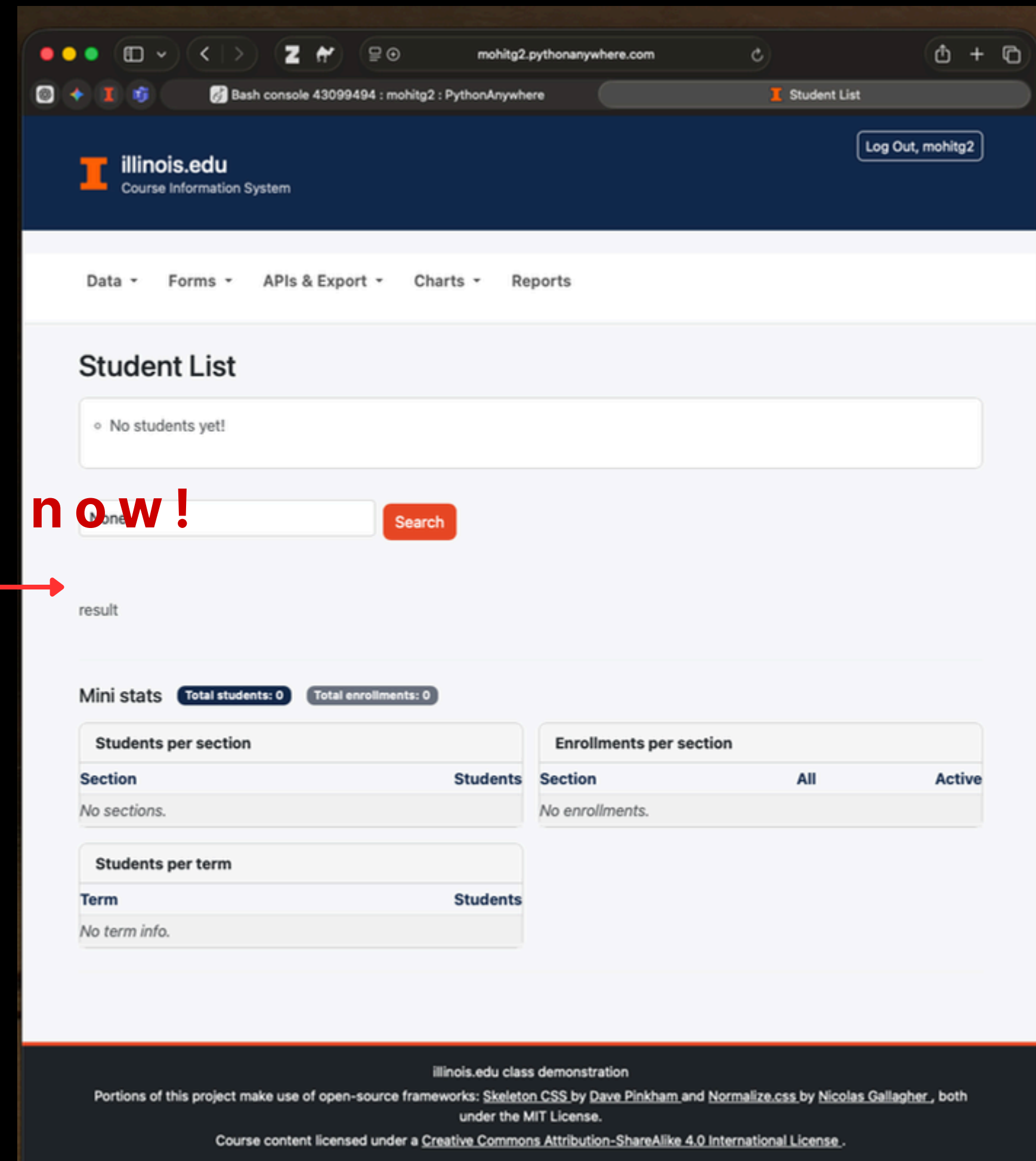
 Reload mohitg2.pythonanywhere.com

Best before date:

We're happy to host your free website – and keep it free – but you'll need to log in at least once every 3 months from today" button below. We'll send you an email when it's disabled so that you don't forget to do that. [See here for more](#)



• It works now!

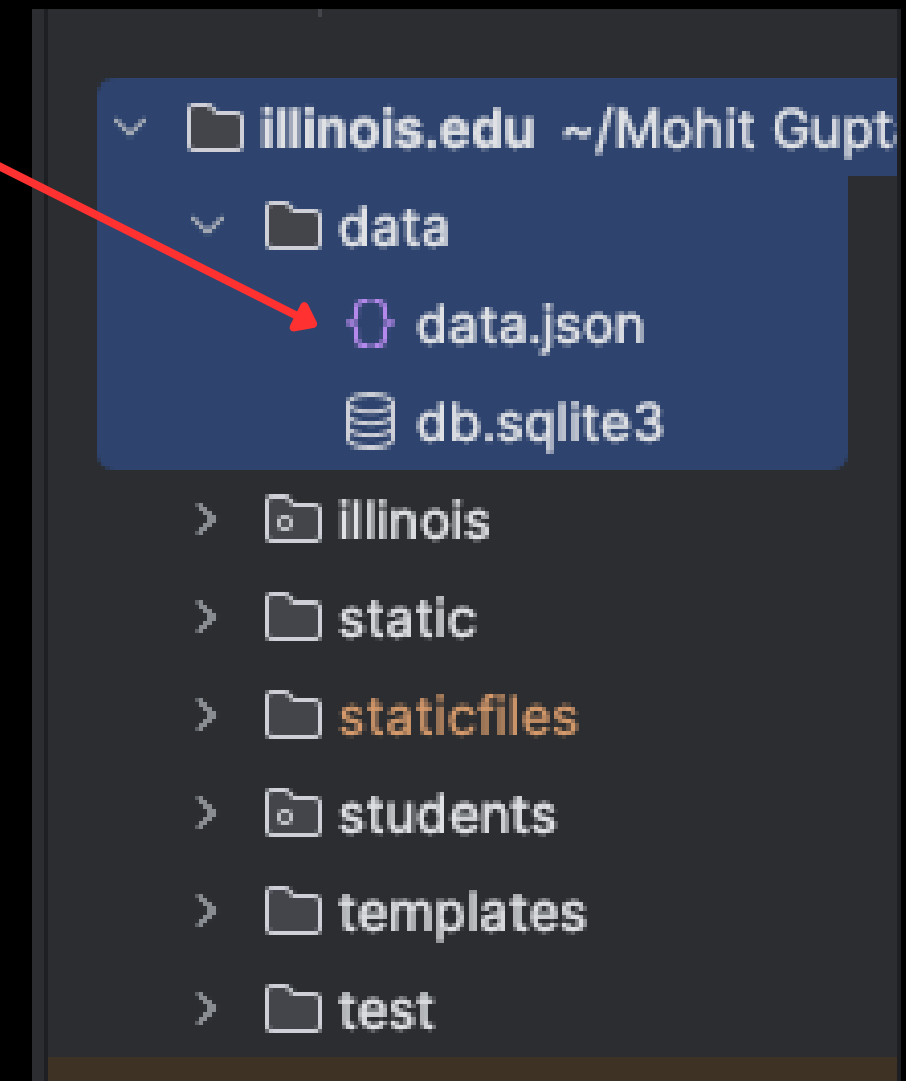


3.g

Method 2:

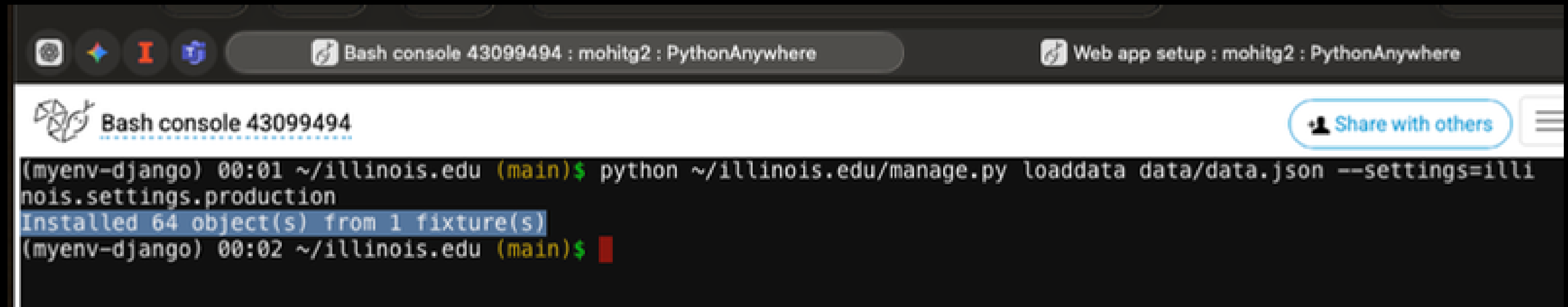
Load old data from **data.json**

- Let's load this datadump from early steps.



- Load the data/data.json using “loaddata”:

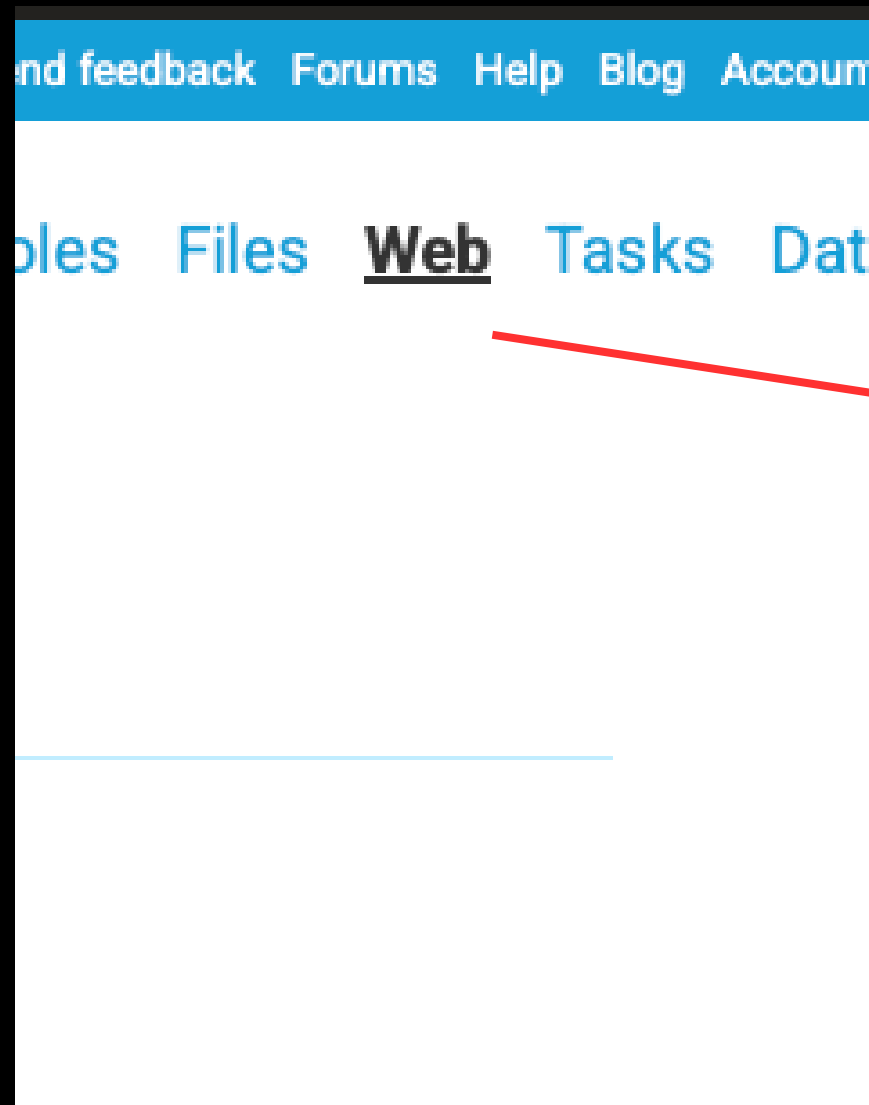
- `python ~/illinois.edu/manage.py loaddata data/data.json --settings=illinois.settings.production`



The screenshot shows a web-based terminal interface with a dark theme. At the top, there are two tabs: "Bash console 43099494 : mohitg2 : PythonAnywhere" and "Web app setup : mohitg2 : PythonAnywhere". The active tab is the Bash console. Below the tabs, the terminal window has a title bar "Bash console 43099494" and a "Share with others" button. The terminal content shows a command being executed in a Django environment:

```
(myenv-django) 00:01 ~/illinois.edu (main)$ python ~/illinois.edu/manage.py loaddata data/data.json --settings=illinois.settings.production
Installed 64 object(s) from 1 fixture(s)
(myenv-django) 00:02 ~/illinois.edu (main)$
```


Reload the production server for it to take effect.



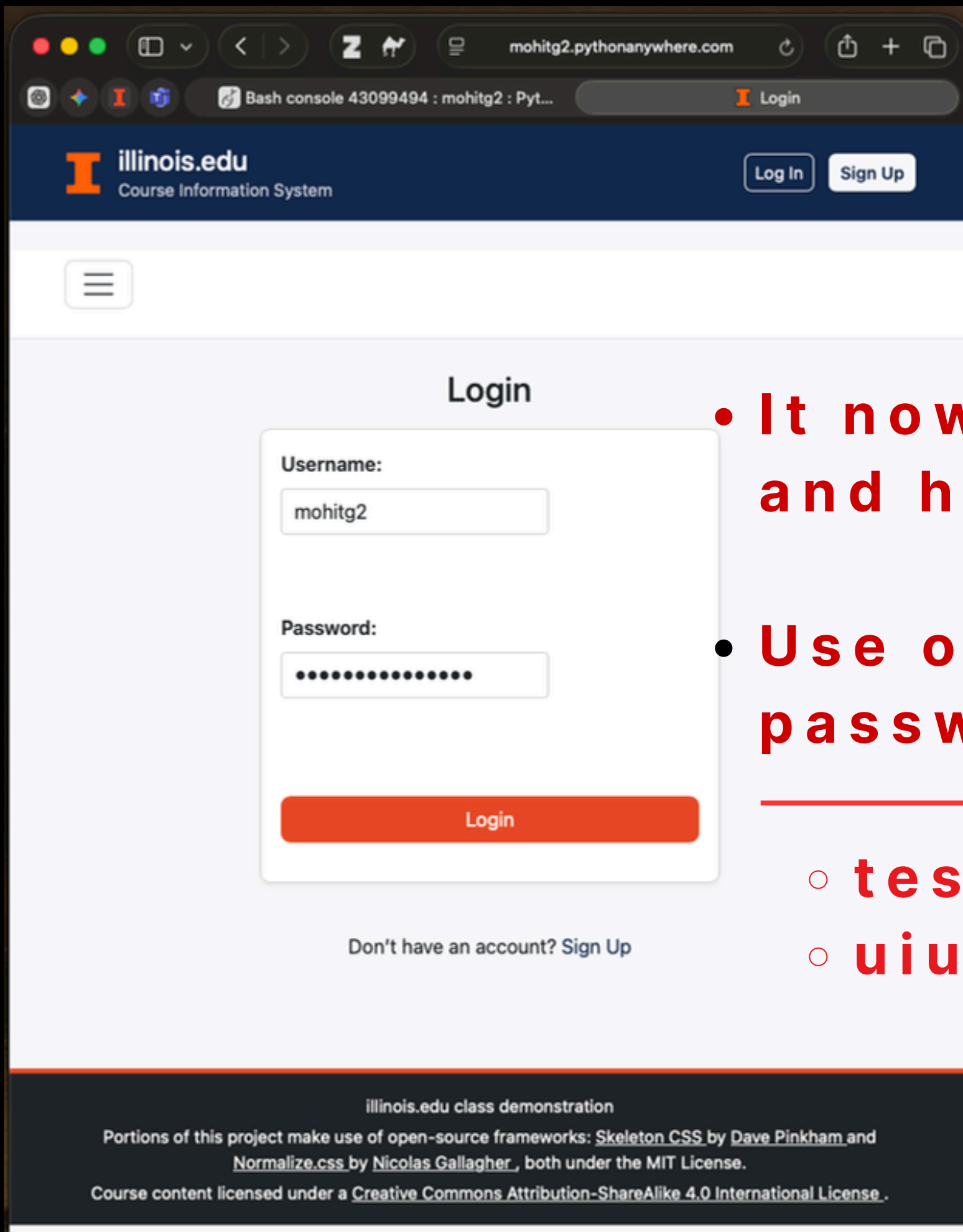
Configuration for [mohitg2.pytho](#)

Reload:

 Reload mohitg2.pythonanywhere.com

Best before date:

We're happy to host your free website – and keep it free – but to keep it running, but you'll need to log in at least once every 3 months from today" button below. We'll send you an email when disabled so that you don't forget to do that. [See here for more](#)

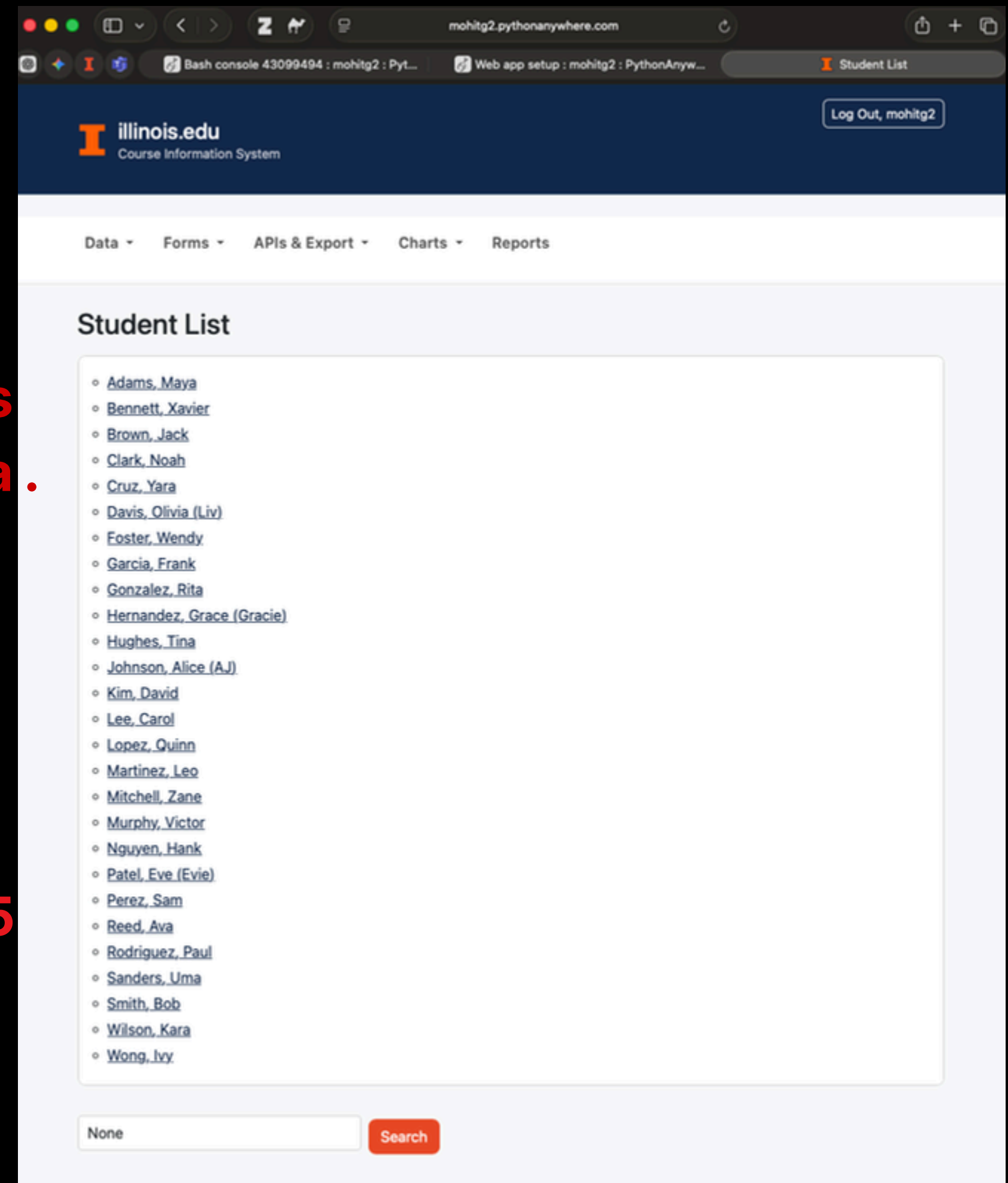


• It now works and has data.

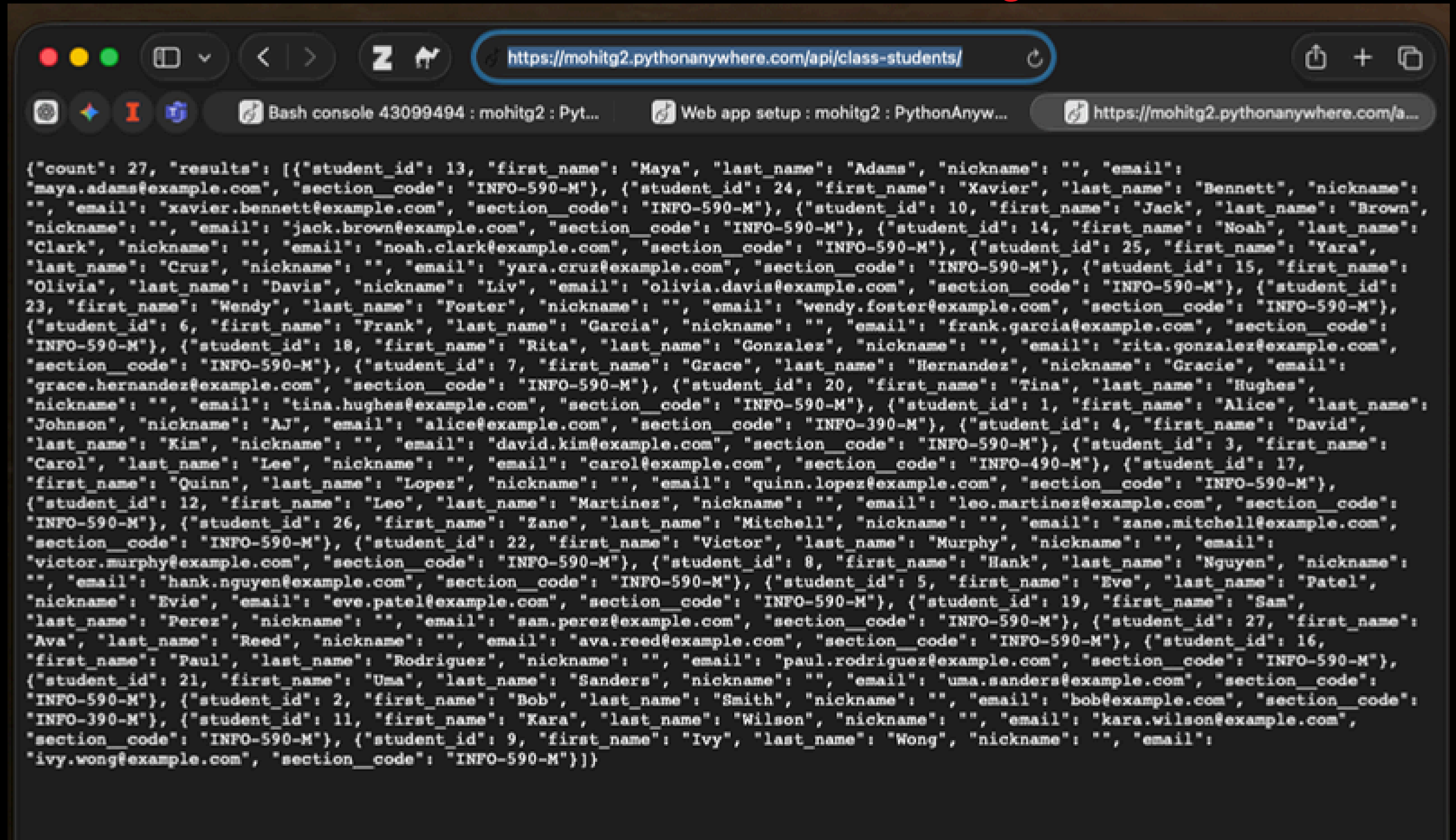
• Use old password:

○ tester

○ uiuc12345



The API is also working



The screenshot shows a web browser window with the address bar displaying `https://mohitg2.pythonanywhere.com/api/class-students/`. The browser tabs include "Bash console 43099494 : mohitg2 : Pyt...", "Web app setup : mohitg2 : PythonAnyw...", and the current page. The main content area displays a large JSON object representing a list of 27 students. Each student record includes fields for `student_id`, `first_name`, `last_name`, `nickname`, `email`, and `section_code`. The JSON is formatted with syntax highlighting.

```
{
  "count": 27,
  "results": [
    {
      "student_id": 13,
      "first_name": "Maya",
      "last_name": "Adams",
      "nickname": "",
      "email": "maya.adams@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 24,
      "first_name": "Xavier",
      "last_name": "Bennett",
      "nickname": "",
      "email": "xavier.bennett@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 10,
      "first_name": "Jack",
      "last_name": "Brown",
      "nickname": "",
      "email": "jack.brown@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 14,
      "first_name": "Noah",
      "last_name": "Clark",
      "nickname": "",
      "email": "noah.clark@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 25,
      "first_name": "Yara",
      "last_name": "Cruz",
      "nickname": "",
      "email": "yara.cruz@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 15,
      "first_name": "Olivia",
      "last_name": "Davis",
      "nickname": "Liv",
      "email": "olivia.davis@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 23,
      "first_name": "Wendy",
      "last_name": "Foster",
      "nickname": "",
      "email": "wendy.foster@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 6,
      "first_name": "Frank",
      "last_name": "Garcia",
      "nickname": "",
      "email": "frank.garcia@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 18,
      "first_name": "Rita",
      "last_name": "Gonzalez",
      "nickname": "",
      "email": "rita.gonzalez@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 7,
      "first_name": "Grace",
      "last_name": "Hernandez",
      "nickname": "Gracie",
      "email": "grace.hernandez@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 20,
      "first_name": "Tina",
      "last_name": "Hughes",
      "nickname": "",
      "email": "tina.hughes@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 1,
      "first_name": "Alice",
      "last_name": "Johnson",
      "nickname": "AJ",
      "email": "alice@example.com",
      "section_code": "INFO-390-M"
    },
    {
      "student_id": 4,
      "first_name": "David",
      "last_name": "Kim",
      "nickname": "",
      "email": "david.kim@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 3,
      "first_name": "Carol",
      "last_name": "Lee",
      "nickname": "",
      "email": "carol@example.com",
      "section_code": "INFO-490-M"
    },
    {
      "student_id": 17,
      "first_name": "Quinn",
      "last_name": "Lopez",
      "nickname": "",
      "email": "quinn.lopez@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 12,
      "first_name": "Leo",
      "last_name": "Martinez",
      "nickname": "",
      "email": "leo.martinez@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 26,
      "first_name": "Zane",
      "last_name": "Mitchell",
      "nickname": "",
      "email": "zane.mitchell@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 22,
      "first_name": "Victor",
      "last_name": "Murphy",
      "nickname": "",
      "email": "victor.murphy@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 8,
      "first_name": "Hank",
      "last_name": "Nguyen",
      "nickname": "",
      "email": "hank.nguyen@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 5,
      "first_name": "Eve",
      "last_name": "Patel",
      "nickname": "Evie",
      "email": "eve.patel@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 19,
      "first_name": "Sam",
      "last_name": "Perez",
      "nickname": "",
      "email": "sam.perez@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 27,
      "first_name": "Ava",
      "last_name": "Reed",
      "nickname": "",
      "email": "ava.reed@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 16,
      "first_name": "Paul",
      "last_name": "Rodriguez",
      "nickname": "",
      "email": "paul.rodriguez@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 21,
      "first_name": "Uma",
      "last_name": "Sanders",
      "nickname": "",
      "email": "uma.sanders@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 2,
      "first_name": "Bob",
      "last_name": "Smith",
      "nickname": "",
      "email": "bob@example.com",
      "section_code": "INFO-390-M"
    },
    {
      "student_id": 11,
      "first_name": "Kara",
      "last_name": "Wilson",
      "nickname": "",
      "email": "kara.wilson@example.com",
      "section_code": "INFO-590-M"
    },
    {
      "student_id": 9,
      "first_name": "Ivy",
      "last_name": "Wong",
      "nickname": "",
      "email": "ivy.wong@example.com",
      "section_code": "INFO-590-M"
    }
  ]
}
```

4. Google OAuth

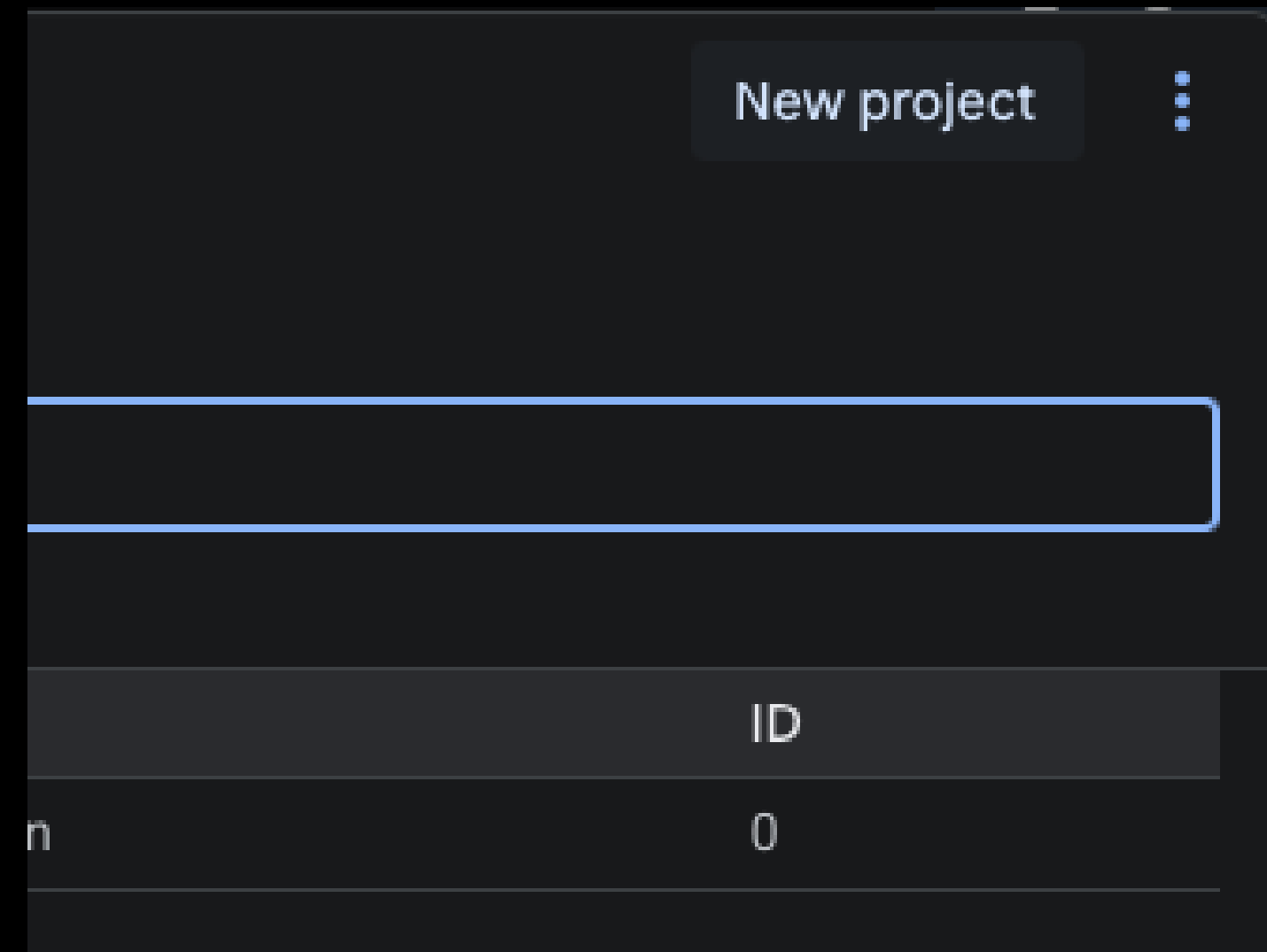
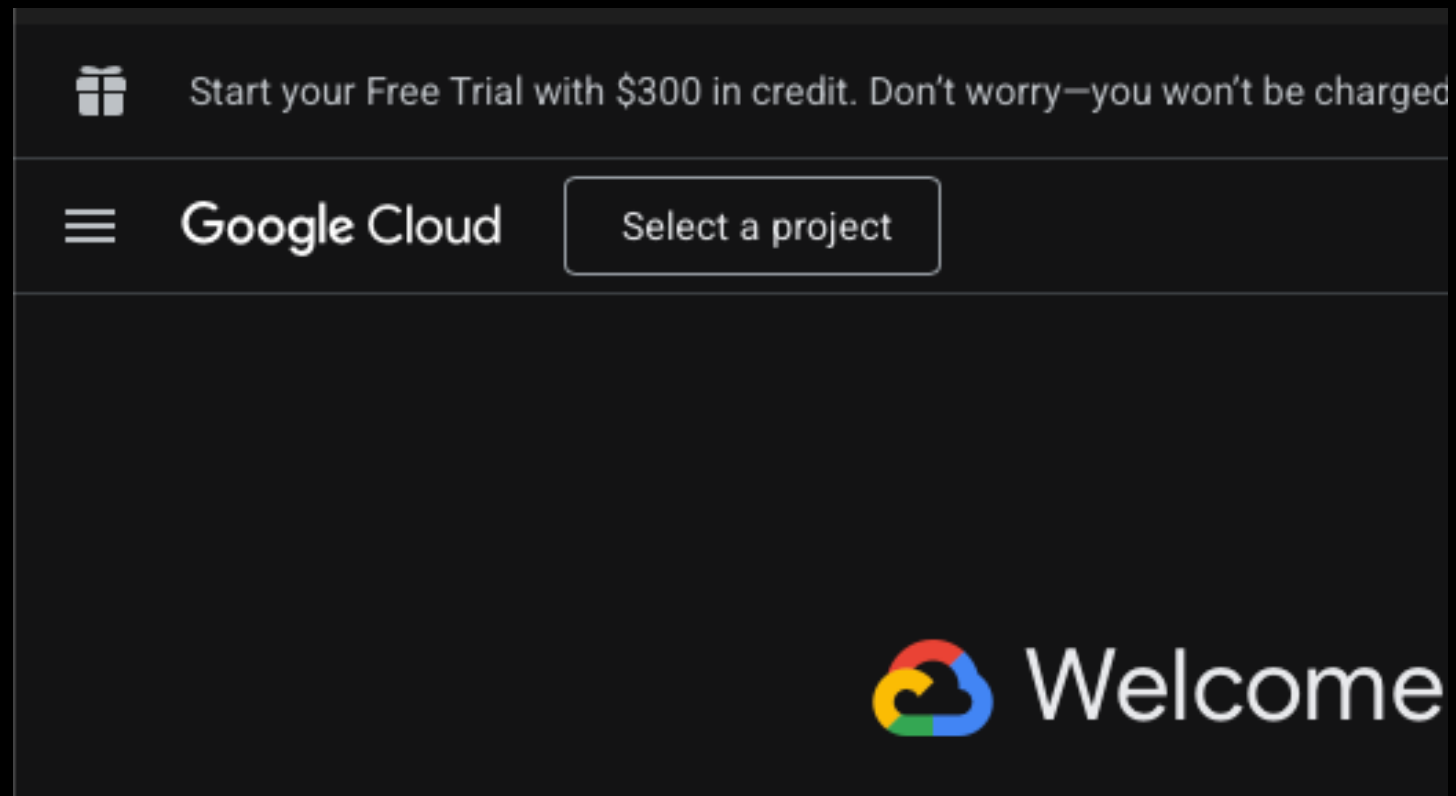
Sources: The Proton Guy: <https://www.youtube.com/watch?v=ipg9WPY3Its>
More Tutorials: https://www.youtube.com/watch?v=_9K0MTBOKNs

4.a

Google Console

Step 4.1:

- **Login to google console:**
 - <https://console.cloud.google.com>
- **Click on “Select a Project”**
- **Then click on “New Project”**



Step 4.2:

- Type the name of the project.
- Now hit “Create”

Project name *

info490-gcl-demo

?

Project ID: info490-gcl-demo. It cannot be changed later. [Edit](#)

Parent resource *

No organization

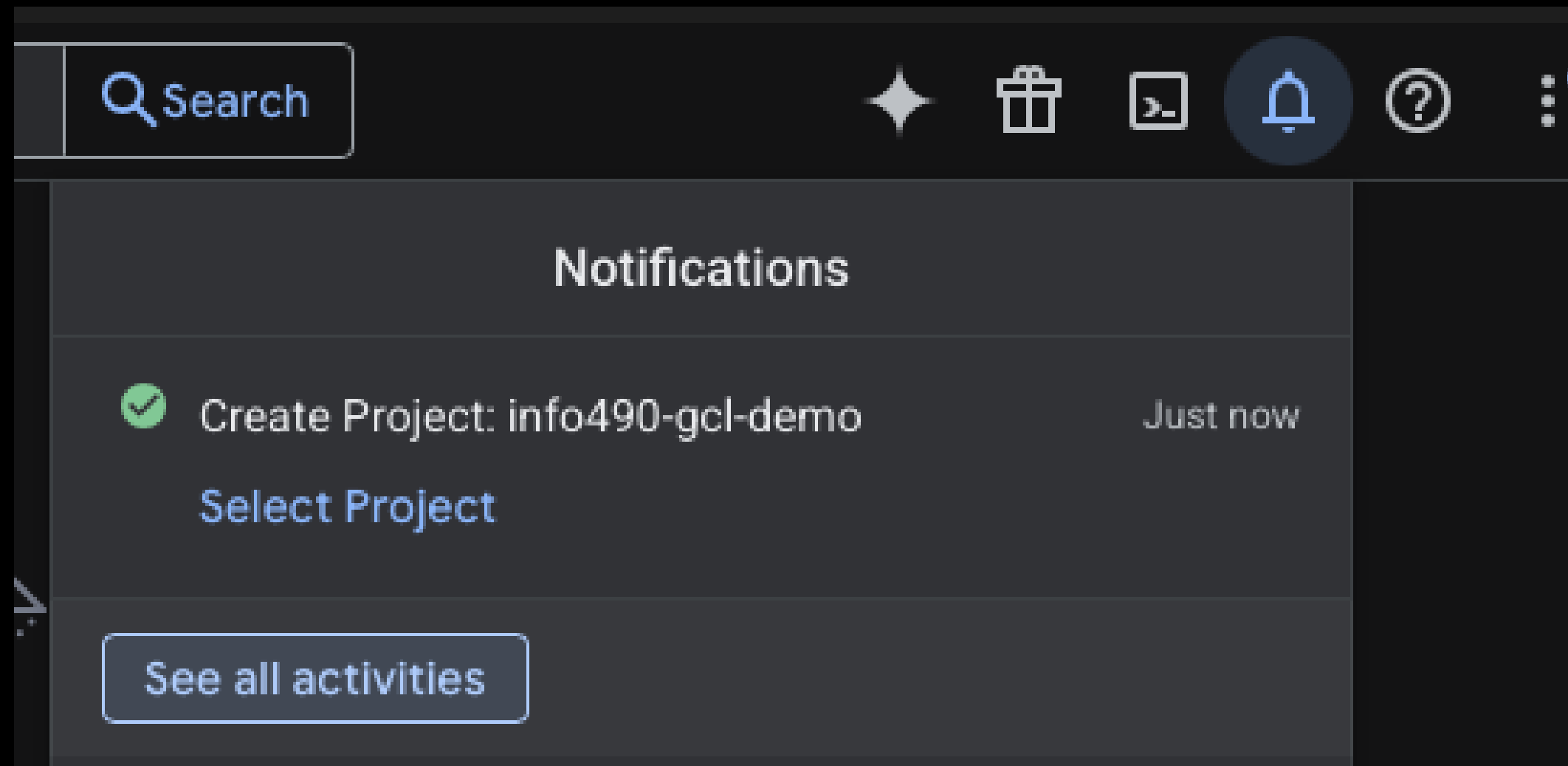
Browse

Select the organization or folder where this resource will be created

Create

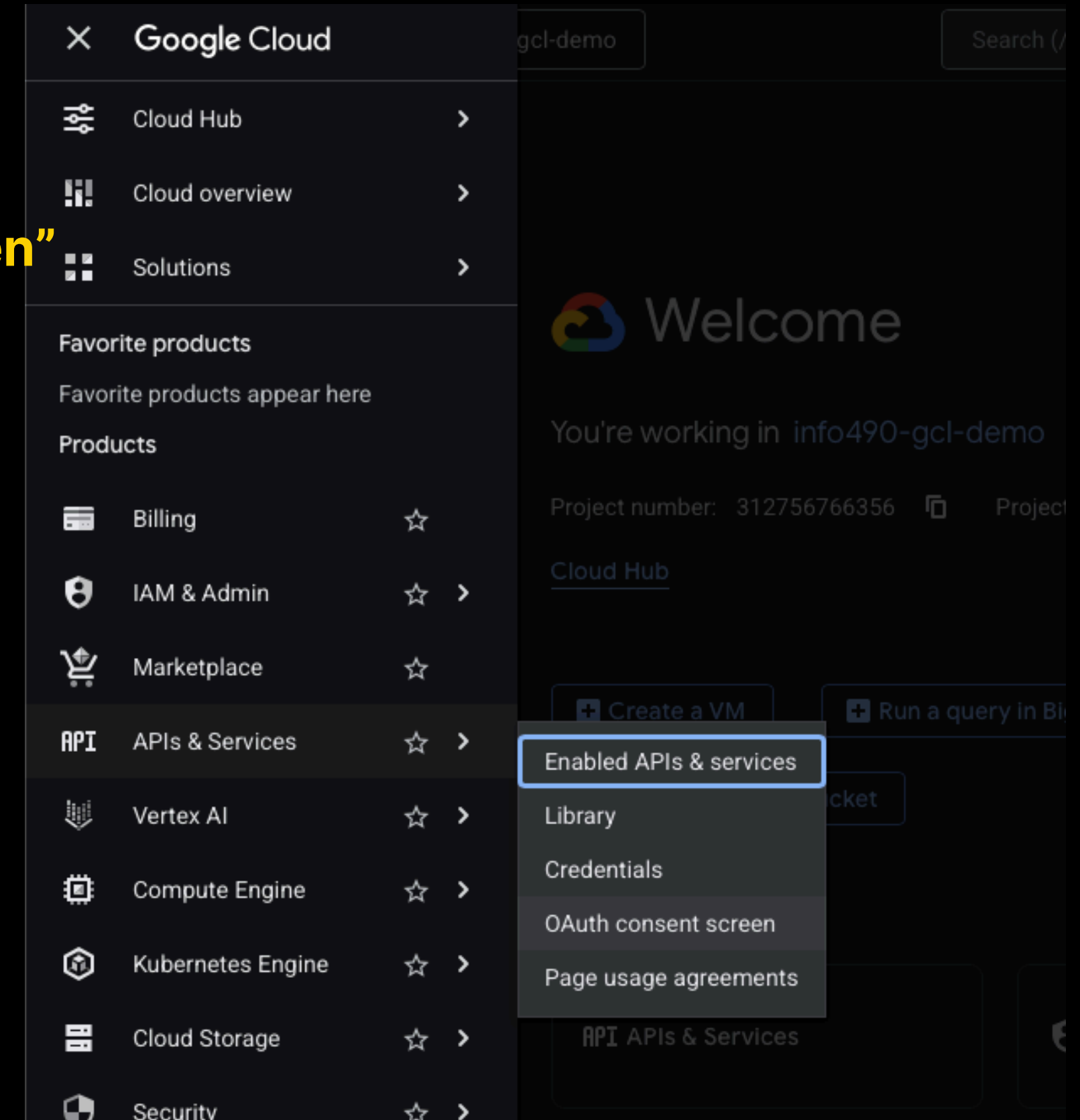
Cancel

- **Open the Project.**
- **You will find brief summary about the project.**



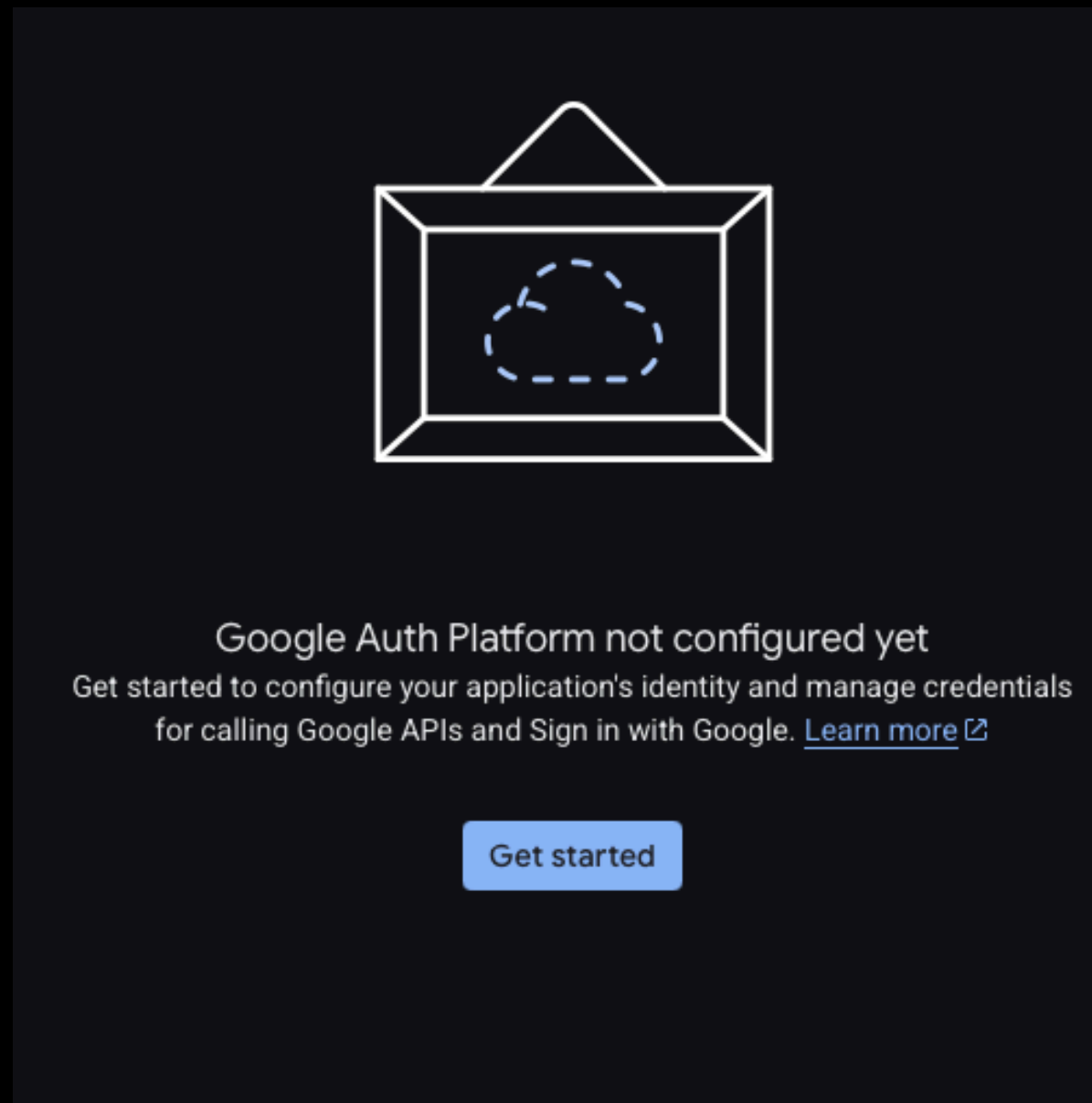
Step 4.3:

- Open the menu and navigate to:
 - “APIs & Services” > “OAuth consent screen”



Step 4.4:

- Click on “Get Started”
- And start filling these details



Project configuration

- 1 App Information
 - App name *
The name of the app asking for consent
 - User support email *
For users to contact you with questions about their consent. [Learn more](#)
 - Next
- 2 Audience
- 3 Contact Information
- 4 Finish

Create Cancel

Fill the details as shown.

App information

This shows in the consent screen, and helps end users know who you are and contact you

App name *

INFO490-GCL-DEMO

The name of the app asking for consent

3 Contact Information

Email addresses *

mohitg2@illinois.edu

These email addresses are for Google to notify you about any changes to your project.

Next

2 Audience

☐ Internal

Only available to users within your organization. You will not need to submit your app for verification. [Learn more about user type](#)

☒ External

Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. [Learn more about user type](#)

Next

4 Finish

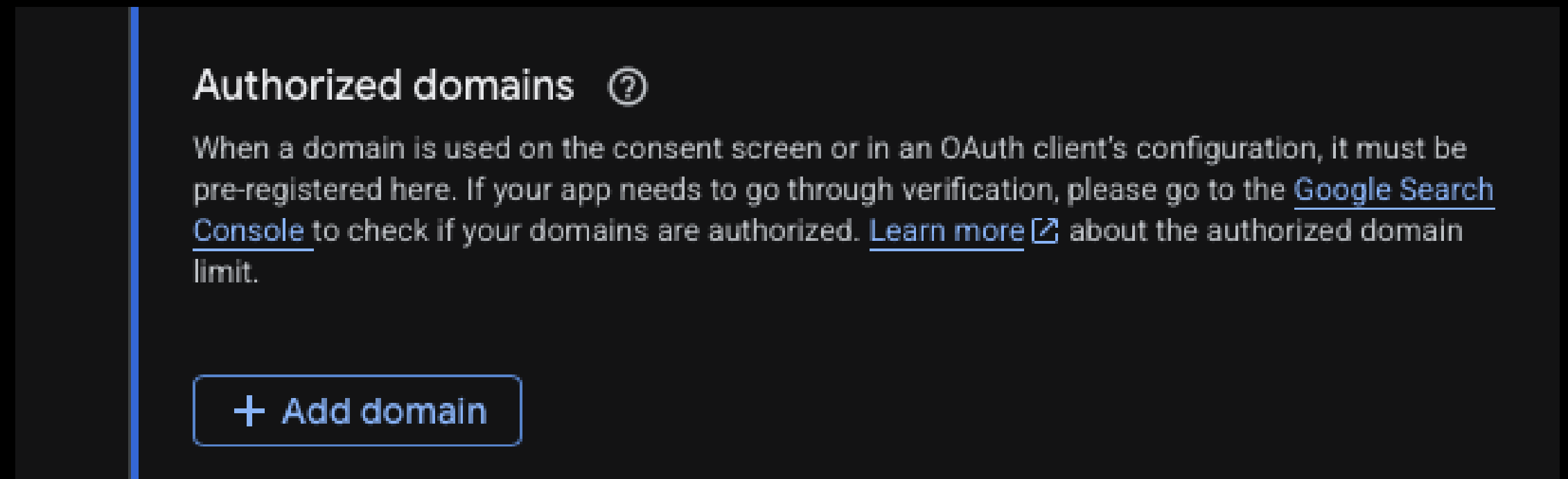
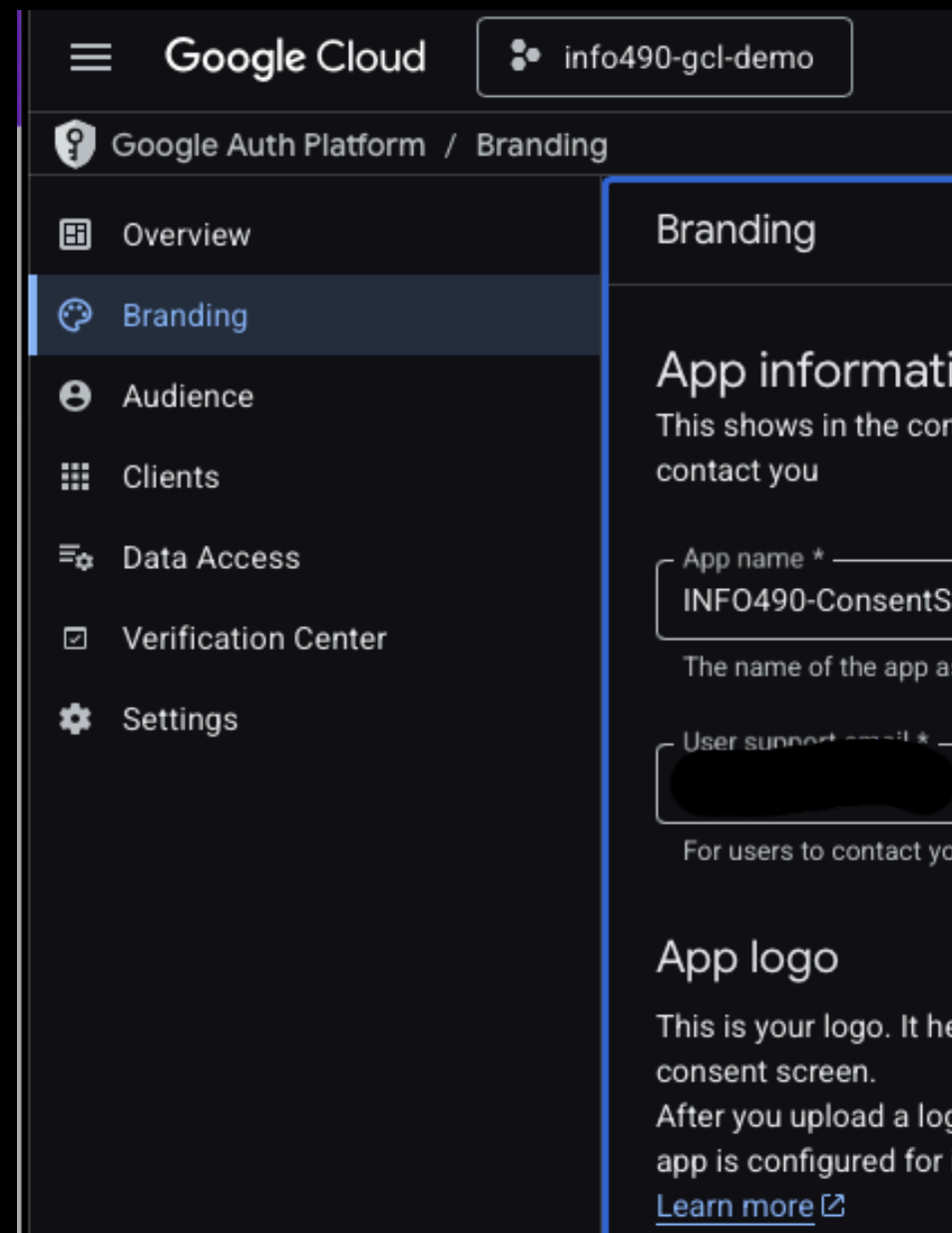
☒ I agree to the [Google API Services: User Data Policy](#)

Continue

Create Cancel

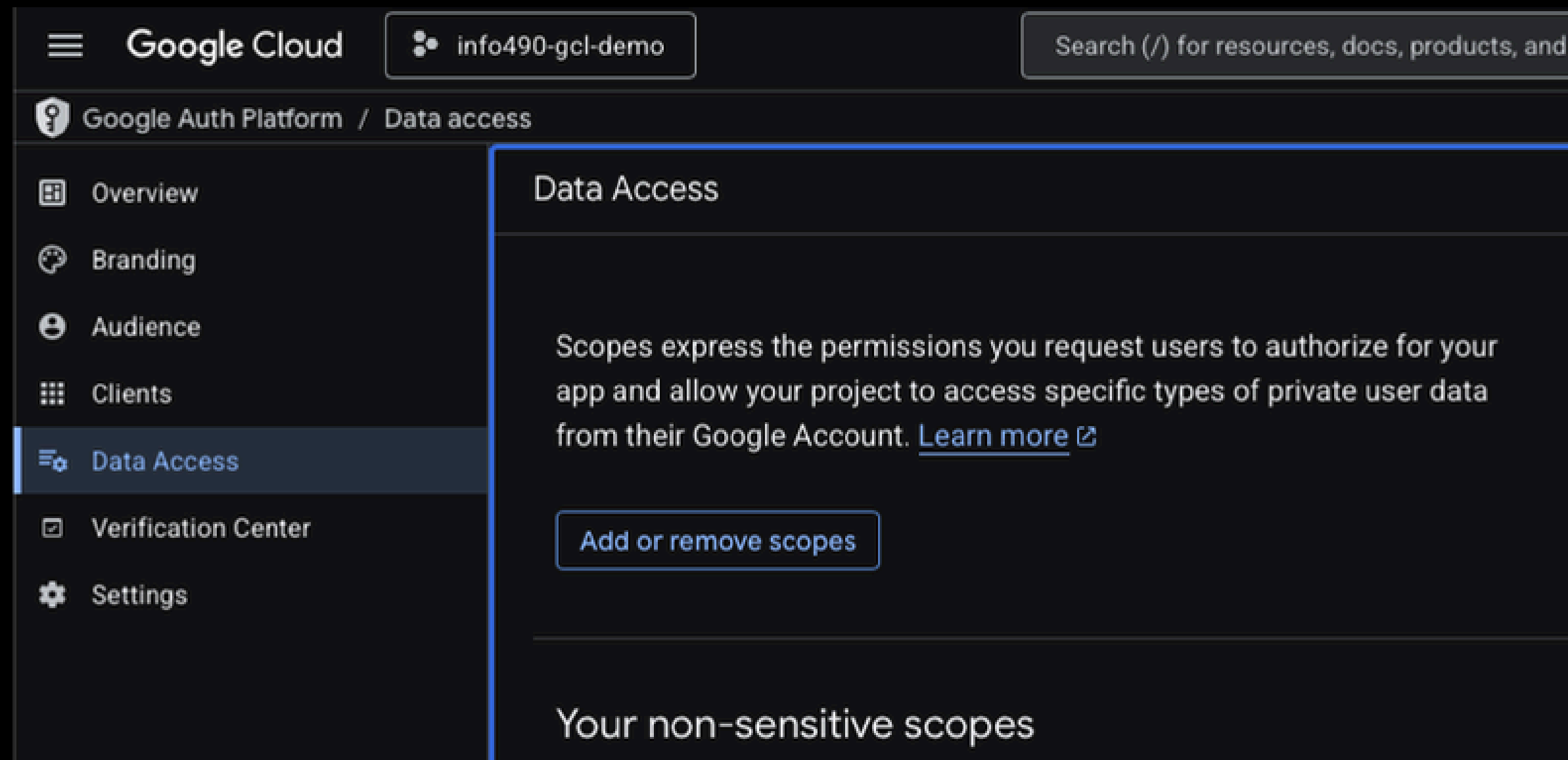
Step 4.5:

- Click on “Branding” and scroll down to “Authorized domains”
- This is where you’ll add live domains in future (not development server URLs)



Step 4.6:

- Now we set what information about the user we want from Google.
- Navigate to "Data Access" > "Add or remove scopes"



X Update selected scopes

i Only scopes for enabled APIs are listed below. To add a missing scope to this screen, find and enable the API in the [Google API Library](#) or use the Pasted Scopes text box below. Refresh the page to see any new APIs you enable from the Library.

Filter Enter property name or value ?

	API ↑	Scope	User-facing description
☒		.../auth/userinfo.email	See your primary Google Account email address
☒		.../auth/userinfo.profile	See your personal info, including any personal info you've made publicly available
☐		openid	Associate you with your personal info on Google
☐	Analytics Hub API	.../auth/bigquery	View and manage your data in Google BigQuery and see the email address for your Google Account
☐	Analytics Hub API	.../auth/cloud-platform	See, edit, configure, and delete your Google Cloud data and see the email address for your Google Account.
☐	BigQuery API	.../auth/bigquery.readonly	View your data in Google BigQuery
☐	BigQuery API	.../auth/cloud-platform.read-only	View your data across Google Cloud services and see the email address of your Google Account
☐	BigQuery API	.../auth/devstorage.full_control	Manage your data and permissions in Cloud Storage and see the email address for your Google Account
☐	BigQuery API	.../auth/devstorage.read_only	View your data in Google Cloud Storage
☐	BigQuery API	.../auth/devstorage.read_write	Manage your data in Cloud Storage and see the email address of your Google Account

Manually add scopes

If the scopes you would like to add do not appear in the table above, you can enter them here. Each scope should be on a new line or separated by commas. Please provide the full scope string (beginning with "https://"). When

- For now, I'm just going to select the first two fields.
- Once done, click "Update"
- This is the data that you'd be pulling from the Google API.
- At last, click "Save"

Your non-sensitive scopes

API ↑	Scope	User-facing description
	.../auth/userinfo.email	See your primary Google Account email address
	.../auth/userinfo.profile	See your personal info, including any personal info you've made publicly available

🔒 Your sensitive scopes

Sensitive scopes are scopes that request access to private user data.

API ↑	Scope	User-facing description
No rows to display		

🔒 Your restricted scopes

Restricted scopes are scopes that request access to highly sensitive user data.

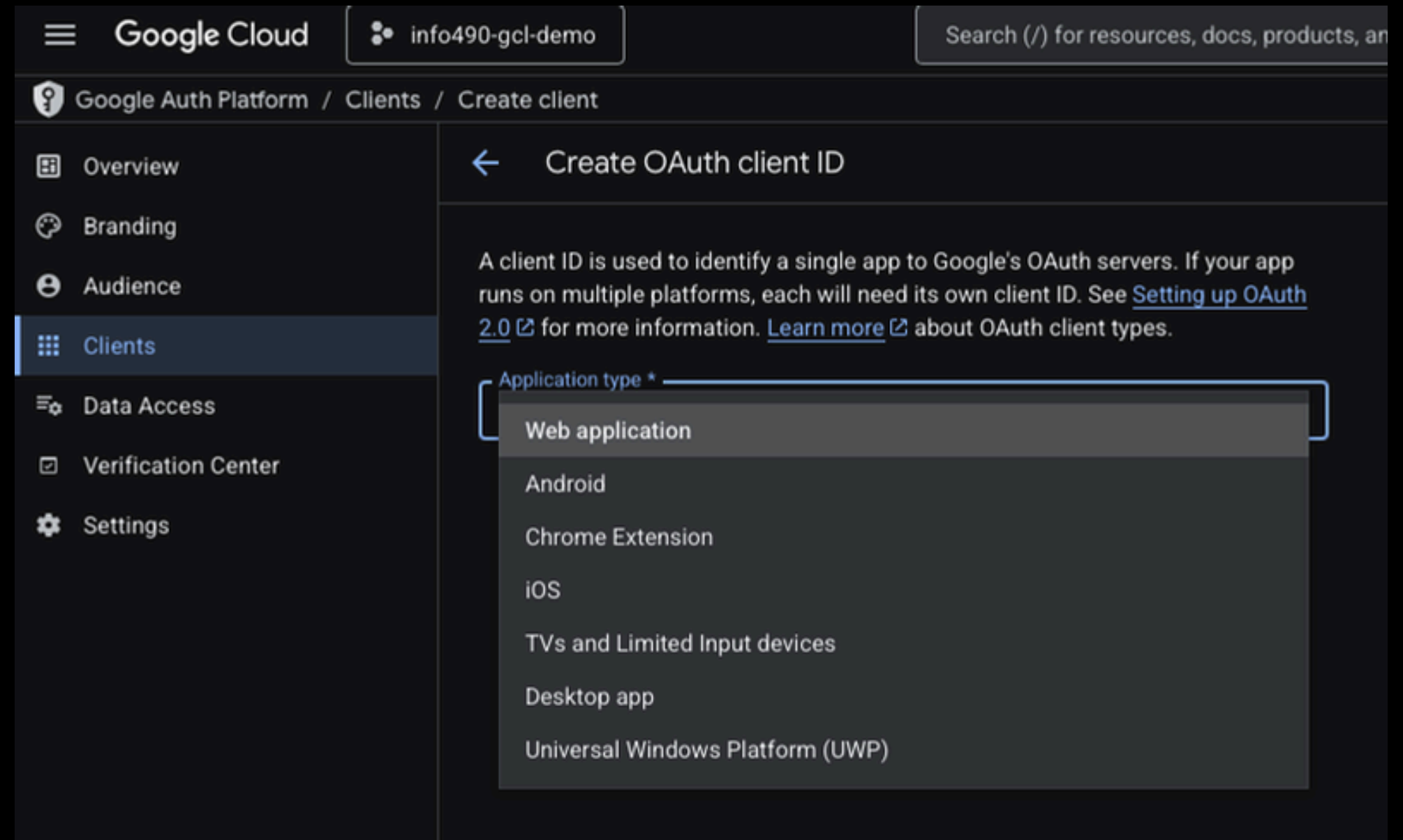
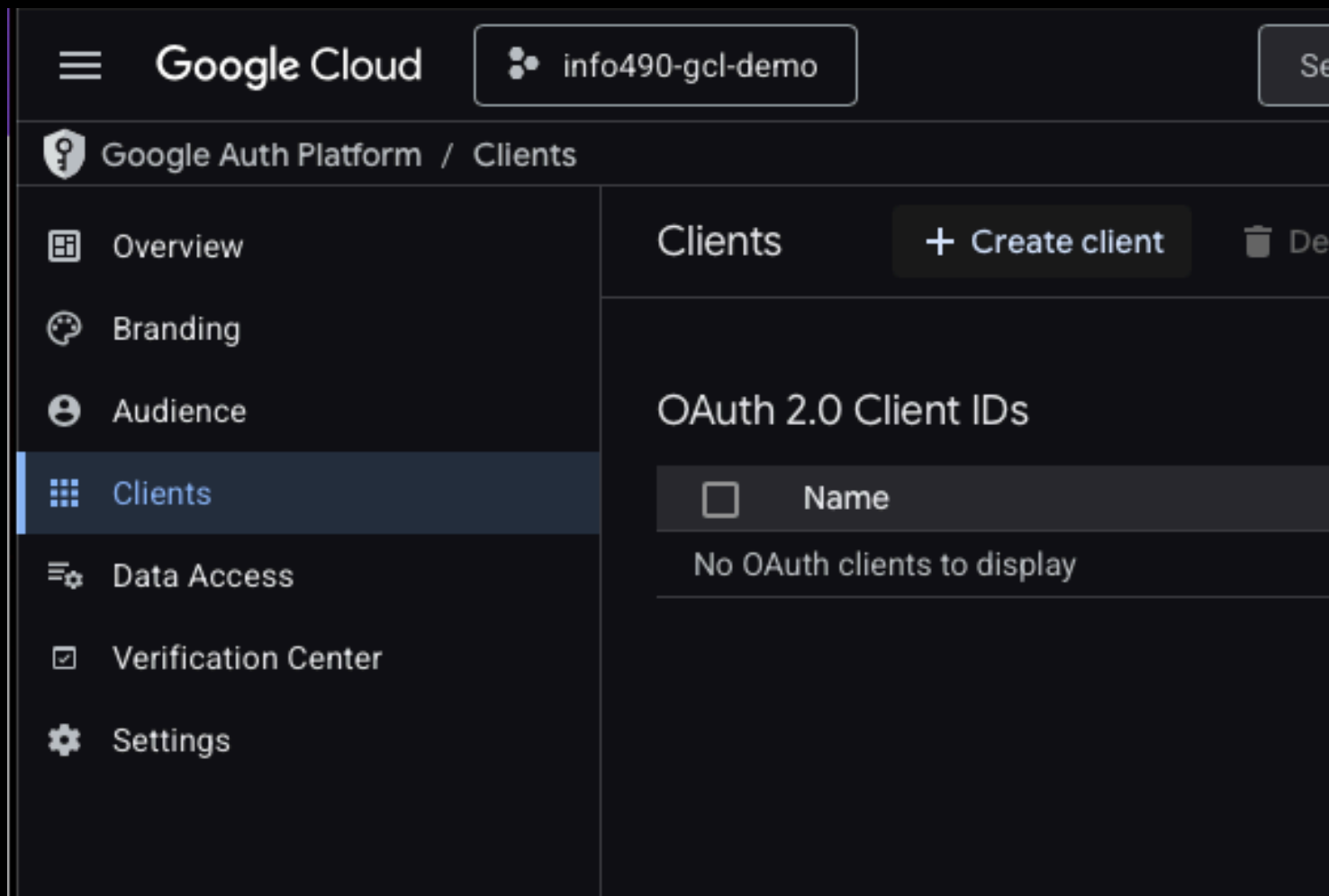
API ↑	Scope	User-facing description
No rows to display		

Save

Discard changes

Step 4.7:

- Click on “Clients” > “Create client”
- Then select application type as “Web application”



- Put any name to this Client ID.

Google Cloud

info490-gcl-demo

Search (/) for resources, docs, products, and

Google Auth Platform / Clients / Create client

Overview

Branding

Audience

Clients

Data Access

Verification Center

Settings

← Create OAuth client ID

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *
Web application

Name *
INFO490-GCL-DEMO-CREDENTIALS

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

- Add the URIs which will be having access to this Client-ID or login API.
- As we will be running this project locally as well, add the local host URIs as is.
- Make sure to enter these URIs as is.
- Then click “Create”

Authorized redirect URIs ?

For use with requests from a web server

URIs 1 *

http://127.0.0.1:8000

URIs 2 *

http://127.0.0.1:8000/accounts/google/login/callback/

+ Add URI

Note: It may take 5 minutes to a few hours for settings to take effect

Create

Cancel

- Download the JSON file of this OAuth Client and paste it in the project

The image shows a composite of three screenshots illustrating the process of downloading an OAuth client JSON file and placing it in a project.

Top Left: OAuth client created dialog

The dialog titled "OAuth client created" provides the following information:

- Client ID:** 312756766356-v1khpbe9gcovthdmi14t5hf2qbulkahq.apps.googleusercontent.com
- Client secret:** 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
- Status:** Enabled
- Action:** A "Download JSON" button is visible.

A red arrow points from the "Download JSON" button towards the file explorer.

Top Right: JSON content

```
{
  "web": {
    "client_id": "312756766356-v1khpbe9gcovthdmi14t5hf2qbulkahq.apps.googleusercontent.com",
    "project_id": "info490-gcl-demo",
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token",
    "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
    "client_secret": "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",
    "redirect_uris": [
      "http://127.0.0.1:8000",
      "http://127.0.0.1:8000/accounts/google/login/callback/"
    ]
  }
}
```

Bottom: Project Explorer

The screenshot shows a file explorer window for a project named "illinois.edu". The file structure includes:

- 0_class_notes
 - Week 1
 - Week 2
 - Week 3
 - Week 4
 - Week 5
- 1. Google Cloud Project Creation
 - 5_1_1.txt
 - client_secret_312756766356-v1khpbe9gcovthdmi14t5hf2qbulkahq.apps.googleusercontent.com.json
- data
- illinois

The file "client_secret_312756766356-v1khpbe9gcovthdmi14t5hf2qbulkahq.apps.googleusercontent.com.json" is highlighted, indicating it is the file to be pasted into the project.

4.b

Django Side Integration

Django-allauth

- We will be using Django-allauth for integrating the Google OAuth Client in our website.
- Do visit it's comprehensive documentation for guide to integrating more OAuths
 - <https://docs.allauth.org/en/latest/>

Step 4.8:

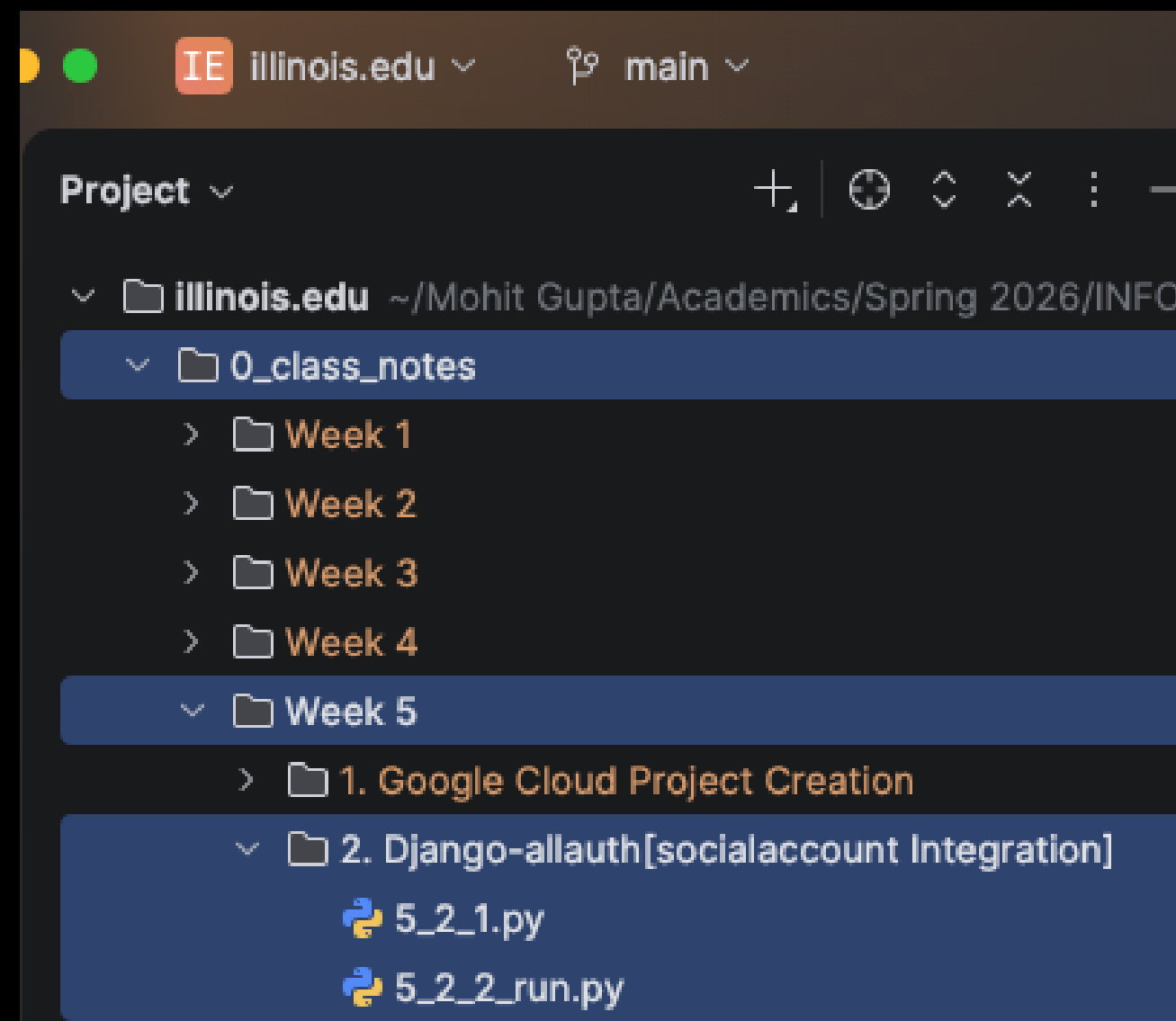
- Install the package:
 - `pip install "django-allauth[socialaccount]"`
 - Make sure to use `"` to enclose the name or else you'll get the following error:

```
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % pip install django-allauth[socialaccount]
zsh: no matches found: django-allauth[socialaccount]

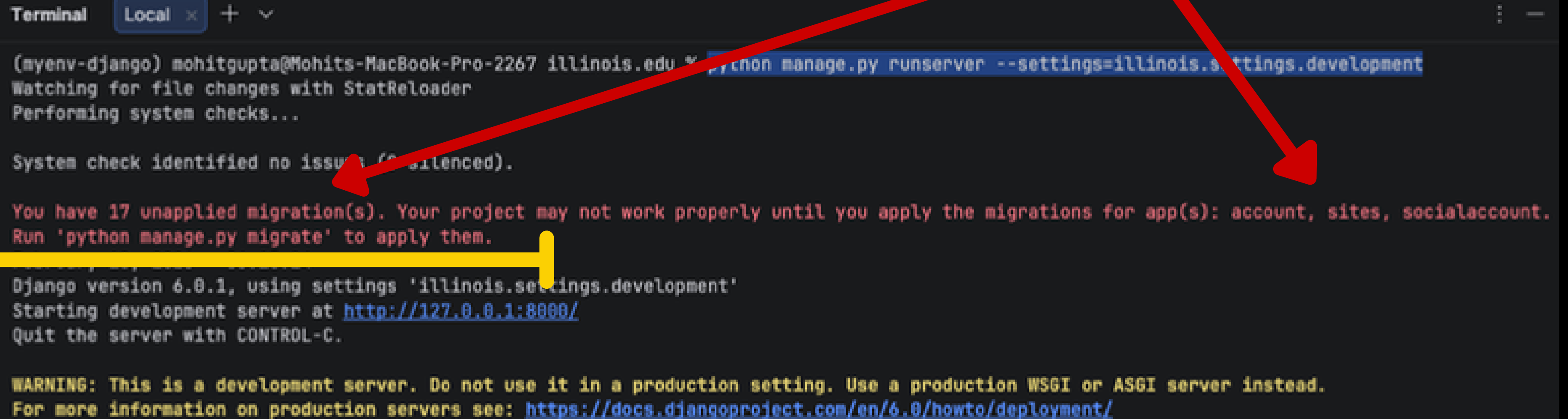
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % pip install "django-allauth[socialaccount]"
Collecting django-allauth[socialaccount]
  Downloading django_allauth-65.14.3-py3-none-any.whl.metadata (9.2 kB)
Requirement already satisfied: Django>=4.2.16 in /Users/mohitgupta/anaconda3/envs/myenv-django/lib/python3.13/site-packages (from django-allauth[socialaccount])
Requirement already satisfied: asgiref>=3.8.1 in /Users/mohitgupta/anaconda3/envs/myenv-django/lib/python3.13/site-packages (from django-allauth[socialaccount])
Collecting oauthlib<4.0, >=3.3.0 (from django-allauth[socialaccount])
```


Step 4.9:

- Now copy and paste the files **5_2_1.py** and **5_2_2.py** to their designated locations.



- When you run the development server, you'll see "17 unapplied migrations"
- But we did not add these new migrations to our table, right?



The image shows a terminal window with the following output:

```
Terminal Local x + v
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % python manage.py runserver --settings=illinois.settings.development
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 17 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): account, sites, socialaccount.
Run 'python manage.py migrate' to apply them.

Django version 6.0.1, using settings 'illinois.settings.development'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
```

Annotations in the image include:

- A red arrow pointing from the text "17 unapplied migrations" in the list above to the line "You have 17 unapplied migration(s)..." in the terminal.
- A red arrow pointing from the text "But we did not add these new migrations to our table, right?" in the list above to the line "Run 'python manage.py migrate' to apply them." in the terminal.
- A yellow bracket highlighting the entire message block starting with "You have 17 unapplied migration(s)..." and ending with "Quit the server with CONTROL-C."

- I now see new tables as well as their columns upon running “showmigrations”.
- Let’s apply these migrations using “migrate”

```
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % python manage.py showmigrations --settings=illinois.settings.development
account
[ ] 0001_initial
[ ] 0002_email_max_length
[ ] 0003_alter_emailaddress_create_unique_verified_email
[ ] 0004_alter_emailaddress_drop_unique_email
[ ] 0005_emailaddress_idx_upper_email
[ ] 0006_emailaddress_lower
[ ] 0007_emailaddress_idx_email
[ ] 0008_emailaddress_unique_primary_email_fixup
[ ] 0009_emailaddress_unique_primary_email
```

```
sites
[ ] 0001_initial
[ ] 0002_alter_domain_unique
socialaccount
[ ] 0001_initial
[ ] 0002_token_max_lengths
[ ] 0003_extra_data_default_dict
[ ] 0004_app_provider_id_settings
[ ] 0005_socialtoken_nullable_app
[ ] 0006_alter_socialaccount_extra_data
students
[X] 0001_initial
[X] 0002_alter_enrollment_options_alter_section_options_and_more
```


- All are now applied.
- Let's check our admin panel.

```
(myenv-django) mohitgupta@Mohits-MacBook-Pro-2267 illinois.edu % python manage.py migrate --settings=illinois.settings.development
Operations to perform:
  Apply all migrations: account, admin, auth, contenttypes, sessions, sites, socialaccount, students
Running migrations:
  Applying account.0001_initial... OK
  Applying account.0002_email_max_length... OK
  Applying account.0003_alter_emailaddress_create_unique_verified_email... OK
  Applying account.0004_alter_emailaddress_drop_unique_email... OK
  Applying account.0005_emailaddress_idx_upper_email... OK
  Applying account.0006_emailaddress_lower... OK
  Applying account.0007_emailaddress_idx_email... OK
  Applying account.0008_emailaddress_unique_primary_email_fixup... OK
  Applying account.0009_emailaddress_unique_primary_email... OK
  Applying sites.0001_initial... OK
  Applying sites.0002_alter_domain_unique... OK
  Applying socialaccount.0001_initial... OK
  Applying socialaccount.0002_token_max_lengths... OK
  Applying socialaccount.0003_extra_data_default_dict... OK
  Applying socialaccount.0004_app_provider_id_settings... OK
  Applying socialaccount.0005_socialtoken_nullable_app... OK
  Applying socialaccount.0006_alter_socialaccount_extra_data... OK
```

- New Tables

- Open "Sites"

Django administration

WELCOME, MOHITG2. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Site administration

ACCOUNTS

Email addresses

+ Add

Change

AUTHENTICATION AND AUTHORIZATION

Groups

+ Add

Change

Users

+ Add

Change

SITES

Sites

+ Add

Change

SOCIAL ACCOUNTS

Social accounts

+ Add

Change

Social application tokens

+ Add

Change

Social applications

+ Add

Change

STUDENTS

Enrollments

+ Add

Change

Sections

+ Add

Change

Students

+ Add

Change

Recent actions

My actions

Change

 mohitg2
User

Change

 mohitg27
User

Step 4.10:

- Click the first entry to edit.
- Change the domain & display names to:
 - http://127.0.0.1:8000

Select site to change

Search

Action:

↕

Run

 0 of 1 selected

☐	DOMAIN NAME	▲	DISPLAY NAME
☐	example.com		example.com

1 site

Change site

example.com

Domain name:

example.com

Display name:

example.com

SAVE

Save and add another

Save and continue editing

- In base.py, SITE_ID = 1
 - SITE_ID is the column name
 - 1 indicates the domain name at index 1.

```
> data
  illinois
    settings
      __init__.py
      base.py
      developmen
```

```
33
34 # New Changes
35 # 1 represents
36 # where the G
37 SITE_ID = 1
38
```

Change site

example.com

Domain name:

Display name:

✓ The site "http://127.0.0.1:8000" was changed successfully.

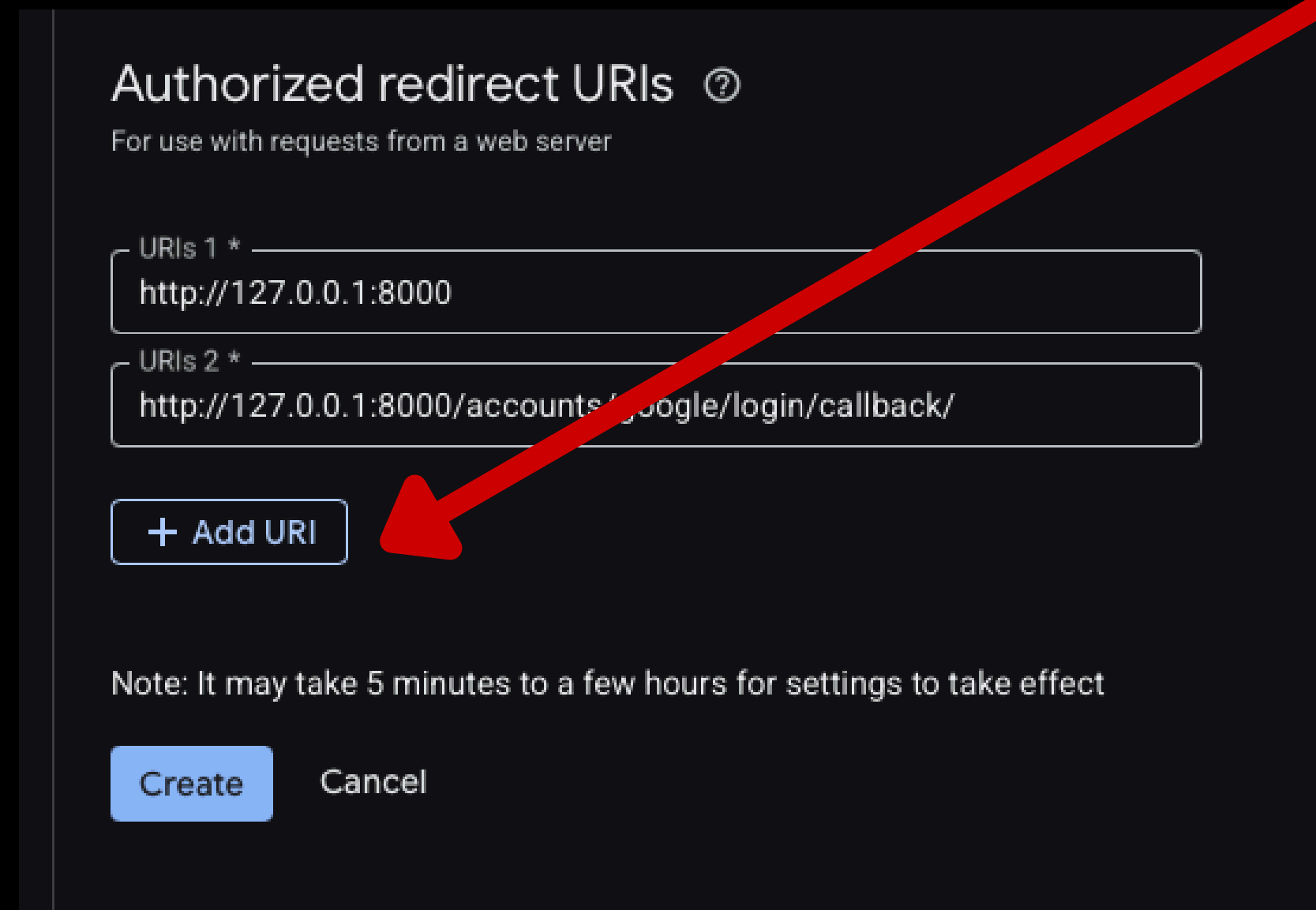
Select site to change

Action: 0 of 1 selected

☐	DOMAIN NAME	DISPLAY NAME
☐	http://127.0.0.1:8000	http://127.0.0.1:8000

1 site

- In future, if you add a new domain name, you can use the OAuth client on that website as well.
- All you need to do is:
 - Add the domain where you'd be integrating the OAuth
 - In the Google Console, also add the new domain name which would be sending the request to the Google Login OAuth Client. (Check step 4.7)



Authorized redirect URIs ⓘ
For use with requests from a web server

URIs 1 *
http://127.0.0.1:8000

URIs 2 *
http://127.0.0.1:8000/accounts/google/login/callback/

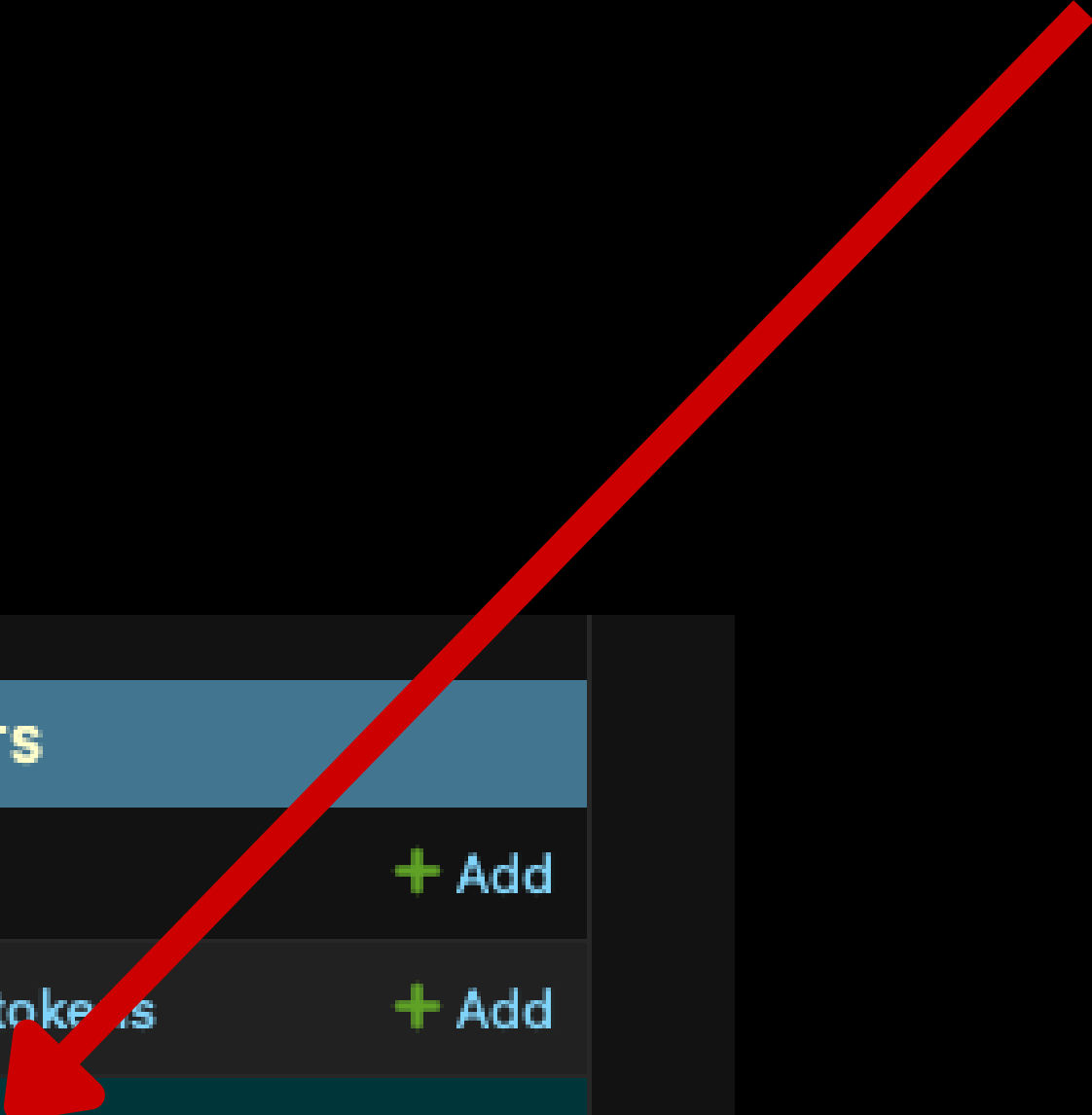
+ Add URI

Note: It may take 5 minutes to a few hours for settings to take effect

Create Cancel

Step 4.11:

- Now in the “Social applications” table, add fill the JSON details that we downloaded from Google Console.



SOCIAL ACCOUNTS	
Social accounts	+ Add
Social application tokens	+ Add
Social applications	+ Add

- Copy these 2 details as is.

```
client_secret_312756766356 x + Gemini
File /Users/mohitgupta/Downloads/client_secret_3127567663...
Google Chrome isn't your default browser Set as default
Pretty-print
{
  "web": {
    "client_id": "312756766356-v1khpbe9gcovthdmi14t5hf2qbulkahq.apps.googleusercontent.com",
    "project_id": "info490-gcl-demo",
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token",
    "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
    "client_secret": "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",
    "redirect_uris": [
      "http://127.0.0.1:8000",
      "http://127.0.0.1:8000/accounts/google/login/callback/"
    ]
  }
}
```

Home > Social Accounts > Social applications > Add social application

Start typing to filter...

ACCOUNTS

Email addresses + Add

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

Sites +

SOCIAL ACCOUNTS

Social accounts + Add

Social application tokens + Add

Social applications + Add

STUDENTS

Enrollments + Add

Sections + Add

Students + Add

Add social application

Provider: Google

Provider ID:

Name: Google

Client id: 312756766356-v1khpbe9gcovthdmi14t5hf2qbulkahq.apps.googleusercontent.com
App ID, or consumer key

Secret key: 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
API secret, client secret, or consumer secret

Key:

Settings: {}

Sites:

Available sites

Choose sites by selecting them and then select the "Choose" arrow button.

Filter

Choose all sites

Chosen sites

Remove sites by selecting them and then select the "Remove" arrow button.

Filter

http://127.0.0.1:8000

Remove all sites

Hold down "Control", or "Command" on a Mac, to select more than one.

SAVE Save and add another Save and continue editing

- Also, make sure to select the website for which you are adding these credentials.

✔ The social application "Google" was added successfully.

Select social application to change

Action: 0 of 1 selected

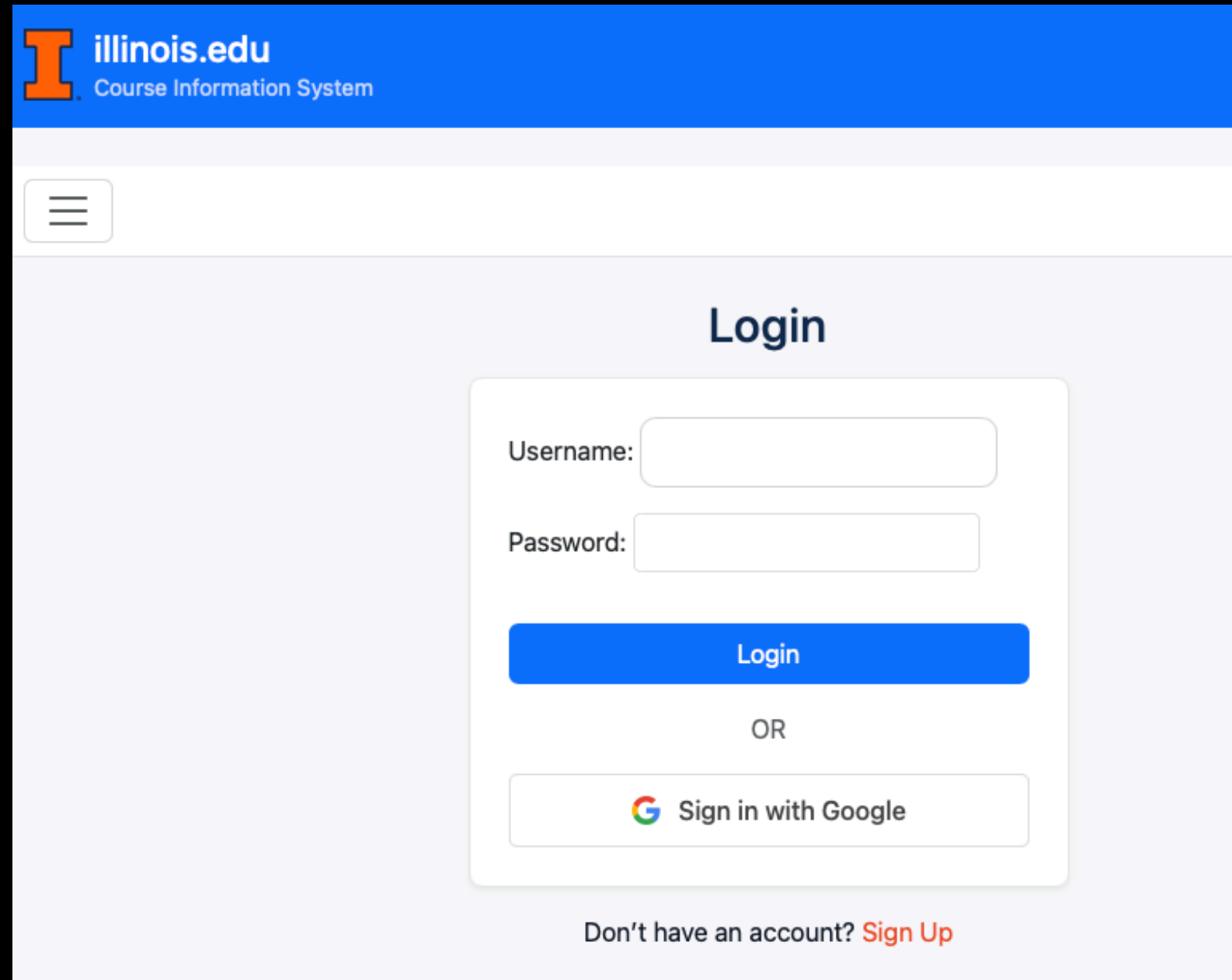
☐	NAME	PROVIDER
☐	Google	google

1 social application

4.c

Adding UI for Google Button

- **Desired output**



The screenshot displays the login interface for the Illinois.edu Course Information System. At the top, a blue header bar contains the orange 'I' logo, the text 'illinois.edu', and 'Course Information System'. Below the header is a white navigation bar with a hamburger menu icon on the left. The main content area has a light gray background and features a central white login box with a gray border. The box is titled 'Login' in bold. It contains two input fields: 'Username:' and 'Password:'. Below these fields is a blue 'Login' button. Under the button is the text 'OR'. At the bottom of the box is a button with the Google logo and the text 'Sign in with Google'. Below the login box, the text 'Don't have an account?' is followed by a red 'Sign Up' link.

illinois.edu
Course Information System

≡

Login

Username:

Password:

Login

OR

 Sign in with Google

Don't have an account? [Sign Up](#)

- But currently, we just have a login and “Sign Up” button.
- Let’s integrate the Google OAuth Button.

The screenshot shows the login page of the illinois.edu Course Information System. The page has a blue header with the university logo and name on the left, and a 'Log In' button on the right. Below the header, there is a 'Log In' link. The main content area features a 'Login' title and a form with 'Username:' and 'Password:' labels, each followed by an input field. A blue 'Login' button is positioned below the input fields. Underneath the button, there is a link that says 'Don't have an account? Sign Up'. The footer contains text about the project's use of open-source frameworks and the Creative Commons license.

illinois.edu
Course Information System

Log In

Log In

Username:

Password:

Login

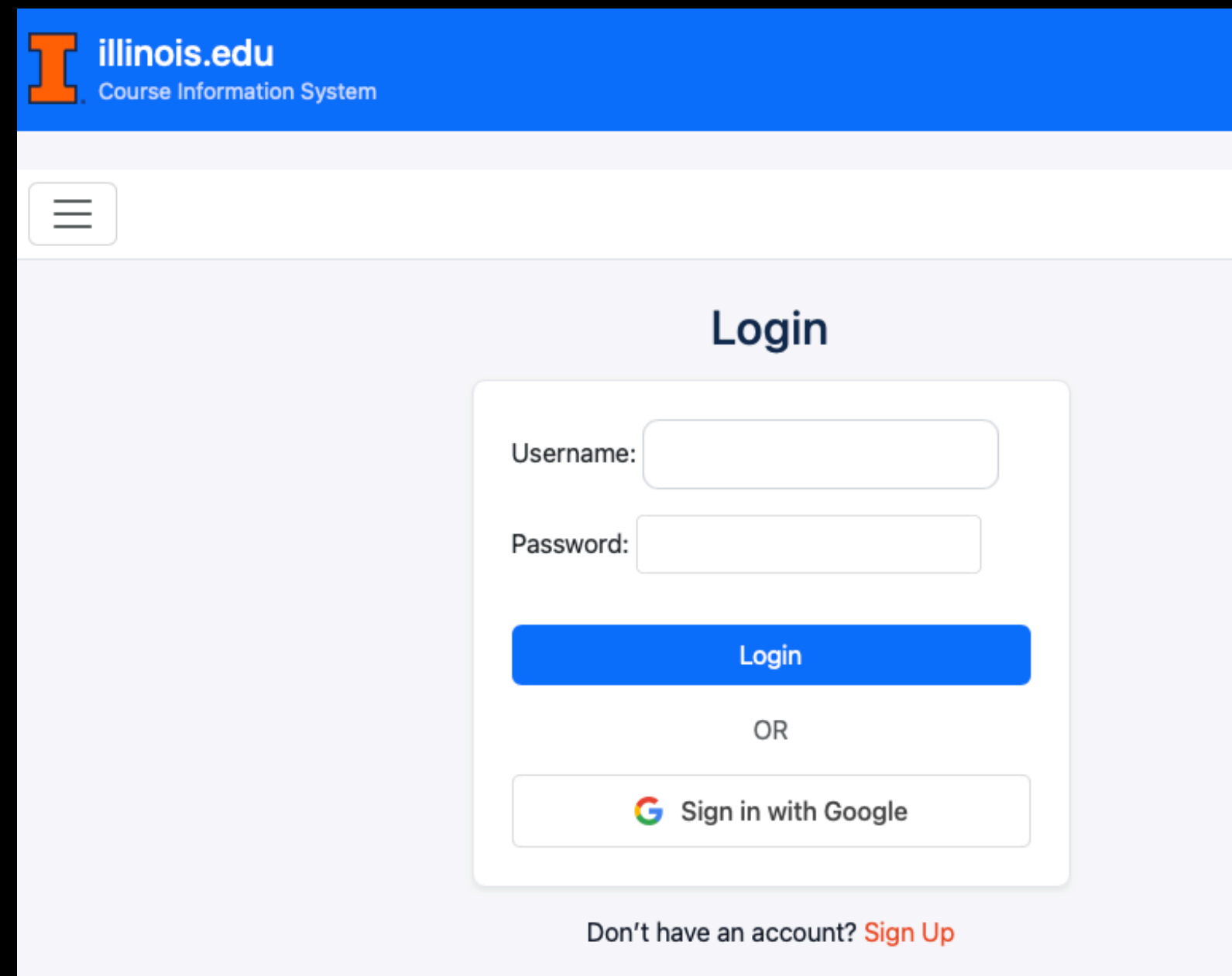
Don't have an account? [Sign Up](#)

illinois.edu class demonstration

Portions of this project make use of open-source frameworks: Skeleton CSS by Dave Pinkham and Normalize.css by Nicolas Gallagher , both under the MIT License.

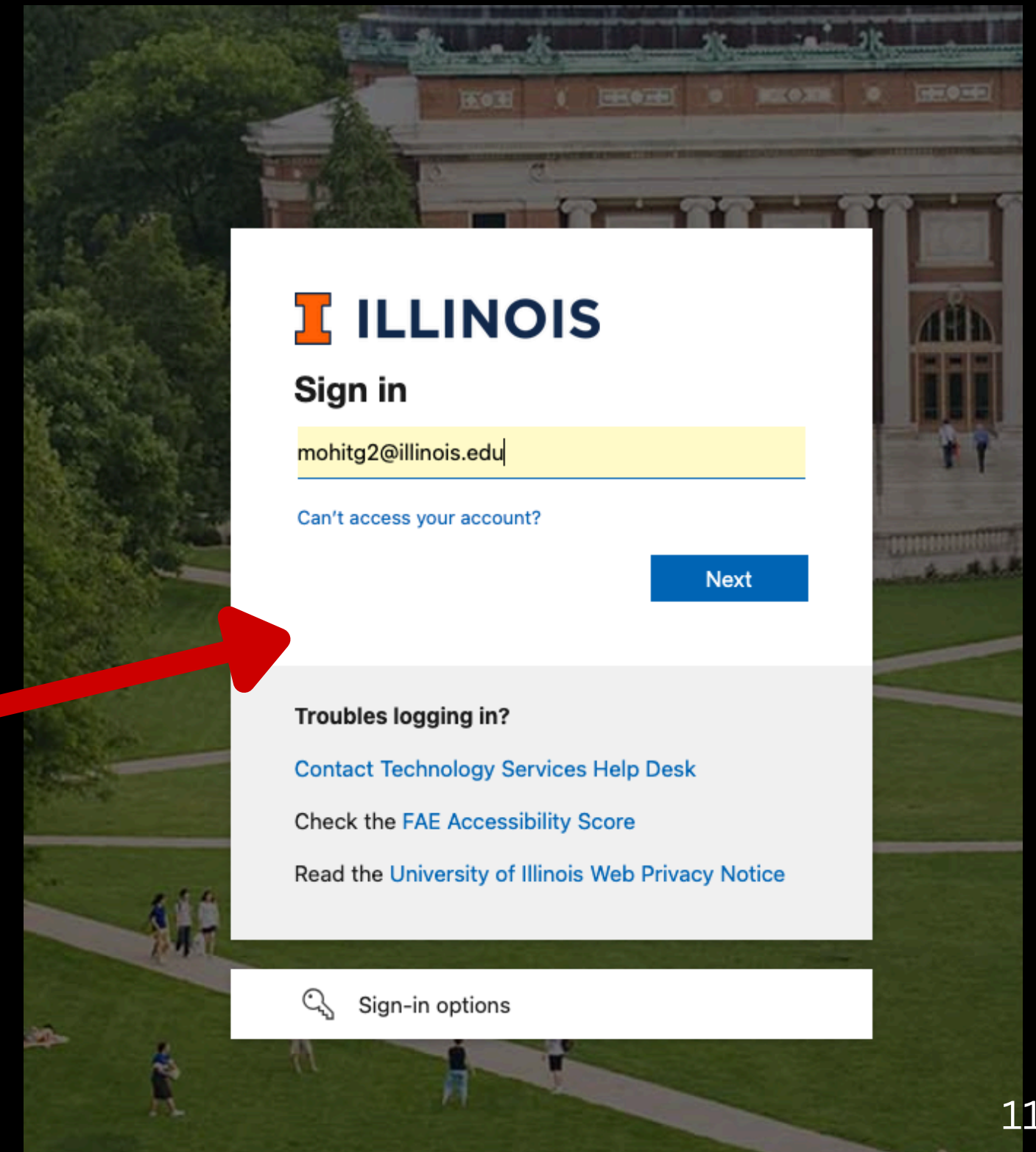
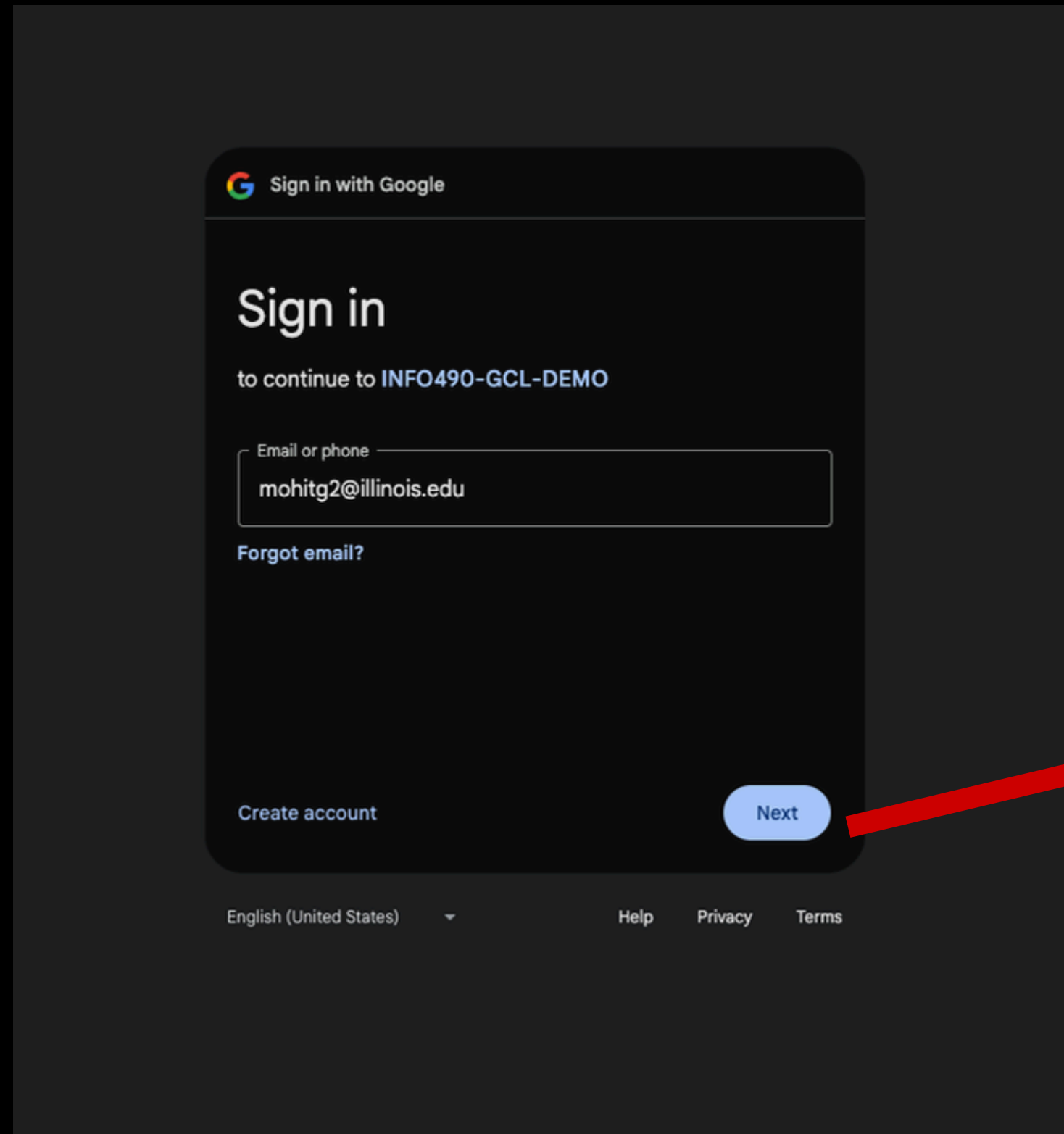
Course content licensed under a Creative Commons Attribution-ShareAlike 4.0 International License .

- **Step 4.12:**
 - **Now copy and integrate files “5_3_1.html” to “5_3_3.css” in their designated directories.**
 - **After you do that, our login screen will look like this:**



The screenshot shows the login interface for the Illinois.edu Course Information System. At the top, there is a blue header with the Illinois logo and the text "illinois.edu Course Information System". Below the header is a white navigation bar with a hamburger menu icon. The main content area is light blue and features a "Login" heading. Underneath the heading is a white login form with two input fields: "Username:" and "Password:". Below these fields is a blue "Login" button. Under the button is the text "OR". Below "OR" is a button with the Google logo and the text "Sign in with Google". At the bottom of the form, there is a link that says "Don't have an account? Sign Up".

- Upon clicking the “Sign in with Google,” you will be taken to the login screen:



 Sign in with Google

Sign in to INFO490-GCL-DEMO

 mohitg2@illinois.edu ▾

Google will allow INFO490-GCL-DEMO to access this info about you

 Mohit Gupta
Name and profile picture

Review INFO490-GCL-DEMO's Privacy Policy and Terms of Service to understand how INFO490-GCL-DEMO will process and protect your data.

To make changes at any time, go to your [Google Account](#).

Learn more about [Sign in with Google](#).

Cancel

Continue

English (United States) ▾

[Help](#)

[Privacy](#)

[Terms](#)

Student List

http://127.0.0.1:8000/student/

Sign in

 **illinois.edu**
Course Information System

Log Out, mohit

Data ▾Forms ▾APIs & Export ▾Charts ▾Reports

Student List

Adams, Maya

Bennett, Xavier

Brown, Jack

Clark, Noah

Cruz, Yara

Previous

Page 1 of 6

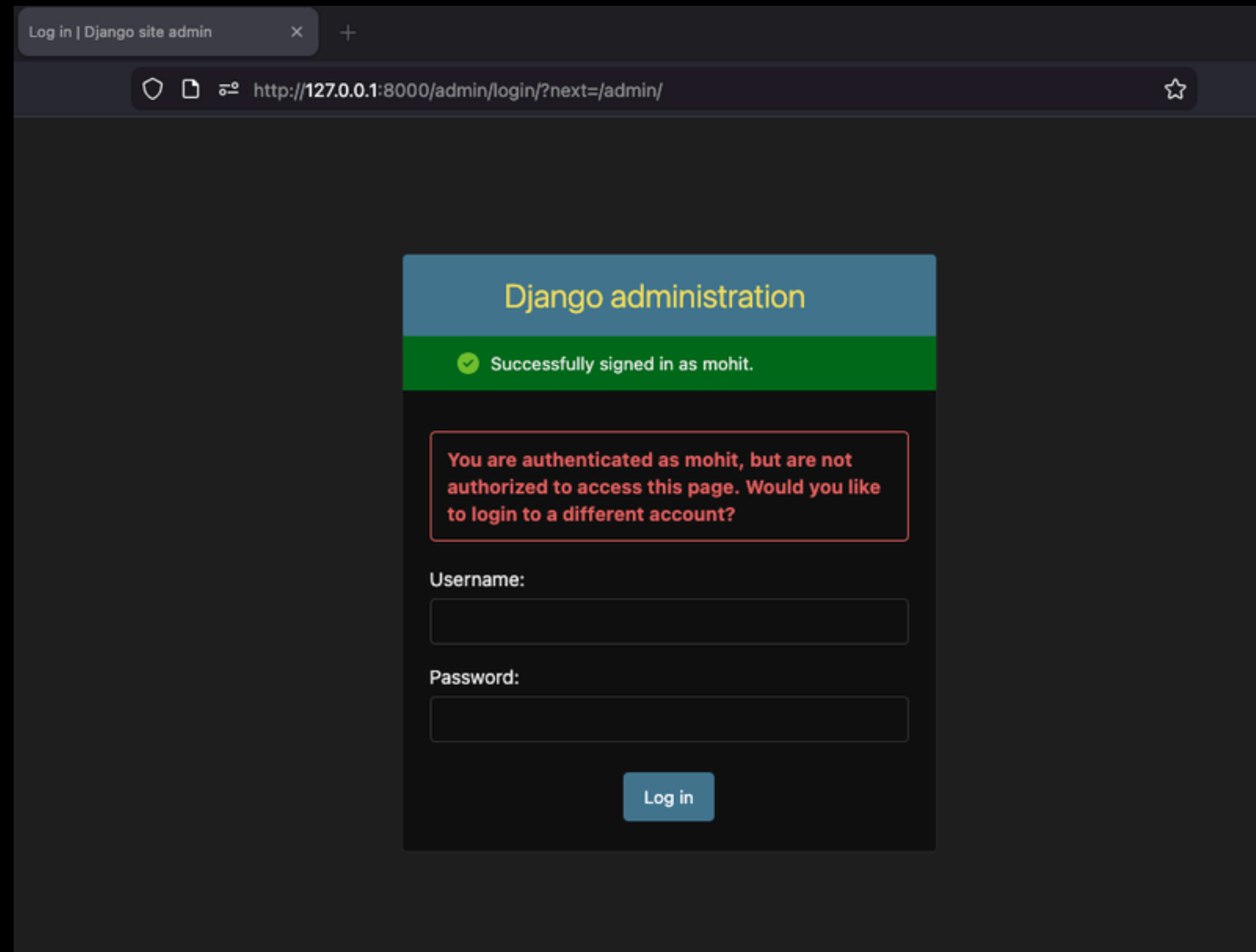
Next

None

Search

result

- **No staff access.**



- “Users” table

Users

Select user to change

Search

Action:

Run

 1 of 2 selected

	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
	mohit	-	Mohit	Gupta	
	mohitg2	mohitg2@illinois.edu	-	-	

2 users

- “Social accounts” table

Social accounts

+ Add

THORIZATION

+ Add

+ Add

Select social account to change

Search

Action:

Run

1 of 1 selected

	USER	UID	PROVIDER
	mohit	111891305594628235106	google

1 social account

- “Social accounts” table

Social accounts › mohit

+ Add

HORIZONTALIZATION

+ Add

+ Add

+ Add

+ Add

+ Add

Change social account

mohit

User:3🔍 mohit

Provider:google

Uid:111891305594628235106

Extra data:

{
 "iss": "https://accounts.google.com",
 "azp": "312756766356-v1khpbe9gcovthdmi14t5hf2qbulkahq.apps.googleusercontent.com",
 "aud": "312756766356-v1khpbe9gcovthdmi14t5hf2qbulkahq.apps.googleusercontent.com",
 "sub": "111891305594628235106",
 "at_hash": "tEfXsl-bdWi4m0kRYqQ0CA",
 "name": "Mohit Gupta",
 "picture": "https://lh3.googleusercontent.com/a/ACg8ocLaYZ0jWHTJZT8s-FA8cQBxZJn_hDeh9yflPKhfXozSevevww4=s96-c",
 "given_name": "Mohit",
 "family_name": "Gupta",
 "iat": 1771399943,
 "exp": 1771403543
}

Z

🔍 https://lh3.googleusercontent.com/a/ACg8ocLaYZ0jWHTJZT8s-FA8cQBxZJn_hDeh9yflPKhfXozSevevww4=s96-c

ACg8ocLaYZ0jWHTJZT8s-FA8cQBxZJn_hDeh9yflPKhfXozSevevww4=s96-c 96x96 pixels



**That's all for
today**

**And that
marks the
end of
Phase-1!**